Synthetic Training Data Construction for Coding Agents

Yiqing Xie

October 23, 2026

Language Technologies Institute
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

Thesis Committee:
Carolyn Rosé (Chair)
Daniel Fried
Graham Neubig
Dan Roth

Submitted in partial fulfillment of the requirements for the degree of Doctor of Philosophy.

Copyright © 2026 Yiqing Xie

Keywords: synthetic data, coding agent, code generation

Abstract

With the development of AI techniques, the role of AI in software development is shifting from script-level, static code generation to project-level coding agents. This dissertation research focuses on the fundamental challenges of training coding agents: the high cost of constructing training data, the difficulty of handling diverse use cases simutaneously, and the performance saturation with respect to data scale. To this end, this thesis is structured around three parts: (1) constructing scalable training data, (2) improving the generalizability of coding agents, and (3) training critic models for coding agents.

One major bottleneck of scaling coding agent training data is the construction of training environments, which are abstractions of real-world coding setups. Building the environments can require substantial human or agent effort, for example, when the environments support the executability and unit testing of an entire code repository. In our work, we aim to find an appropriate level of abstraction for training environments, with the goal of simplifying their construction while preserving training effectiveness.

Building on this lightweight environment construction process, we extend coding agents' abilities to diverse downstream coding tasks and conduct systematic analyses on the generalizability of coding agents across different tasks, which is still under-explored in existing research on both single-task post-training or coding agent pre-training. In particular, we develop principles for designing synthetic training tasks and characterize atomic abilities that transfer across coding tasks.

Even with these advances in coding agent training, empirically, we observe that for a specific model, additional training data yields little or no further gain after a certain point. Our analysis further shows that the performance is substantially bottlenecked by the limited high-level planning ability: producing intermediate steps about what should be done and in what order, as opposed to low-level implementation: generating concrete executable actions to perform the intermediate steps. In our work, we aim to train an additional critic model that specializes in planning and dynamically provides guidance to the coding agent in inference.

Together, this thesis aims to build scalable data to train effective coding agents and critic models that generalize across diverse coding tasks.

Contents

1 Introduction **1**
1.1 Thesis Overview . . . 3
1.2 Proposed Timeline . . . 5

2 Literature Review **6**
2.1 Coding Agent Overview . . . 6
2.1.1 Benchmarks for Coding Agents . . . 6
2.1.2 Coding Agent Infrastructure . . . 7
2.1.3 Training Paradigms for Coding Agent . . . 8
2.2 Scalable Coding Agent Post-Training . . . 8
2.3 Coding Agent Pre-Training . . . 9
2.4 Improving the High-level Planning Ability of Coding Agents . . . 10

I Training Coding Agents with Scalable Synthetic Data **12**

3 Constructing Scalable Code Translation Training Data with Comparable Corpora and Multiple References **14**
3.1 Introduction . . . 14
3.2 Methodology . . . 17
3.2.1 Problem Formulation . . . 17
3.2.2 Training on Comparable Corpora . . . 18
3.2.3 Finetuning with Multiple References . . . 20
3.3 Results . . . 20
3.3.1 Experimental Setup . . . 21
3.3.2 Main results . . . 22
3.3.3 Performance Analysis . . . 23
3.3.4 Influence of Comparable Corpora . . . 24

3.3.5 Influence of Multiple References . . . 26
3.3.6 Hyper-parameter analysis . . . 27
3.3.7 Case Studies . . . 27
3.4 Chapter Summary . . . 28

4 Constructing Scalable Coding Agent Training Environments with Sandbox Testing 29
4.1 Introduction . . . 29
4.2 Methodology . . . 31
4.2.1 Repository and Function Curation . . . 32
4.2.2 Sandboxing: Key to Environment Setup . . . 32
4.2.3 Test Generation . . . 34
4.2.4 Quality Control and Filtering . . . 35
4.3 Resulting Datasets: REPOST-TRAIN and REPOST-EVAL . . . 36
4.3.1 Quality Verification with Human Study . . . 37
4.4 Code Generation Training Experiments . . . 38
4.4.1 Experimental Setup . . . 39
4.4.2 Experimental Results . . . 40
4.5 Coding Agent Training Experiments . . . 41
4.5.1 Experimental Setup . . . 41
4.5.2 Experimental Results . . . 42
4.6 Chapter Summary . . . 43

II Analyses and Principles: What Makes Coding Agent Training Effective? 44

5 Constructing Effective Synthetic Tasks for Training Coding Agents 47
5.1 Introduction . . . 47
5.2 Methodology . . . 49
5.2.1 Analysis of Coding Agent Task Transferability . . . 50
5.2.2 Principles for Training Task Design . . . 52
5.2.3 HYBRID-GYM Tasks . . . 52
5.2.4 Verification of HYBRID-GYM Tasks . . . 54
5.3 Results . . . 55
5.3.1 Experimental Setup . . . 55
5.3.2 Main Results . . . 56

5.3.3 Detailed Results . . . 58
5.4 Analysis of Task Transferability . . . 59
5.4.1 Output Format Must Match Downstream Tasks . . . 59
5.4.2 Script-level Agentic Tasks Do NOT Generalize to Repo-level Tasks . . . 60
5.4.3 Task and Trajectory Complexity Matters . . . 60
5.4.4 Even Successful Trajectories of the Same Task Instances may have Different Impact . . . 61
5.4.5 Sampling Strategies Affects Effectiveness . . . 62
5.5 Chapter Summary . . . 62

6 Discovering Effective Demonstration Behaviors for Training Coding Agents 64
6.1 Introduction . . . 64
6.2 Analysis Method: Discovering Influential Behaviors of Demonstrations . . . 67
6.2.1 **CATRadar**: Automatic Candidate Behavior Discovery . . . 67
6.2.2 Data Intervention Methods . . . 68
6.3 Analysis Results: What Behaviors Affect Coding Agent Training? . . . 70
6.3.1 **CATRadar** Discovers Diverse Candidates . . . 70
6.3.2 Trajectory-Level Features are Not Good Indicators of Training Effectiveness 71
6.3.3 **CATRadar** Discovers Effective Problem-Solving Strategies . . . 72
6.3.4 Planning Steps have Different Effects on Different Models . . . 73
6.3.5 Verification Abilities Transfer from Simple Tasks to Complex Tasks . . . 74
6.3.6 Training on Reasoning Text Increases Trajectory Length . . . 74
6.4 Practical Utility of Our Discoveries . . . 75
6.4.1 Method 1: Behavior-Aware Data Selection . . . 75
6.4.2 Method 2: Adding Synthesized Agent Steps to Training Trajectories . . . 76
6.4.3 Method 3: Behavior-Conditioned Demonstration Generation . . . 77
6.5 Conclusions . . . 77

III Training Auxiliary Models to Assist Coding Agents 79

7 Improving Generator Factuality by Training a Critic Model 81
7.1 Introduction . . . 81
7.2 Methodology . . . 83
7.2.1 FenCE: Factuality Evaluator Training . . . 83
7.2.2 Improving Generator Factuality with FenCE . . . 85

7.3 Experiments on Evaluator Training . . . 87
7.3.1 Experimental Setup . . . 87
7.3.2 Main Results . . . 87
7.3.3 Result Analyses . . . 88
7.4 Experiments on Generator Factuality . . . 89
7.4.1 Experimental Setup . . . 89
7.4.2 Main Results . . . 91
7.4.3 Result Analyses . . . 93
7.5 Chapter Summary . . . 94

8 Improving Coding Agent Effectiveness by Training a Critic Model 98
8.1 Introduction . . . 98
8.2 Methodology . . . 100
8.2.1 Formal Definitions . . . 100
8.2.2 Designing the Critique Structure . . . 101
8.2.3 Distilling Critic Models with SFT . . . 102
8.2.4 Improving Critic Models with DPO . . . 103
8.3 Experiments . . . 105
8.3.1 Experimental Setup . . . 105
8.3.2 Implementation Details . . . 107
8.3.3 Main Results . . . 107
8.3.4 Detailed Ablations and Analyses . . . 107
8.3.5 Qualitative Analysis and Case Studies . . . 109
8.4 Conclusion and Future Works . . . 111

9 Conclusion 113

IV Appendices 115

A Appendix for Chapter 6 116
A.1 Additional Related Work . . . 116
A.2 Experimental Setup . . . 116
A.3 Computation of Significance . . . 121
A.4 List of Candidate Behaviors . . . 121
A.5 Detailed Analysis Results . . . 121
A.5.1 Exploration Length is Not a Good Indicator of Trajectory Effectiveness . 121

A.5.2 Strong Teacher Does Not Always Translate to Strong Student 121
A.5.3 Non-Tool-Call Text Increases Trajectory Length 122
A.6 Concrete Data Intervention Implementation for each Candidate Behavior 122
A.6.1 Label Demonstrations with the Presence of Repo-Exploration Strategies 123
A.6.2 Prompt the Models to Adopt our Repo-Exploration Strategies 124
A.6.3 Add Task-Tracker Steps to Training Demonstrations 125
A.6.4 Add Verification Steps to Training Demonstrations 125
A.6.5 Verbosity Experiments 127

B Appendix for Chapter 8 **129**
B.1 Analyses on the Gap between Teacher and Student Critic Models 129
B.2 Training and Inference Details 129
B.2.1 Inference Details 129
B.2.2 Training Details 130
B.3 Method Preliminaries 131
B.4 Critic Prompts 132
B.4.1 High-Level Prompt 132
B.4.2 Detailed Prompt 134
B.4.3 Budget-Aware Instructions 137
B.5 Opus Teacher Prompt Ablation 137
B.6 Inference Pricing 137
B.7 Statistical Significance 138
B.8 Qualitative Case Studies 138
B.8.1 Untrained vs. trained critic on the same loop 138
B.8.2 Routing around a dependency failure 140
B.8.3 Backseat driving in the detailed teacher prompt 140
B.8.4 Repeated demand on `django__django-13012` 141

Bibliography **143**

List of Figures

1.1 The timeline of how the paradigm of the collaboration between human and AI coding assistants shifts from script-level static code generation to project-level coding agent. 1

3.1 The standard pipeline for code translation (colored in light grey) and the CMTrans pipeline (colored in pink). The comparable corpora are both naturally occurring and model generated. We generate multiple references by CMTrans. 15

3.2 An example of parallel and comparable data. Parallel examples are line-by-line aligned. Programs in a comparable example may have different algorithms and structures (e.g., global code vs. class in this case), but may still contain lines that can be matched, as highlighted in pink, blue, and yellow. 15

3.3 Perplexity of validation set during finetuning. 16

3.4 An example of CMTrans for Java-to-Python translation. We compare the pipeline of CMTrans (*red*) to the standard pipeline (*grey*) of code translation (e.g., fine-tuned CodeT5, Wang et al. 110) and the self-supervision-and-fine-tuning method (*purple*) of TransCoder-ST (Roziere et al. 94). 18

3.5 Translation results with different amount of parallel data. We mark the relative gain of CMTrans over CodeT5. 24

3.6 Number of unique correct translations in 20 candidates for each test example. We use beam search for each method, so the generated candidates are guaranteed to be distinct. 26

3.7 Effect of the size of comparable corpora. 26

3.8 Effect of the max number of references for each example. 27

3.9 Case study: a comparable example generated by our method with an NL-to-code generation model. We highlight the lines in the Java program and the generated Python program that can be matched. 28

4.1 We can use the coding environments built by REPOST for training and evaluation. We first apply the code generation model to generate candidate solutions with the original repository as context. Then we evaluate the solutions by executing the evaluation script built by REPOST. For evaluation, we directly compute Pass@k scores. For training, we add all successful solutions to the train set and further finetune the model. 30

4.2 The REPOST coding environment construction framework. We sandbox the target function and its dependencies to a separate evaluation script for execution, which avoids building the entire repository. We design careful quality control strategies with iterative quality improvement and post-filtering. The outcome of REPOST is a set of executable repo-level coding environments, which can be used for training and evaluation. 32

4.3 An example where the LLM successfully creates a mock class, `Mock_API`, to replace real external API calls. In this way, while the function body of the target function `API_call` remains exactly the same as in the original codebase, it can be executed without making real API calls. 33

4.4 (a) Pass@1 scores on RepoEval with different numbers of training examples. (b) Pass@1 scores on RepoEval with different methods to sample 2,000 training examples. Sample-by-Example has a broader repository coverage and achieves better Pass@1. The performance is further enhanced with RFT (Distill). 41

5.1 (***Left***) We decompose general coding agent tasks into a set of intermediate components and compute the percentage of agent actions spent on each component. Our training tasks partially cover verification and fully cover reasoning, repository exploration, and implementation, which consist of around 68% of the actions. (***Right***) Example actions for each component. Compared to a baseline training method, SWE-Gym [84], training with our HYBRID-GYM significantly reduces failures due to insufficient reasoning, insufficient exploration, and failed file editing, improving the final resolved rate on SWE-Bench Verified from 10.6% to 15.0%. 48

5.2 The error analysis of different methods on 50 instances sampled from SWE-Bench Verified. 50

5.3 Scaling law analysis. Performance on SWE-bench Verified improves consistently as training data size increases from around 5% (250 trajectories) to 100% (4.4k trajectories). 56

5.4 Controlled experiments on training data characteristics. (a) **Output Format:** Removing the file editing actions (`str_replace`) from function localization trajectories causes a large drop in SWE-bench resolution rate. (b) **Repo-Exploration:** script-level code generation (LCB) does not effectively transfer to repo-level issue-solving and even underperforms documentation generation, a simple repo-level task. (c) **Task Complexity:** Transfer improves as training tasks become more complex. (d) **Trajectory Complexity:** With a fixed data size, training on longer (more agent steps) trajectories substantially improves downstream performance. 59
5.5 Effect of teacher model and data selection. (a) Effect of **sampling strategy** at fixed training budget. Repository diversity improves training but using the same repositories as in evaluation does not. (b) training on issue localization trajectories with different **teacher models**. o3-mini (edited) indicates the same set of trajectories as o3-mini, but with text and action steps combined. 61

6.1 To find demonstration behaviors that affect the effectiveness of training, (*left*) prior works rely on human experts to manually make hypotheses, while (*right*) **CATRadar** narrows down the search space by automatically discovering candidate behaviors by extracting and comparing the behaviors of different trained agents. 65
6.2 Across different training sets, on the same sets of task instances, successful demonstrations generated by different teacher models lead to substantially different performance, and higher teacher performance does not necessarily transfer to better student performance. 66
6.3 Overview of the **CATRadar** automatic behavior discovery framework (§6.2.1). We run two trained coding agents on the same task instances, use a discovery agent to propose behavioral differences with a well-defined judge metric, and verify the distinguishability of each behavior by fitting a logistic regression. . . 67
6.4 Student performance (Qwen2.5Coder-7B) is not significantly correlated with trajectory-level features such as teacher success rate or trajectory length. 70
6.5 Planning steps in demonstrations have different effects on different models. We **highlight** significant performance changes after removing all task-tracker actions from demonstrations (with p-value < 0.05). 72
6.6 When the verbosity of reasoning text increases (including non-tool-call text and content of the "think" actions), the trajectory length in general increases, and the performance first increases and then decreases. 73

7.1 (*Left*) The framework to train an evaluator, FENCE, by augmenting public datasets with textual critiques and more diverse knowledge sources. We show the details in Figure 7.2. (*Right*) The framework to improve model factuality with FENCE. We construct training data by leveraging FENCE to revise and score the generator's responses. Details of response revision are shown in Figure 7.3. . . 82

7.2 Framework of evaluator training. (*left*) Existing public datasets for evaluator training. Each example contains a claim, a source document, and the ground truth (GT) label of whether the claim is supported by the document. (*Right*) We augment the datasets with textual critiques and more diverse source documents obtained by tools. We use zero-shot Llama-3-70B-chat as the instruction-tuned model in our experiments. 83

7.3 The framework to revise model responses without introducing lesser-known facts. We iteratively (1) use FENCE to evaluate the factuality of each claim, (2) replace false information (if any) with the correct fact or remove it from the response, depending on whether it corresponds to a lesser-known fact, and (3) continue generating the next passage. For every claim, we prompt the generator "*Is this claim factual?*" without providing any source documents. If the generator outputs "*unknown*", we assume that the claim corresponds to a lesser-known fact. 85

7.4 Statistics of generated responses, where we group the prompts by the popularity of the person to write biography for, as labeled in the FActScore dataset. We compare zero-shot Llama3-8B-chat (denoted as "*Base Model*"), SFT + FENCE ("*SFT*"), and our method, Edit/Remove + Coarse ("*DPO*"). 92

7.5 The factuality of claims with different topics, where we pre-define the topics and use Llama3-70B-chat to predict whether each claim covers any topic(s). . . 92

7.6 Hyper-parameter analysis. We investigate how percentage of facts changes (a) as number of iterations to revise candidate responses increases, and (b) with different numbers of preferred responses for each prompt. 93

8.1 Inference setup. A trained critic model C provides guidance to a frozen coding agent π_A for every k steps. The agent is responsible for all concrete actions and the critic model inspects the current trajectory and generates high-level feedback for its next steps. 99

8.2 SFT training pipeline for critic models. Based on our analysis (§8.2.2), the critic contains error detection, budget planning, and only provides high-level feedback instead of concrete actions. We run the frozen coding agent π_A on disjoint training tasks and query the teacher critic C_T every k steps. We collect (trajectory prefix, feedback) pairs (z_t, f_t^*) to train a critic C_θ by supervised fine-tuning (SFT). 102
8.3 DPO training pipeline for critic models. Given a critic model checkpoint $C_{SFT,\theta}$, for every k steps produced by the coding agent π_A, we use $C_{SFT,\theta}$ to generate $N = 10$ critiques, and prompt an LLM judge model R to select the best and worse critique, which forms (chosen, rejected) pairs (f_c, f_r) to train $C_{SFT,\theta}$ by direct preference optimization (DPO). We only keep a (f_c, f_r) pair if the chosen critique is both correct and provides clear instruction and the rejected critique misses any of the criteria. 104
8.4 Cost versus performance on SWE-Bench Verified across six coding agents. Each agent shows its base (●) and the 8B SFT critic (★); the two Qwen agents additionally show the `Claude-Opus-4.6` teacher critic (○). The cost is for the agent **plus** the critic model. 110

A.1 The Hybrid-Gym function localization prompt. `module_type` is `function` or `class`. 117
A.2 The R2E-Gym prompt [41], used by some of the R2E-Gym rollouts. 118
A.3 The issue-solving prompt used to evaluate the students on SWE-Bench Verified and used in some of the R2E-Gym rollouts. 120
A.4 The exploration length of demonstrations is also not a good indicator of demonstration effectiveness. 121
A.5 When we increase the verbosity of non-tool call text, the trajectory length in general increases. However, the performance does not change much. 124
A.6 The strategy blocks added to the teacher's function localization prompt (first box; `module_type` is `function` or `class`) and to the student's issue-solving prompt (second box). 126

B.1 Error categorization results. 130
B.2 Guidance quality analysis. 131

List of Tables

3.1 Number of problems per dataset. "Gen-comp" is the comparable corpora dataset we generate. C-to-J, C-to-P, ... denote the test examples of C++-to-Java, C++-to-Python translation, etc. * denotes data we build. 20

3.2 Java-Python translation results on TransCoder-test. We copy the results of all the baselines reported by Ahmad et al. [2]. CB and CA@1 stand for CodeBLEU and Computational Accuracy. We highlight the **best** results under each metric with Bold and underline the second-best results. 21

3.3 Java-C++ and Python-C++ translation results on TransCoder-test. The CA@1 results of TransCoder and TransCoder-ST are copied from the TransCoder-ST paper. We evaluate BLEU and CodeBLEU using their released checkpoints. We finetune and evaluate CodeT5 and TransCoder-ST-ft on our own. 21

3.4 Ablation studies of CMTrans. We report CA@1 results. J-to-P and P-to-J stand for Java-to-Python and Python-to-Java results. We put the experiments on each type of comparable corpora in an individual block. 23

3.5 The overlap between the types of constructs in the translation outputs and the ground truth translations. 24

3.6 Syntax Accuracy (SA) on TransCoder-test before finetuning, which evaluates whether the program can be compiled without syntax errors. 25

4.1 Statistics of REPOST-TRAIN and REPOST-EVAL compared to existing **execution-based** code generation datasets. R2E (Reproduced) applies the R2E method to the same set of input repositories as REPOST-TRAIN, but results in a smaller number of repositories and examples. "Repo?" and "Auto Test?" refer to the repo-level setting and automatically generated tests. 36

4.2 Detailed statistics of our datasets. 37

4.3 Agreement between human and GPT-4o on checking (1) the functionality equivalence between the sandboxed and original function, and (2) the test correctness. When GPT-4o predicts "Yes" for both quality checks, it has a high agreement with human. 37

4.4 Human study results. We ask the participants to complete the function with the same context we use to evaluate code generation models in §4.4. 38

4.5 Code generation training results. We evaluate Pass@1 for all experiments. For RepoEval, we use the "Oracle" repo-level context as used in their original paper. 39

4.6 Performance of the Qwen2.5Coder-7B agent on SWE-Bench Verified, SWT-Bench Verified, and Commit-0 Lite before and after training. 42

4.7 Performance of the Qwen2.5Coder-7B agent on SWE-Bench Verified after training with R2E and REPOST-TRAIN. For fair comparison, for both methods, we build function generation training data from the same set of repositories on GitHub. 43

5.1 The use of Linux commands for repository exploration in our proposed training tasks (left) and downstream tasks (right). We show the percentage of each command used in the agent steps targeting repository exploration. 49

5.2 We categorize the agent's actions based on the intermediate components of the task, where the categories are pre-defined. 49

5.3 The average number of OpenHands tools called in each trajectory. 53

5.4 Performance of the student and teacher models on a subset of instances sampled from each training task. 53

5.5 Statistics of HYBRID-GYM and its subsets. Following SWE-Smith, we report the average per-instance cost of setting up the training environment for rollout. Compared to existing datasets, HYBRID-GYM covers more repositories and requires only 2 docker images to build all training instances. 55

5.6 Results on three real-world coding tasks. We show whether the datasets contain swe, swt, cmt, or synthetic tasks (syn) in training. Results with a white background are **task transfer** results. Results with a light grey background are trained with in-domain tasks. We highlight the best result for each dataset and show the relative gain and loss compared to the base model, Qwen2.5Coder. HYBRID-GYM achieves strong task transfer performances across all three benchmarks and further improves in-domain datasets (e.g., SWE-Play). 57

5.7 Performance of training Qwen2.5Coder-7B on each of the HYBRID-GYM tasks and the issue-solving task. We sample 500 instances from each of our tasks. Even without finetuning on issue-solving, function localization achieves a larger performance gain than SWE-Gym, which has 491 issue-solving instances. We report the resolved rates on SWE-Bench Verified, SWT-Bench Verified, and Commit-0 Lite. 58

6.1 Some of the behaviors discovered and examined by our framework. We present the full list of candidate behaviors in §A.4. We use "✓" to indicate that the behavior is shown to be relevant to the final performance for some student models in related works and use "U" to indicate that the behavior is tested but not shown to be relevant. "LEGO" [132], "Hybrid" [123], and "Others" refer to existing works. 69

6.2 A case study showing that demonstrations with different repo-exploration strategies can all be labeled as "sufficient exploration" by the LLM judge. 71

6.3 Verification procedure in a simple training task (e.g., function localization) transfers to more complex tasks (e.g., issue-solving). However, the effect of verification steps depends on the student and teacher model. We highlight significant yellowapplegreen!25yellowyellow[redacted] and yellowred!16yellowyellow [redacted] in resolved rates (with p-value < 0.05). 72

6.4 Data selection results with the repo-exploration strategies we discovered. Parenthesized results are absolute gains compared to the lower resolved rate of the two ablations. We highlight gains for resolved rates that are significantly better than yellowapplegreen!25yellowyellow[redacted] and yellowapplegreen!10yellowyellow[redacted] (with p-value < 0.05). 75

6.5 Student performance (Qwen2.5Coder-7B) on SWE-Bench Verified under different teacher models and prompting strategies. We highlight yellowapplegreen!25yellowyellow[redacted]. 76

6.6 We can improve performance by adding synthetic task-planning steps in training trajectories. We highlight yellowapplegreen!25yellowyellow[redacted]. 76

7.1 Performance (BAcc) of evaluator models on the test split of LLM-AggreFact. We separate Llama3-8B-chat based models, larger open-source models, and proprietary models into different blocks. We highlight the best-performing open-source model for each dataset. Results with † are reported in Tang et al. [99]. 88
7.2 The accuracy of the critiques and the tool-extracted source documents we obtained. We randomly sampled 15 examples for each predicted label and manually check the accuracy of each example. 89
7.3 Comparison between our method and baselines on FActScore and TruthfulQA. "E/R" stands for "Edit/Remove". All baselines use the zero-shot models as the evaluator in training and our method uses FENCE. 90
7.4 Ablation study that compares different training recipes when equipped with FENCE as the evaluator. "SFT + FENCE", "Edit", and "Coarse" denote equipping "SFT", "EVER-Pref", and "FactTune-FS" in Table 7.3 with the FENCE evaluator. 91
7.5 An example of the textual critique we generate. We highlight the most relevant sentence in the document. In this case, our generated critique aligns with the judgment label. 95
7.6 An example where we obtain multiple relevant source documents by calling a multiplicity of tools. 96
7.7 An example where we obtain multiple relevant source documents by calling a multiplicity of tools. The tool-extracted documents all contain the information about Chadwick's birthday, but have diverse formats. 97

8.1 Results on SWE-Bench Verified. We report the better result from $k \in \{5, 10\}$ for every configuration. *Non-Loop %* is the fraction of trajectories that do *not* repeat a Bash command in three consecutive steps. "+ SFT" is a critic trained on teacher critiques generated with the high-level prompt (a `CWM-32B` + `Qwen3-Next-80B-A3B` mix). "+ SFT + DPO" further optimizes the 4B SFT critic with DPO on critique preference pairs. All arrows compare to "No Critic" setting. **Bold** highlights the best non-Opus value in each column. 106
8.2 Resolve rate (%) on 300 Multi-SWE-Bench instances. The 8B CWM+Qwen critic uses k=5 and is trained only on Python trajectories. Arrows show gains over the "No Critic" baseline. 108
8.3 Resolved rate (%) of critic models trained with different SFT corpus on SWE-Bench Verified. All critic models use k=5; arrows show gains over no critic. . . 108

8.4 Resolve rate (%) on SWE-Bench Verified for 8B critic models trained on SFT data with the detailed critiques vs. the high-level teacher critiques (our data). . 109
8.5 `Claude-Opus-4.6` critic model backseat driving on the 50 SWE-Bench Verified mini examples. Avg. Critique Lines denotes the average number of lines in the critiques. Copy% denotes the fraction of lines in the submitted code patch that are directly copied from the critic model. $\geq 0.30\%$ counts the instances where more than 30% of the coding agent's final submission lines are directly copied from the critic model. We test critique frequencies of $k = 5$ and 10. 111

A.1 The candidate behaviors discovered by **CATRADAR** that we do not conduct further experiments on. 122
A.2 Demonstrations of Claude-Opus-4.7 and Claude-Opus-4.5 on the Hybrid-Gym instance `sopel-irc__sopel-1344_101` (both successful). `<repo>` abbreviates the repository path, and file paths are shown relative to it. Claude-Opus-4.7 opens the target file directly from a listing cut at 50 files, while Claude-Opus-4.5 takes a more comprehensive strategy: picking several candidate files with detailed reasoning, and then reading them one-by-one (action 3). 123

B.1 `Claude-Opus-4.6` teacher prompt comparison on SWE-Bench Verified (500 instances). *Non-Loop* is the percentage of trajectories without three consecutive repetitions of the same Bash command. Total cost is per-instance agent-plus-critic cost. Bold marks the better percentage metric within each agent; ties are unbolded. 137
B.2 Hosted-inference rates used to convert token counts to the dollar costs in this paper. Opus critic costs include prompt-caching: cache-write tokens billed at $1.25\times$ input ($6.875/Mtok) and cache-read at $0.10\times$ input ($0.55/Mtok); the reported Opus cost combines these. 138
B.3 Statistical significance of the headline resolve-rate gains in Table 8.1 on SWE-Bench Verified. Parentheses show percentage-point gains over no critic; 95% CIs are paired McNemar–Wald intervals on these gains. All critic runs use $k = 5$. Significance markers: $^{***}p < .001$, $^{**}p < .01$, and $^{*}p < .05$. 138

Chapter 1

Introduction

With the rapid advancement of AI techniques, the role of AI in software development is evolving from script-level, static code generation toward project-level coding agents. Unlike traditional code assistants that are lmited to producing isolated snippets of code, coding agent infra-structures such as SWE-Agent [128] and OpenHands [106] provide tools for the LLM to understand broader repository context, track dependencies across files, develop multi-step development plans, etc. This enables them to assist with more complex tasks such as feature implementation, refactoring, debugging, testing, and maintenance of the entire codebase.

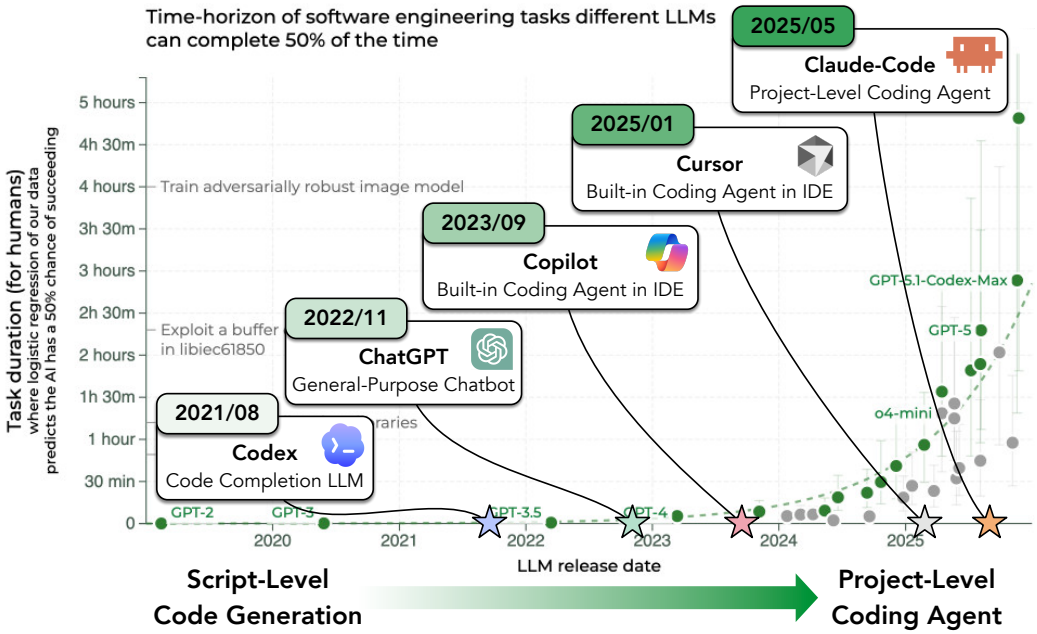


Figure 1.1: The timeline of how the paradigm of the collaboration between human and AI coding assistants shifts from script-level static code generation to project-level coding agent.

Although coding agents have shown strong performance on recent benchmarks that simulate real-world scenarios, there remains substantial room to improve both the effectiveness and feasibility of coding agent training. We identify several fundamental challenges as follows:

The High Cost of Constructing Training Data. Although the scale of training data has been shown to be a major factor in coding agent performance [130], constructing such data remains highly expensive. One major bottleneck is the construction of training environments that serve as abstractions of real-world coding setups, which typically involve docker environments with all required external packages installed, test suites for major features in a codebase, etc. Given the current state of LLM capacity, this process still demands substantial human or agent effort [130]. Such resources are particularly scarce in academia, where the engineering, computational, human, and financial resources are often limited, hindering the development of systematic research on coding agent training.

The Difficulty of Handling Diverse Use Cases Simutaneously. In real-world deployment, coding agents must operate across diverse tasks and scenarios, and can be paired with different tools, execution harnesses, and inference-time techniques. However, most existing open-sourced post-training datasets still focuses on training agents for a single task under a fixed inference setting and with a predefined set of tools, which does not reliably generalize to other settings [145]. What still remains underexplored is how well skills transfer across these dimensions and how such transferability can be improved. Such lack of understanding creates a significant gap between current coding agent research and practical real-world applications.

The Performance Saturation with Respect to Data Scale. Even if the first two challenges are addressed, empirically, we observe that for a specific model, especially the one with limited size (e.g., Qwen2.5Coder-7B and 14B [34]), increasing the size of training data yields little or no further gain after a certain point [41, 130]. To better understand this limitation, we break down the abilities required for coding agent training into high-level planning (producing intermediate steps about what should be done and in what order) and low-level implementation (generating concrete executable actions to perform the intermediate steps). An example of high-level planning is decomposing the function localization task into discovering the repository structure, localizing relevant files, and locating the relevant functions inside the candidate files. An example of low-level implementation is using the "file-editing" tool to generate code patches with no errors and retries. Our analysis suggests that high-level planning remains a major bottleneck: even after training, there is still a substantial gap between these models and frontier models such as Claude Sonnet 4.6 [6]. Although increasing the size of the base model can raise the performance ceiling, it also requires more computational resources, which

is particularly limited in academia.

In this thesis, we aim to mitigate the three challenges by (1) constructing scalable training data, (2) improving the generalizability of coding agents, and (3) training critic models to assist coding agents.

1.1 Thesis Overview

This thesis contains three parts targeting the three above challenges, respectively.

Part 1: Coding Agent Training with Scalable Synthetic Data. As abstractions of real-world coding setups, training environments are typically complex and require substantial human or agent effort to construct. In our work, we aim to find an appropriate level of abstraction for training environments. Specifically, inspired by our work that investigates the abstraction level of training data for code translation [119], which is published in **EMNLP Findings 2023**, we identify the execution support for the entire codebase to be the major bottleneck of training environment construction, and explore training settings that avoid code execution or only require execution for a particular module in the codebase. This simplifies the construction of training environments while preserving training effectiveness [120]. This work is published in **COLM 2025**.

Part 2: Training Coding Agents that Generalize across Tasks. Most existing open-sourced coding agent post-training datasets only focus on a single task, which does not reliably generalize to other tasks. In our work, we open a direction of training coding agents that generalize across tasks. Specifically, we conduct systematic analyses on the commonalities shared across diverse downstream coding tasks, and design synthetic training tasks that can generalize across tasks. The work has been accepted to **ICML 2026**. In the **proposed work**, we take a step forward and analyze the exact trajectory behaviors (including problem-solving strategies and concrete actions) that are transferrable across tasks. Then we propose a data selection method based on such desired behaviors beyond task success.

Part 3: Training Critic Models to Assist Coding Agent. While the first two parts aim to improve the effectiveness of coding agent training, empirically, we still observe that the performance satuates with respect to data scale after a certain point, and the high-level planning ability remains a major bottleneck. To address this limitation, in our work, we develop critic models that provide guidance to the coding agent at inference time. While the main coding agent is trained to jointly balance multiple abilities, including high-level planning and low-level implementation, the critic model is specialized for planning. Since the optimum for

jointly optimizing multiple abilities does not necessarily align with the optimum for planning, training a specialized critic model may lead to stronger planning performance and, in turn, more effective guidance for the coding agent. We first show that the high-level idea of "training a specialized model to provide guidance to the main model" can be beneficial in general NLP tasks such as factuality generation [121] in an **ACL 2025** paper. Then in the **proposed work**, we apply this idea to coding agent training and train critic models to assist coding agents in inference. We show that there is a large room of improvement for training critic models for coding agents, and propose a recipe to train critic models.

1.2 Proposed Timeline

- May 2026 — Thesis Proposal
- May 2026 – Jun 2026 — Submit Proposed Work I (chapter 8) to EMNLP 2026
- May 2026 – Sept 2026 — Submit Proposed Work II (chapter 6) to ICLR 2027
- Sept 2026 – Oct 2026 — Keep Exploring Proposed Work I (chapter 8)
- Oct 2026 – May 2027 — Thesis Writing and Job Search
- May 2027 — Thesis Defense

Chapter 2

Literature Review

The structure of this chapter is as follows: In §2.1, we introduce the overview of coding agent training. In §2.2, we discuss existing work that constructs scalable training data, and introduce how our work opens a direction of constructing scalable coding agent training data with lightweight environments. Since most coding agent training datasets only focus on a single task, to train coding agents that are capable across a diverse set of applications, in §2.3, we further discuss the generalizability of coding agents, where our work aims to reveal the key abilities that are transferrable across different coding tasks. While all the above works show effective ways to directly train coding agents, empirically, we still observe that the performance satuates with respect to data scale after a certain point, and the high-level planning ability remains a major bottleneck. To address this limitation, in §2.4, we discuss how existing work improves the planning ability by either indirect approaches such as optimizing the context, memory, tools, etc, or direct approaches that explicitly train a planner agent. Our work opens a direction of critic model training, which falls into the direct approach. The critic model is explicitly optimized to tackle the planning bottleneck, and can be integrated with a fixed coding agent without retraining it.

2.1 Coding Agent Overview

2.1.1 Benchmarks for Coding Agents

Coding agents have shown strong performance on recent repository-level benchmarks. The SWE-Bench issue-solving benchmark [44] evaluates the ability to resolve a GitHub issue in a real-world repository by generating a code patch. Following SWE-Bench, SWE-bench Pro [18] is a more challenging issue-solving benchmark that captures realistic, complex, enterprise-

level problems beyond the scope of SWE-Bench. Similarly, SWE-bench Multimodal [129] further targets visual, user-facing software domains and includes multimodal context (e.g., images) in issue-solving. Multi-SWE-bench [135] extends the issue-resolution setting to repositories to nine different programming languages.

Beyond issue-solving benchmarks, SWT-Bench [71] and TestGenEval [38] target test generation, where the goal is to generate a set of test functions for a GitHub issue. Commit-0 [143] evaluates the ability to generate the entire library from scratch. The results will be evaluated by the number of tests passed. ArtifactsBench [137] assesses the transformation from multimodal instructions into high-quality and interactive visual artifacts. SWE-Perf [31] and SWE-fficiency [65] evaluate the ability to improve code efficiency (i.e., execution time) in addition to correctness. KernelBench [81] targets the generation of GPU kernels. SUPER [10], EnvBench [22], and TerminalBench [67] evaluate the agent's ability to run bash commands for setup and execution tasks. Multiple benchmarks are focusing on function generation [19, 40, 53, 54, 120, 139, 141]. Given the description of the function, the task is to complete the function body. All these benchmarks are designed to simulate real-world use cases for coding agents and require repository exploration and patch-style edits.

2.1.2 Coding Agent Infrastructure

Approaches to solving the above benchmarks include workflow-based pipelines and agent frameworks. Workflow-based systems explicitly structure the solution process into stages, as in Agentless [116]. In comparison, coding agents can dynamically call tools to interact with the environment, enabling end-to-end problem solving.

Agent infrastructure such as SWE-Agent [128] and OpenHands [106] provide agent-computer interfaces for tool use and editing. SWE-Agent introduces the concept of agent-computer interface: a structured set of interactions for searching/navigating code, viewing and editing files, and executing commands/tests, all with careful context management. Building upon SWE-agent, mini-SWE-Agent [128] achieves similar performance with a simpler control flow. Live-SWE-agent explores online self-evolution of SWE agents [118]. SWE-Search [8] improves SWE-Agent using planning (e.g., Monte Carlo Tree Search) with iterative refinement. Compared to SWE-Agent, OpenHands [106] provides a wider range of tools that allow the agent to interact with the world in similar ways to those of a human developer: by writing code, interacting with a command line, and browsing the web.

2.1.3 Training Paradigms for Coding Agent

To train coding agents, SWE-Gym [84] adopts rejection sampling finetuning, which applies a teacher model to generate trajectories, evaluates the correctness, and finetunes the student model on the successful trajectories. SWE-RL [113] trains the agent model with reinforcement learning, with the similarity score between ground-truth and LLM-generated solutions as reward. A follow-up work, ML-Agent [62], extends SWE-RL with exploration-enriched fine-tuning, step-wise RL, and an agentic ML-specific reward module.

2.2 Scalable Coding Agent Post-Training

We focus on the training of end-to-end coding agents in this section. As shown in multiple existing works [41, 84, 130], the performance of coding agents significantly improves when we scale up the training data, especially for base models with larger sizes. For instance, SWE-Smith show that the performance of coding agents with Qwen2.5Coder-32B [34] as the base model is still not converged when we have around 5k training trajectories.

Towards this challenge, existing works focus on reducing the effort of constructing training data. SWE-Gym [84] presents the first set of training environments for coding agents. The authors manually construct training environments using the same approach as SWE-Bench [44]. Each training environment contains a GitHub issue, the ground truth code patch to resolve it, and the corresponding GitHub snapshot with all required packages installed, which is equipped with the test function for the issue. The resulting dataset has 491 training trajectories. Following SWE-Gym, to reduce the effort of constructing training environments, R2E-Gym [41] improves the scalability of executable training environments by using test-generation and back-translation directly from commits. Although it still requires human effort to set up the repositories, it reduces the reliance on human-written issues or unit tests and thereby improves the training data scale to 3.3k trajectories. A concurrent work, SWE-Smith [130] generates scalable executable environments at a scale by automatically synthesizing bugs, which obtains 5.0k training trajectories. The authors also reduce the number of dockers in the training dataset by keeping only one docker image for each repository. The only part that requires human effort is the installation of repositories, where the authors explicitly mention that it is necessary to have human agent collaboration to ensure installation success rate. SWE-Play [145] reduces human effort with a completely automatic approach. It uses a coding agent to propose training tasks, build the repositories from scratch, implement the tasks, and generate tests to verify the correctness. Although it avoids human effort, the environment construction process is still challenging and expensive for agents to complete,

resulting a dataset with only 704 training trajectories.

There is still a large room of improvement for constructing scalable training data. Particularly, existing works either still require human effort in repository setup or test generation, or suffer from low success rates with a fully automatic approach. One potential reason is that they use the same executability setting in training and in downstream evaluation. Specifically, they require the repository to be executable, which encourages code execution and verification in training trajectories and enables unit-testing to select successful trajectories. In our work, we investigate: to what extent can training environments without executable repositories still be effective for coding agent training? We will answer this question by analyzing the effect of executability in coding tasks and constructing unit-testing without executable repositories.

2.3 Coding Agent Pre-Training

In addition to the open-sourced coding agent post-training datasets introduced in section 2.2, another line of work focuses on coding agent pre-training, with coding tasks as a subset of training tasks. Although such methods may be trained on massive user requests for coding tasks, they typically do not reveal explicit training details or conduct systematic analysis on the generalizability across different coding tasks.

In particular, Claude-3.5-Sonnet [3] is trained on an internal tool set to write, edit, and execute code. GPT-4.5 [74] is trained on using command-line tools and Python to modify files within the codebase. Devstral [91] trains the model to solve real GitHub issues and runs over multiple agent infrastructures. Kimi-K2 [100] is trained on large-scale agentic data synthesis by generating tool-use demonstrations in the combination of simulated and real environments. Qwen3-Coder [126] applies long-horizon "Agent RL" to encourage multi-turn tool interactions, with large parallel environment infrastructure. DeepSeek-V3.2 [17] uses with massive agent training data synthesis, with more than 1.8k environments and 85k instructions, where the code-agent environments are mined from GitHub issue-PR pairs with automated environment setup and JUnit test validation. MiniMax-M2.5 [69] is trained with RL in hundreds of thousands of complex real-world environments, covering more than 10 languages and 200k real environments. GLM-5 [133] constructs long-context data for coding agent training. Kimi K2.5 [101] improves the parallelism of RL environments and introduces a self-directed agent swarm paradigm. Recent work such as Claude-Opus-4.7 [7] and GPT-5.3-Codex [79] focus on enhancing the safety of the models in coding environments, such as the robustness against prompt injection attacks.

What is missing in both single-task post-training (section 2.2) and coding agent pre-training

(section 2.3) is the systematic analysis on the generalizability across different coding tasks. For instance, it is still under-explored on what conditions the coding agent abilities are transferrable across different coding tasks, and what exact abilities are shared across different tasks. In our work, we investigate the principles to design training tasks that transfer effectively across a broad range of coding agent tasks. In a proposed work, we further analyze the key agent behaviors that benefit the transferability across multiple coding tasks.

2.4 Improving the High-level Planning Ability of Coding Agents

To improve the coding agent's planning ability, one line of work focuses on indirect approaches such as optimizing the context, memory, tools, etc. Reflexion [96] adds reflective text in an episodic memory buffer to enhance decision-making. RepoCoder [139], RLCoder [109], and Repoformer [114] improve inference-time context by training a retriever model. MGDebugger [95] improves debugging by decomposing problematic code into a hierarchical tree structure of subfunctions, with each level representing a particular granularity of error RepoGraph [82] provides a more comprehensive view for the repository by summarizing its structure and content into a graph, where the agent can call tools to view the graph. With a similar high-level idea, an empirical study [25] summarizes the repository structure into a README file, which can be used to guide the agent's repository exploration.

Another line of work focuses on direct approaches that explicitly train a planner agent. MetaGPT [32] performs coding tasks with a distributed software-company pipeline, where "planning" is decomposed into requirement analysis, architecture design, and system design, and are performed by the produce manager, architect, and project manager, respectively. Similarly, ChatDev [89] presents a waterfall-style sequence of design, coding, code completion, review, and testing phases. AgentVerse [13] develops a four-stage loop including expert recruitment, collaborative decision-making, action execution, and evaluation. MapCoder [37] uses a four-agent pipeline: retrieval, planning, coding, and debugging. In addition to the above pipeline-based approaches, AutoGen [115] adopts a more flexible approach by allowing multiple agents to converse with each other to accomplish tasks. HyperAgent [87] has a centralized Planner that generates resolution strategies and dispatches them asynchronously to the navigator, code editor, and executor agents.

In our work, we propose to train a critic model that is specialized in planning and can provide guidance to the coding agent at inference time, which is a type of direct approach that target the planning bottleneck more explicitly. Compared to other direct approaches, our approach is more flexible and generalizable for the following reasons: (1) our critic model can

be directly integrated with a fixed coding agent without retraining it. It does not need to be paired with other specialized sub-agents that are each optimized for a particular intermediate step of the task, and (2) the critic model can be applied at any point during inference and at any desired frequency, thereby enabling flexible control over its use based on user preferences.

Part I

Training Coding Agents with Scalable Synthetic Data

Overview of Part I. This part comprises two chapters organized around a single theme: *by reducing the complexity of constructing training data in appropriate ways, we could potentially improve scalability while preserving effectiveness*.

Chapter 3 illustrates this idea through a code translation task. A key bottleneck of the scale of training data is the difficulty of obtaining parallel data pairs (i.e., functionally equivalent code written in different programming languages). To address this, we show that constructing comparable code pairs (i.e., code snippets with similar functionality) is substantially easier, and that training on such pairs remains effective. Experiments further demonstrate that combining comparable and parallel data significantly outperforms training on parallel data alone.

Chapter 4 applies the same principle to training coding agents. Here, the primary bottleneck is constructing the training environment: typical workflows require installing an entire repository (on average 26.5 external packages), which demands substantial effort from both humans and agents. We mitigate this challenge by bypassing repository installation via sandbox testing, which greatly simplifies environment construction (by installing only 2.1 packages per instance). Importantly, the resulting training examples remain effective because the agent can still access the full repository when completing the task.

Chapter 3

Constructing Scalable Code Translation Training Data with Comparable Corpora and Multiple References

3.1 Introduction

In this chapter, we show a successful case of constructing scalable and effective training data for code translation.

Code translation is a special type of machine translation that translates between programming languages. It is widely applied in software engineering to migrate a codebase into another programming language. Recent code translation models typically follow the pretrain-finetune pipeline. As shown in Figure 3.1, in pretraining, with denoising objectives such as masked span or identifier prediction [1, 93, 110], the model learns to produce sequences in both languages. When finetuned on parallel data [2], which are program pairs in the source and target language that are aligned line-by-line, the model learns functional equivalence: identifying programs with the same functionality, either in the same language or between languages. We show an example of parallel data in Figure 3.2.

One major challenge of code translation is that parallel data is typically limited. For instance, the TransCoder dataset [93] only contains 466 Python-C++ pairs. Constructing parallel data requires substantial human effort and cannot be easily scaled. With limited fine-tuning examples, it is difficult for a model to learn functional equivalence across programming styles and domains.

To mitigate the data scarcity issue, we pose three questions: **(1) What does the model need to learn to perform code translation? (2) Does everything need to be learned from parallel**

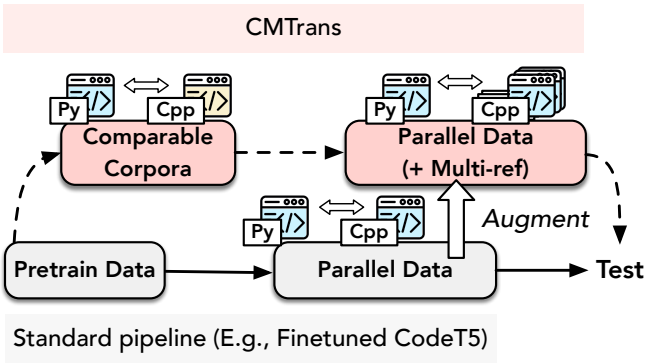


Figure 3.1: The standard pipeline for code translation (colored in light grey) and the CMTrans pipeline (colored in pink). The comparable corpora are both naturally occurring and model generated. We generate multiple references by CMTrans.

Source (Python)

```
board = []
check = False

for i in range (8):
    board.append( str(input()) )
for x in board:

    if "BB" in x or "WW" in x:
        print ( "NO" )
        check = True
        break

    if x != "BW"*4:
        if x != "WB"*4:
            print( "NO" )
            check = True
            break


if check == False:
    print( "YES" )
```

Parallel (Java)

```
String[] board = new String[8];
Boolean check = false;
Scanner in = new Scanner( System.in );
for( int i = 0; i < 8; i++ )
    board[i] = in.next();
for (int i = 0; i < 8; i++){
    String x = board[i];
    if (x.contains("BB") || x.contains("WW")){
        System.out.println( "NO" );
        check = true;
        break;
    }
    if (!x.equals("BW".repeat(4)))
        if (!x.equals("WB".repeat(4))){
            System.out.println( "NO" );
            check = true;
            break;
        }
}
if (check == false)
    System.out.println( "YES" );
```

Comparable (Java)

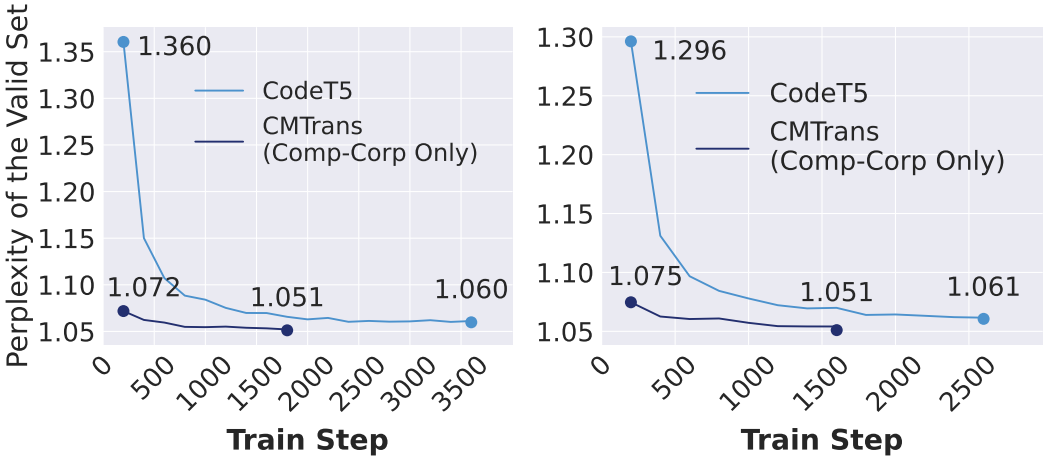
```
import java.util.*;
import java.lang.*;
import java.io.*;
public class Ideone {
  public static void main( String[] args ){
    Scanner in = new Scanner( System.in );
    int f = 0;
    for( int i = 0; i < 8; i++ ){
      String s = in.next();
      in.nextLine();
      for ( int j = 0; j < 7; j++ ){
        if ( s.charAt(j) == s.charAt(j+1) ){
          f++;
        }
      }
    }
    if(f == 0) System.out.println( "YES" );
    else System.out.println( "NO" );
  }
}
```

Figure 3.2: An example of parallel and comparable data. Parallel examples are line-by-line aligned. Programs in a comparable example may have different algorithms and structures (e.g., global code vs. class in this case), but may still contain lines that can be matched, as highlighted in pink, blue, and yellow.

data? (3) If parallel data is still necessary, how to obtain parallel data in an efficient way?

We answer the first question: what the model needs to learn, by comparing the model before and after finetuning. We observe that (1) the model learns to generate fluent code in the target language. As shown in Figure 3.3, with the source language as input, before finetuning, the model continues to generate code in the source language, and the ground truth translation in the target language has a high perplexity. (2) The model learns the functionality equivalence between the source and target languages. Namely, it learns the implementation of the same functionality in both languages.

As for the second question, we show that both (1) and (2) can be learned from *comparable corpora* to some extent, a term we borrow from natural language translation, where it refers



(a) Java-to-Python (b) Python-to-Java

Figure 3.3: Perplexity of validation set during finetuning.

to texts on similar topics in different languages [28, 36]. Here, we use it to refer to programs with similar functionality in different languages [2, 48]. As shown in Figure 3.2, although the programs paired in a comparable example may have different algorithms and structures (e.g., global code vs. class functions), they are likely to have similar constructs and may even have some lines that can be matched.

In this paper, we *study what the model learns from comparable corpora* by building three types of comparable examples: (1) *Naturally existing*, where we leverage independently-written solutions of the same coding problem; (2) *Generated*, where we collect programs with docstrings in one language and apply a code generation model to generate programs in another language; and (3) *Retrieved*, where we either retrieve a program's k nearest neighbors (KNN) or simply choose a random program in another language. Among them, (1) contains cleaner examples, which are guaranteed to be bug-free. (3) covers programs from a larger variety of sources, providing more diverse training signals.

Since comparable corpora only contain program pairs with similar functionality, parallel data is still necessary to translate the precise functionality across different programming languages. As an answer to the third question, we notice that the majority of finetuning data only have one reference translation (e.g., 82.5% in AVATAR, Ahmad et al. 2). However, code with different surface forms may have the same functionality (e.g., the interchange between `for loops` and `while loops`). This makes it possible to augment the dataset by adding multiple reference translations with the same functionality, which reduces overfitting to a single translation and boosts the model to fully capture code semantics.

As a result, in this paper, we *generate multiple translation references* for the finetuning data.

Specifically, after training on comparable corpora, we finetune a model on the original parallel data, generate multiple translations for each example, and use automatically generated test cases to filter out incorrect translations. By training on different functionally equivalent programs, we reduce overfitting to a single output and improve the modeling of the target language.

Combining the two techniques, we name our full approach CMTrans, a code **Trans**lation model trained with **C**omparable corpora and **M**ultiple references. Extensive experiments show that CMTrans significantly improves CodeT5 [110], which is initialized from the same pre-trained checkpoint, for an average of 7.5% Computational Accuracy (CA@1) over translation between 6 language pairs. CMTrans also significantly outperforms the state-of-the-art method in 5 out of 6 language pairs, while reaching parity on the other one.

Analyses of our two techniques suggest that: (1) All three types of comparable corpora (including random program pairs) improve the syntax accuracy and perplexity of the translation outputs and lead to better final performance. (2) Both naturally existing and generated comparable corpora help the model generate constructs that match the input. The combination of them gives the largest performance gain. (3) By training with multiple references, the model generates more unique correct translations within a certain budget, which indicates that better functional equivalence of the target language is learned.

3.2 Methodology

In this section, we introduce our data augmentation method that trains the model first on comparable corpora, which are either naturally existing, generated, or retrieved (§3.2.2) and then on parallel data with additional references generated by CMTrans (§3.2.3).

3.2.1 Problem Formulation

We formulate code translation as a sequence-to-sequence (Seq2Seq) generation problem following existing work [2, 64, 93]. The input is the code in the source language $S = \{s_1, s_2, ..., s_n\}$, which is a sequence of tokens. We apply a model to translate the input code into the target language $T = \mathcal{M}_\theta(S) = \{t_1, t_2, ..., t_m\}$, where the model $\mathcal{M}_\theta$ is typically finetuned for code translation. Alternatively, the model may generate k candidate outputs for each input: $\mathcal{M}_\theta(S, k) = \mathbb{T} = \{T^{(1)}, T^{(2)}, ..., T^{(k)}\}$.

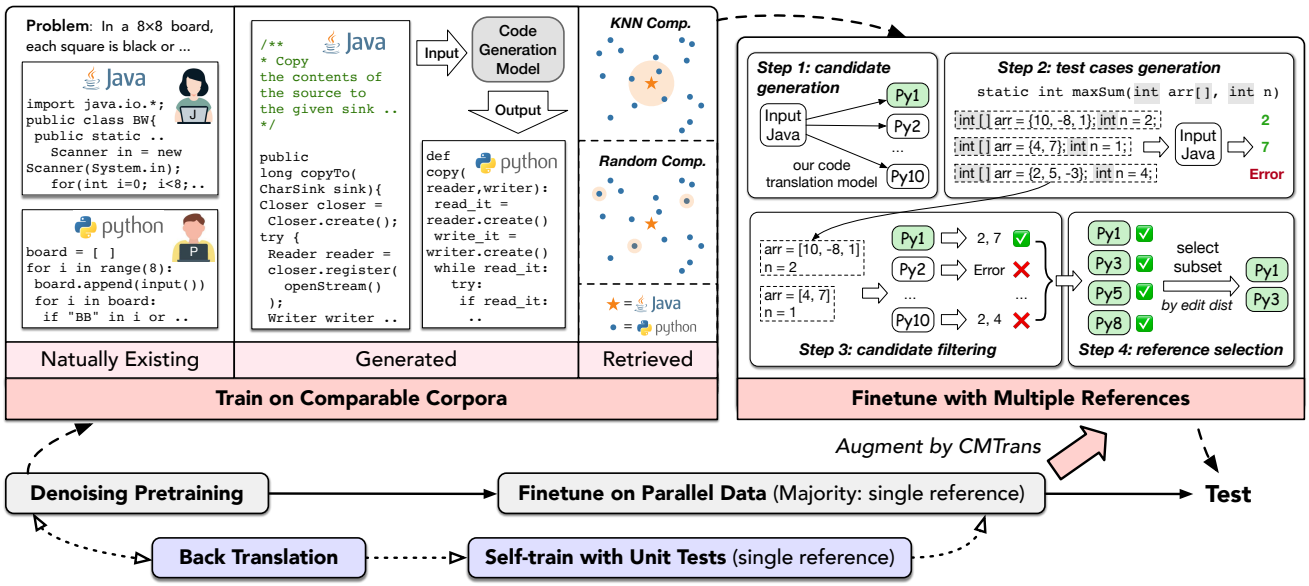


Figure 3.4: An example of CMTrans for Java-to-Python translation. We compare the pipeline of CMTrans (*red*) to the standard pipeline (*grey*) of code translation (e.g., finetuned CodeT5, Wang et al. 110) and the self-supervision-and-fine-tuning method (*purple*) of TransCoder-ST (Roziere et al. 94).

3.2.2 Training on Comparable Corpora

As shown in Figure 3.4, we study three types of comparable corpora: naturally existing, generated, and retrieved ones. Here we introduce how we build each type of comparable examples and train CMTrans on them.

Naturally Existing Comparable Examples. We make use of comparable corpora collected from programming contests by existing datasets [2, 48, 88], which mainly consist of solutions to the same contest problems written in different languages. These comparable examples are typically confined to specific domains such as dynamic programming or graph theory.

Generated Comparable Examples. To cover programs in more diverse domains, we present a method that automatically generates comparable examples using natural-language-to-code generation models.

Specifically, we leverage a monolingual corpus of functions with natural language documentation (i.e., docstrings) to describe their functionality. In the experiments, we use GitHub functions with docstrings extracted by CodeSearchNet [35]. For each function S_c in the corpus, we feed the natural language documentation to a code generation model finetuned in the other language, which is trained to generate a program based on the given natural language

description. Similarly to code translation, the code generation model can generate multiple candidate outputs $\{T_c^{(1)}, T_c^{(2)}, ..., T_c^{(k)}\}$. We then select the one with the highest probability, which is paired up with the extracted program as a comparable example.

Retrieved Comparable Examples. To study the effect of the quality of comparable corpora, we further build **KNN Comparable Corpora**. We compute the embedding of all the programs in the source and target language in the dataset using a finetuned code retrieval model [35]. For each program S in the source language with embedding $emb(S)$, we retrieve its k nearest neighbors in the target language by the cosine similarity between their embeddings: $sim(S, T) = \langle emb(S), emb(T) \rangle$.

Finally, we build **Random Comparable Corpora** by pairing random programs in the source and target language. In principle, such program pairs do not contain any information on functional equivalence and allow us to better understand whether comparable corpora can improve the modeling of the target language and hence improve the translation quality.

Model Training. After constructing a corpus of comparable examples $\mathcal{D}_c = \{(S_c, T_c)\}$, we input the program in the source language S_c into CMTrans $\mathcal{M}_\theta$ and maximize the log-likelihood of its corresponding program in the target language $T_c = \{t_{c,1}, t_{c,2}, ..., t_{c,m}\}$:

$$\begin{aligned} \mathcal{L}_{MT}(\mathcal{D}_c) &= \sum_{(S_c, T_c) \in \mathcal{D}_c} P_\theta(T_c|S_c) \\ P_\theta(T_c|S_c) &= -\sum_i \log \left(P_\theta \left(t_{c,i} | S_c, t_{c,1} \dots t_{c,i-1} \right) \right) \end{aligned} \tag{3.1}$$

Here θ is the parameter of $\mathcal{M}_\theta$, which is initialized from an encoder-decoder model pretrained on code [110]. After training the model on the corpus comparable examples $\mathcal{D}_c$ till convergence, we finetune it on the dataset of parallel examples $\mathcal{D}_p$, where the loss $\mathcal{L}_{MT}(\mathcal{D}_p)$ is computed by Equation 3.1 as well.

By maximizing the probability of the target T_c, the model will learn to generate fluent code in the target language. Furthermore, in general, programs in a comparable example often exhibit the same types of constructs. In the examples in Figure 3.2, to check whether a board is valid, no matter what algorithm is used, the program will always need a loop to read the board and use if statements for the validity check. As a result, in principle, the model can also learn to generate the same types of constructs as the input program S_c, which is beneficial for generating accurate translations.

Dataset →	Train (Comparable)			Train (Parallel)	Test
	Gen-comp	XCodeEval	AVATAR-comp	AVATAR-para	TransCoder-test
# C++ ↔ Java	4,053*	3,414	–	3,226*	482 C-to-J, 467 J-to-C
# C++ ↔ Python	4,053*	4,376	–	3,226*	464 C-to-P, 467 P-to-C
# Java ↔ Python	22,181*	–	5,937	3,391	464 J-to-P, 482 P-to-J
Source	Github (Java ↔ Python) AIZU, AtCoder (Others)	Codeforces	AtCoder, Codeforces, ProjectEuler, CodeJam, GeeksforGeeks, LeetCode,	GeeksforGeeks	GeeksforGeeks

Table 3.1: Number of problems per dataset. "Gen-comp" is the comparable corpora dataset we generate. C-to-J, C-to-P, ... denote the test examples of C++-to-Java, C++-to-Python translation, etc. * denotes data we build.

3.2.3 Finetuning with Multiple References

In addition to comparable corpora, we also provide CMTrans with more diverse training signals by finetuning with multiple references. By providing the model with programs with the same functionality, we encourage the model to learn a better representation space for the target language and hence benefit the translation.

Since the majority of source programs only have one reference in existing datasets of parallel examples [2], we apply a series of steps to generate additional references, which are illustrated in Figure 3.4. The first step is to finetune a model with the original parallel data, and then use the finetuned model to generate multiple translation candidates $\{T_c^{(1)}, T_c^{(2)}, ..., T_c^{(k)}\}$ for each source program in the parallel data.

In the second and third steps, similar to TransCoder-ST [94], we leverage automatically generated unit tests to filter out candidates with different behaviors as the source program. Specifically, we extract the input arguments of the source programs, randomly produce a set of test inputs based on the argument types, and feed the test inputs to the source program. We filter out test inputs that cause compilation or runtime errors. Finally, we feed the remaining test inputs to each candidate and only keep the ones that have exactly the same output as the source program.

Notice that some of the translations may only have small differences (e.g., `i++` vs. `i+=1`). To obtain a diverse subset of references, we select the most distinct k translations for each source program by their string edit distance. These k translations are added as additional references to the finetuning set.

3.3 Results

In this section, we conduct experiments to answer four research questions: (**RQ1**) How do CMTrans and its ablations perform compared against state-of-the-art approaches for code

Model ↓	Java-to-Python			Python-to-Java			Avg of 6 Pairs		
	BLEU	**CB**	**CA@1**	**BLEU**	**CB**	**CA@1**	**BLEU**	**CB**	**CA@1**
TransCoder [93]	72.4	67.9	49.1	65.4	70.7	35.7	72.0	75.0	51.7
DOBF [50]	72.2	67.5	52.2	67.7	71.2	44.4	—	—	—
TransCoder-ST [94]	73.1	68.7	68.5	70.0	71.9	58.1	71.3	74.9	66.3
CodeBERT [23]	52.0	48.9	10.4	45.4	45.0	4.2	—	—	—
CodeT5 [110]	79.4	72.5	61.0	79.0	75.9	52.7	83.6	80.0	62.6
PLBART [1]	79.9	73.2	68.9	80.5	76.8	57.5	—	—	—
TransCoder-ST-ft [94]	79.3	72.9	69.4	81.4	78.4	62.0	81.8	80.2	67.6
CMTrans	**80.1**	**74.2**	**73.5**	**84.3**	**82.1**	**66.0**	**84.9**	**82.0**	**70.1**

Table 3.2: Java-Python translation results on TransCoder-test. We copy the results of all the baselines reported by Ahmad et al. [2]. CB and CA@1 stand for CodeBLEU and Computational Accuracy. We highlight the **best** results under each metric with Bold and underline the second-best results.

Model ↓	C++-to-Java			C++-to-Python			Java-to-C++			Python-to-C++		
	BLEU	**CB**	**CA@1**	**BLEU**	**CB**	**CA@1**	**BLEU**	**CB**	**CA@1**	**BLEU**	**CB**	**CA@1**
TransCoder [93]	84.0	86.7	65.1	75.2	73.4	47.1	83.6	85.4	79.8	51.6	65.7	32.6
TransCoder-ST [94]	78.8	85.2	68.0	73.1	73.0	61.3	76.7	83.7	**84.6**	55.8	67.1	56.7
CodeT5 [110]	90.9	90.0	65.1	82.9	75.4	56.5	89.1	**88.5**	81.6	79.8	77.9	58.5
TransCoder-ST-ft [94]	88.7	90.1	68.3	75.4	74.3	62.5	89.3	87.8	**84.6**	76.7	77.6	59.1
CMTrans	**91.6**	**90.5**	**71.4**	**83.7**	**77.2**	**64.2**	**89.7**	88.4	84.4	**82.0**	**79.3**	**61.2**

Table 3.3: Java-C++ and Python-C++ translation results on TransCoder-test. The CA@1 results of TransCoder and TransCoder-ST are copied from the TransCoder-ST paper. We evaluate BLEU and CodeBLEU using their released checkpoints. We finetune and evaluate CodeT5 and TransCoder-ST-ft on our own.

translation? (**RQ2**) What can the model learn from comparable corpora? (**RQ3**) What can the model learn from multiple references? (**RQ4**) How is CMTrans affected by the size of comparable corpora and the number of references?

3.3.1 Experimental Setup

We initialize CMTrans from the pretrained checkpoint of CodeT5 [110], an encoder-decoder model pretrained on code files from varied languages with denoising objectives.

Datasets. We list the dataset statistics in Table 3.1. All the methods are evaluated on the TransCoder-test dataset [93]. We train our method on xCodeEval [48], a comparable corpora dataset, and AVATAR [2], which contain both comparable corpora and parallel functions (denoted as AVATAR-comp and AVATAR-para, respectively). All the supervised baseline meth-

ods are finetuned on AVATAR-para before evaluation.

Since AVATAR-para does not contain C++ functions, we add a parallel C++ function for each training example in AVATAR-para. Specifically, we generate 50 C++ translations for each Java function by TransCoder-ST, TransCoder, and finetuned CodeT5 and filter the translations with unit tests.

Construction of Comparable Corpora. We conduct the study on different types of comparable corpora (Comp-Corp) on Java ↔ Python translation. We use AVATAR-comp[1] as the naturally existing Comp-Corp. KNN and random Comp-Corp are also retrieved from AVATAR-comp. To build the dataset of generated Comp-Corp (denoted as Gen-Comp), we generate from functions with docstrings in the CodeSearchNet [35] dataset.

We train CMTrans first on the best combination of comparable corpora, which is natural and generated Comp-Corp, and then finetuned on AVATAR-para. For language pairs other than Java ↔ Python, we use xCodeEval as naturally existing Comp-Corp and generate Comp-Corp from CodeNet [88].

Evaluation metrics. Our primary metric is the Computational Accuracy (CA@k), which evaluates whether at least 1 out of k translation candidates generates the same outputs as the reference for all the given test inputs. Following previous work [2], we report **CA@1** results and also report **BLEU**, which computes the overlap between candidate and reference translations [86], and **CodeBLEU**: the weighted average of token level match, syntax level match, and Dataflow match [92].

3.3.2 Main results

Table 3.2 and Table 3.3 show the code translation results on TransCoder-test.

CMTrans substantially outperforms CodeT5, which is initialized from the same pretrained checkpoint and finetuned with the original parallel data, by an average improvement of 7.5% CA@1. CMTrans also significantly outperforms the state-of-the-art methods, TransCoder-ST-ft, on 5 out of 6 language pairs, and reaches parity on Java-to-C++ translation. Note that TransCoder-ST-ft generates test cases for 103,488 functions and executes these test cases over 4 iterations of training. In comparison, CMTrans only generates test cases for 3,391 functions and executes the test cases once, resulting in better efficiency.

Our results show that the advantage of CMTrans is larger on translations between Java and Python. The reason may be that the parallel data we generate for translations involving C++

[1]There are two versions of AVATAR-comp with slight differences. We use the first version because most of our experiments were finished before the second version was released.

Model ↓	J-to-P CA@1	P-to-J CA@1
CodeT5	61.0	52.0
+ Random Comp-Corp	64.7	54.6
+ KNN Comp-Corp	63.8	55.4
+ Generated Comp-Corp	64.0	63.1
+ Natural Comp-Corp	68.8	62.9
+ All Comp-Corp	62.5	55.4
+ Natural & Generated	70.0	64.1
+ Multi-Ref	68.1	61.6
CMTrans	**73.5**	**66.0**

Table 3.4: Ablation studies of CMTrans. We report CA@1 results. J-to-P and P-to-J stand for Java-to-Python and Python-to-Java results. We put the experiments on each type of comparable corpora in an individual block.

are only verified by automatically generated test cases, which might not cover all the boundary cases and could introduce noise to the finetuning set. Following previous work [2, 94], we report the results of one checkpoint.

3.3.3 Performance Analysis

Ablation studies. Table 3.4 presents the ablations of our approach. We first study the effectiveness of each type of comparable corpora and some combinations of them. All the "+ Comp-Corp" ablations denote training CodeT5 on comparable corpora and then finetuneing on AVATAR-para. We also ablate "+ Multi-Ref", which is finetuned on multiple references directly after pretraining. The results show that both Multi-Ref and the best Comp-Corp provide a large performance gain and stacking them (the full CMTrans) has the best performance.

As for the effectiveness of different Comp-Corp, we can see that code translation can benefit from all these types of Comp-Corp. The combination of Natural and generated Comp-Corp has the best performance. The reason might be both Comp-Corp have relatively high data quality and further adding the other two Comp-Corp in training introduces noise to the model and hinders the learning of functional equivalence.

Notice that with the same finetuning set, *Comp-Corp Only* still outperforms TransCoder-ST-ft, which indicates that compared to self-training, training on comparable corpora has not only better efficiency but better effectiveness. We hypothesize that the reason may be that the comparable corpora contain more diverse structures and contents. This provides more diverse training signals to the model and hence improves the generalization.

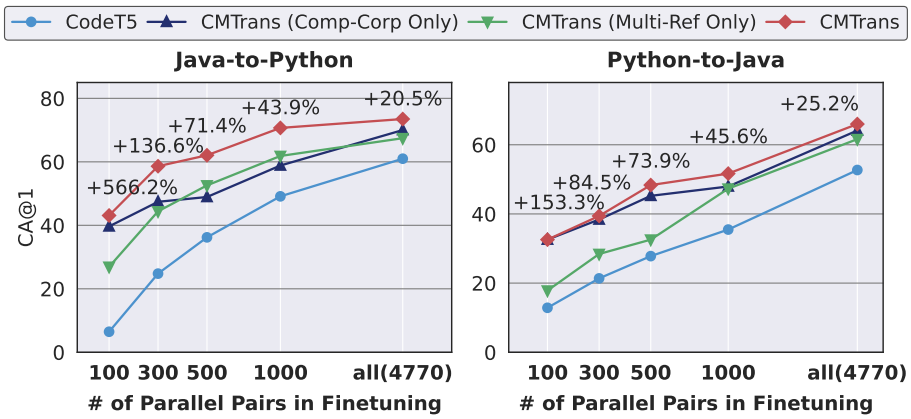


Figure 3.5: Translation results with different amount of parallel data. We mark the relative gain of CMTrans over CodeT5.

Model ↓	Java-to-Python									Python-to-Java								
	LOOP			IF			ELSE IF			LOOP			IF			ELSE IF		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
CodeT5 (No finetune)	100.0	28.5	44.4	99.1	35.4	52.2	0.0	0.0	0.0	100.0	14.6	25.5	100.0	36.6	53.6	0.0	0.0	0.0
+ KNN Comp-Corp	89.4	94.6	91.9	82.1	64.3	72.1	41.6	57.6	48.3	87.9	98.9	93.1	90.2	50.9	65.1	30.5	16.4	21.3
+ Generated Comp-Corp	99.9	98.9	**99.4**	99.0	93.2	**96.0**	84.6	79.2	**81.8**	98.3	96.7	**97.5**	98.7	94.8	**96.7**	83.3	86.4	**84.8**
+ Natural Comp-Corp	95.8	85.4	90.3	91.2	73.9	81.6	50.0	49.6	49.8	87.4	98.9	92.8	88.2	80.4	84.1	57.8	33.6	42.5

Table 3.5: The overlap between the types of constructs in the translation outputs and the ground truth translations.

Translation with Limited Parallel Data. To analyze how well CMTrans tackles the data scarcity challenge, we compare the performance of CodeT5 and CMTrans when finetuned on parallel data with different sizes. For simplicity, we use CMTrans (Comp-Corp Only) to denote the "CodeT5 + Natural & Generated" ablation and use CMTrans (Multi-Ref Only) to denote "CodeT5 + Multi-Ref".

As shown in Figure 3.5, the relative gains of CMTrans as well as our ablations are more pronounced when the parallel data is more limited. For example, when there are only 100 parallel examples for Java-to-Python translation, CodeT5 obtains 6.5 CA@1 while CMTrans, (Comp-Corp Only), and (Multi-Ref Only) obtain 43.1, 39.7, and 26.9 CA@1, respectively. This demonstrates the effectiveness of our two data augmentation techniques to tackle data scarcity.

3.3.4 Influence of Comparable Corpora

To answer **RQ2**, we hypothesize that training on comparable corpora allows the model to produce fluent code in the target language while using similar constructs as the input.

Fluency of outputs. To validate our hypothesis, we compare the perplexity of the reference

Model ↓	**J-to-P SA**	**P-to-J SA**
No finetuning on parallel pairs		
CodeT5 [110]	1.1	1.7
+ Random Comp-Corp	20.0	41.3
+ KNN Comp-Corp	22.0	**56.4**
+ Generated Comp-Corp	**41.4**	43.2
+ Natural Comp-Corp	34.1	54.6
With finetuning on parallel pairs		
CodeT5 [110]	95.3	69.7
+ Random Comp-Corp	96.3	69.7
+ KNN Comp-Corp	96.3	71.0
+ Generated Comp-Corp	**97.6**	71.2
+ Natural Comp-Corp	97.4	**76.4**

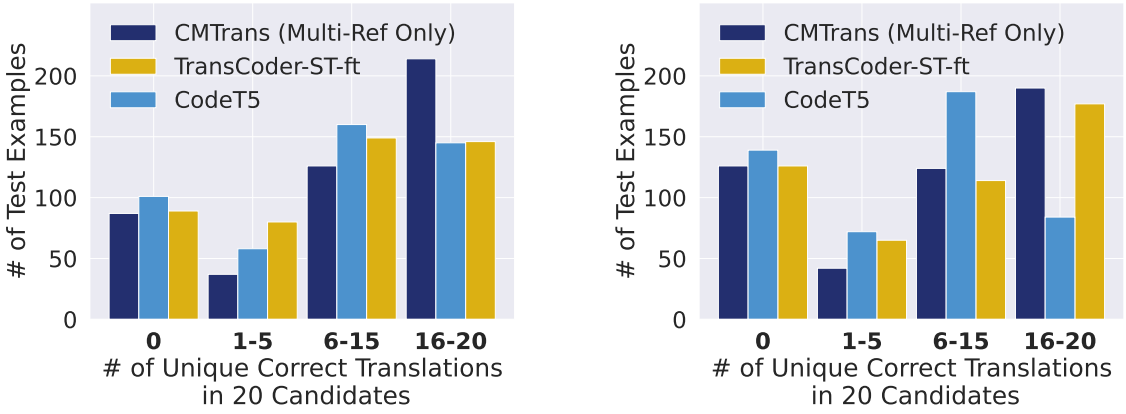
Table 3.6: Syntax Accuracy (SA) on TransCoder-test before finetuning, which evaluates whether the program can be compiled without syntax errors.

translations (in the target language) during finetuning, which reflects the fluency of outputs. As shown in Figure 3.3, after training on comparable corpora, the perplexity is substantially reduced before finetuning. It also converges to a lower value after finetuning on parallel data.

Similarly, Table 3.6 shows that in most scenarios, training on comparable corpora leads to fewer syntax errors both before and after finetuning, which means the programs CMTrans generates not only have a high token-level overlap with the reference translations but are also syntactically correct. The reason might be comparable corpora (including the random Comp-Corp) train the model to maximize the probability of a complete program in the target language, improving its language modeling ability.

Generation of matching constructs. We observe that programs in a comparable pair typically contain similar constructs (e.g., in AVATAR-comp, for 83.41% of Java programs with if statements, the corresponding Python programs also contain if statements). To assess whether CMTrans also learns to generate the correct types of constructs, for each type of construct, we consider whether the reference translation has this type of construct as the ground truth and whether the translation output has it as the predictions. Then we compute the accuracy, recall, and F1 scores.

As shown in Table 3.5, after training on KNN, generated, and natural Comp-Corp, the F1 score is highly improved. The generated Comp-Corp has the highest F1 scores. The reason might be we input both the documentation and the program to the generation model, so the generated program follows the same algorithm as the input program, which results in a larger



(a) Java-to-Python (b) Python-to-Java

Figure 3.6: Number of unique correct translations in 20 candidates for each test example. We use beam search for each method, so the generated candidates are guaranteed to be distinct.

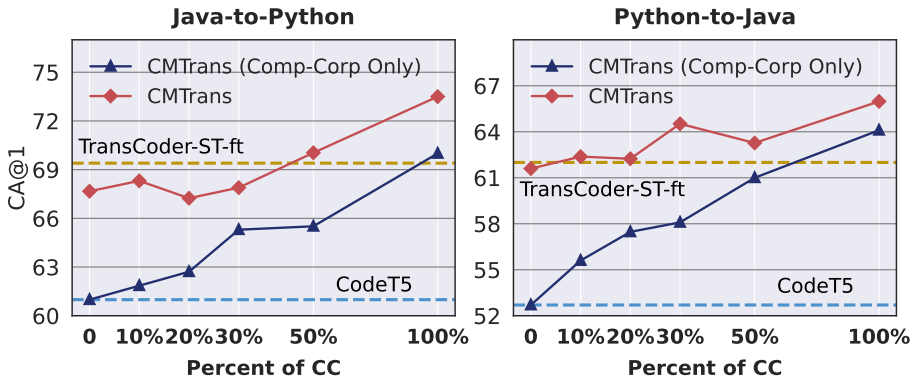


Figure 3.7: Effect of the size of comparable corpora.

percentage of comparable examples that share the same types of constructs.

3.3.5 Influence of Multiple References

As for **RQ3**, we hypothesize that training on additional references can reduce overfitting to a single translation. To validate this hypothesis, we show in Figure 3.6 that when trained with multiple references, CMTrans can generate more unique correct translations using beam search within the same number of candidate outputs. For instance, in Java-to-Python translation, there are 214 test examples where CMTrans generates ≥ 16 unique translations, while there are only 145 and 146 examples for CodeT5 and TransCoder-ST-ft, respectively. Furthermore, the scores under the "0" group indicate that after training with multiple references, the model generates at least one correct translation for more test examples. In other words, in

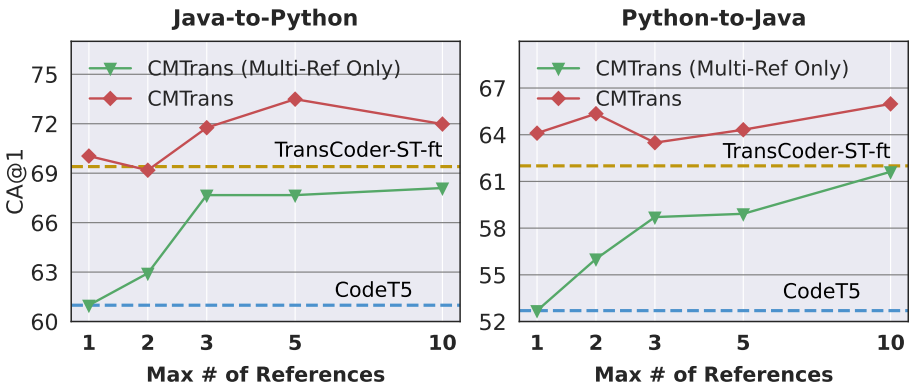


Figure 3.8: Effect of the max number of references for each example.

addition to CA@1, training CodeT5 on multiple references also improves on CA@20.

3.3.6 Hyper-parameter analysis

To answer **RQ4**, we analyze two hyper-parameters:

Size of comparable corpora. We sample the same percentage of comparable examples from AVATAR-comp and Gen-comp and combine the sampled data. CMTrans is finetuned with at most 5 references in all trials. As shown in Figure 3.7, as we increase the size of comparable corpora, CMTrans (Comp-Corp Only) always has better performance. While the trends are not monotonic for CMTrans, it still obtains the best performance with full comparable corpora.

Number of references. We also examine the effect of the maximum number of references for each parallel example in finetuning. We observe that the performance of CMTrans does not always increase when finetuned with more references. The reason might be that CMTrans may generate more unique correct translations for some training examples than others. As a result, the training signals the model received from different examples could be unbalanced, especially when the maximum number of references for each example is large.

3.3.7 Case Studies

We show a constructed comparable example in Figure 3.9. As shown in Figure 3.9, both the Java function extracted from GitHub and the generated Python function remove a config by name and return this config. The only difference is the Java function obtains the name of each config by a helper function, while the generated Python function directly accesses the "`name`" attribute. We also observe that this function is likely to belong to a large software project, which has a different nature from coding problems (e.g., the example in Figure 3.2). As a result,

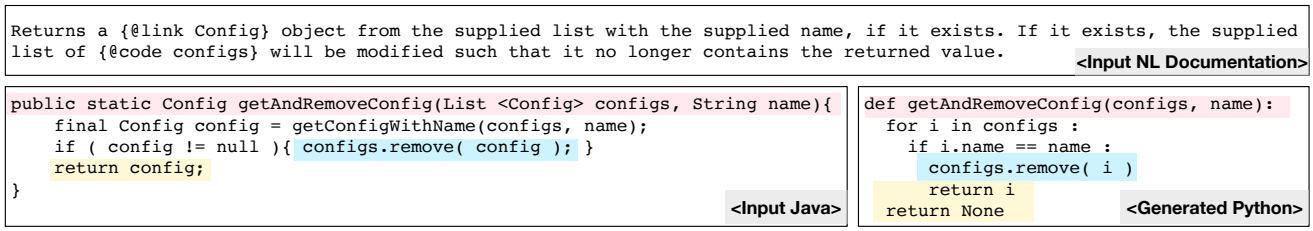


Figure 3.9: Case study: a comparable example generated by our method with an NL-to-code generation model. We highlight the lines in the Java program and the generated Python program that can be matched.

combining collected and generated comparable examples provides diverse training signals to CMTrans.

3.4 Chapter Summary

In this chapter, we present two data augmentation techniques for code translation: comparable corpora and multiple references. We study multiple ways of building comparable corpora, such as feeding natural language code documentation to code generation models. Additionally, we generate multiple reference translations using CMTrans and filter references using automatically generated unit tests. Experiments on translation between 6 language pairs show that our two techniques improve CodeT5 substantially, by an average of 7.5% CA@1. The analyses further show that after training on comparable corpora, the model learns to generate more fluent code with the same types of constructs. With multiple references, the model learns to generate a larger number of unique correct translations.

While this chapter constructs scalable and effective training data for code translation, it focuses on **script-level** and **static** generation. Namely, the input and output are both code scripts, and the model generates code using one pass, without interacting with the environment. The next chapter introduces the construction of scalable and effective training data for **repo-level** coding problems under the **coding agent** setting, which aims to generate code with real-world repositories as context. During inference, the model can interact with the environment by viewing files, editing files, executing bash commands, etc..

Chapter 4

Constructing Scalable Coding Agent Training Environments with Sandbox Testing

4.1 Introduction

The previous chapter improves the scalability of script-level code translation data while remaining effective. In this chapter, we construct scalable and effective repository-level (repo-level) coding agent training data by reducing the complexity of training environment construction.

Repo-level code generation aims to generate code with real-world, naturally occurring repositories as context, which aligns well with real-world software development. However, most existing repo-level training datasets still treat code generation as a traditional sequence prediction task [35, 44], neglecting the executable nature of code generation: the feedback can be obtained from execution. In comparison, existing script-level code generation methods leverage execution feedback to improve model training [60, 72, 138]. They leverage online coding platforms as a natural resource to build large-scale algorithm problem datasets with test cases. Such execution feedback is useful in constructing training targets [72, 138] or reward signals [42, 60].

While online coding platforms serve as a natural resource for large-scale algorithm problem datasets with test cases, it is non-trivial to build **execution-based** datasets for **repo-level** code generation at a **large scale**. One major challenge is to set up executable environments. Existing repo-level datasets typically conduct *integration testing* that executes test files inside the repositories, which requires building the entire repository [40, 44]. As shown in prior re-

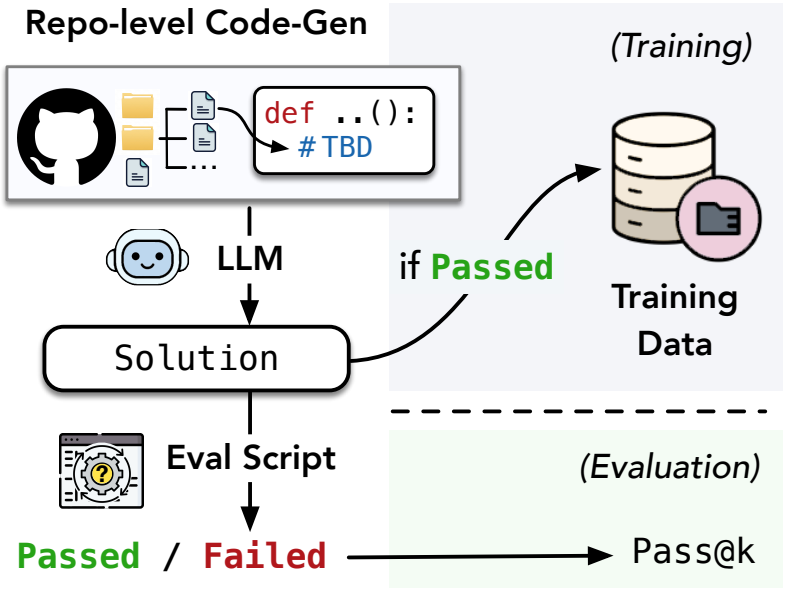


Figure 4.1: We can use the coding environments built by REPOST for training and evaluation. We first apply the code generation model to generate candidate solutions with the original repository as context. Then we evaluate the solutions by executing the evaluation script built by REPOST. For evaluation, we directly compute Pass@k scores. For training, we add all successful solutions to the train set and further finetune the model.

search, this setup process is extremely challenging [10], which may include installing external packages, determining execution context (e.g., relative import), debugging installation errors or runtime errors, etc. Hence, existing methods for coding environment construction either require huge manual effort [44, 84, 139] or, when automated, suffer from low success rate [40], limiting the scale of resulting datasets.

In this work, instead of integration testing, we present REPOST, an **automated** framework to construct **Repo**-level coding environments with **S**andbox **T**esting. Specifically, given a function in a GitHub repository, we sandbox the function and its local dependencies to a separate script and generate tests with an LLM. To control the quality of the evaluation script, we iteratively resolve environment or runtime errors and improve test coverage. We also conduct execution-based, AST-based, and LLM-based quality checks and only keep examples where the functionality of the sandboxed function does not alter and the tests are valid, reasonable, and have high coverage. The resulting dataset allows us to obtain execution feedback in training and evaluation. As shown in Figure 4.1, the models generate the target function with the entire GitHub repository as context. We then use the evaluation script to obtain execution feedback.

Compared to integration testing used by previous datasets, we would like to highlight the

benefits of sandbox testing in constructing **scalable** coding environments: (1) In general, the external dependencies of a function are typically much simpler than those of a repository. By isolating the function and its local dependencies, we can execute the function by *only installing the necessary packages*. (2) If any execution error occurs, we can debug the separate scripts *without modifying the original repository*. This ensures that the datasets remain naturalistic: code generation models can still access the original, real-world repository.

With the high scalability of our framework, we construct coding environment datasets for both training and evaluation: RepoST-Train and RepoST-Eval. By the time the paper was published, RepoST-Train was the largest repo-level code generation dataset with execution support, with 7,415 functions sampled from 824 repositories. The large scale enables training on RepoST-Train and evaluating on other benchmarks such as RepoEval [139] or HumanEval [11].

Experiments show that by training on RepoST-Train, the method achieves performance gain on both algorithm problems in HumanEval [11] and repo-level tasks in RepoEval [139]. For instance, we improve Qwen2.5-Coder by 5.5% Pass@1 on HumanEval and 3.5% Pass@1 on RepoEval and RepoST-Eval.

Note that RepoST-Train provides coding tasks based on real-world repositories that have execution feedback, it can also be applied to coding agent training [106, 128]. For instance, after training on only 500 successful function generation trajectories sampled from RepoST-Train, we improve the resolved rates of Qwen2.5Coder-7B [34] from 1.8% to 7.8% on the SWE-Bench issue-solving dataset [44], from 0.23% to 2.54% on the SWT-Bench test generation dataset [71], and from 4.80% to 9.25% on the Commit-0 library generation dataset [143].

4.2 Methodology

The framework of RepoST is illustrated in Figure 4.2. After sampling GitHub repositories and extracting functions (§4.2.1), We sandbox each function and its local dependencies to a separate script (§4.2.2), generate tests (§4.2.3), and conduct quality control on the executability of the scripts, the functionality of the sandboxed function, and the test quality (§4.2.4). When we use our environments to train (§4.4) code generation models, the models can still access the original GitHub repository, and we provide execution feedback by executing the evaluation scripts. To create user-friendly datasets, **we keep a shared Docker environment for all the evaluation scripts**.

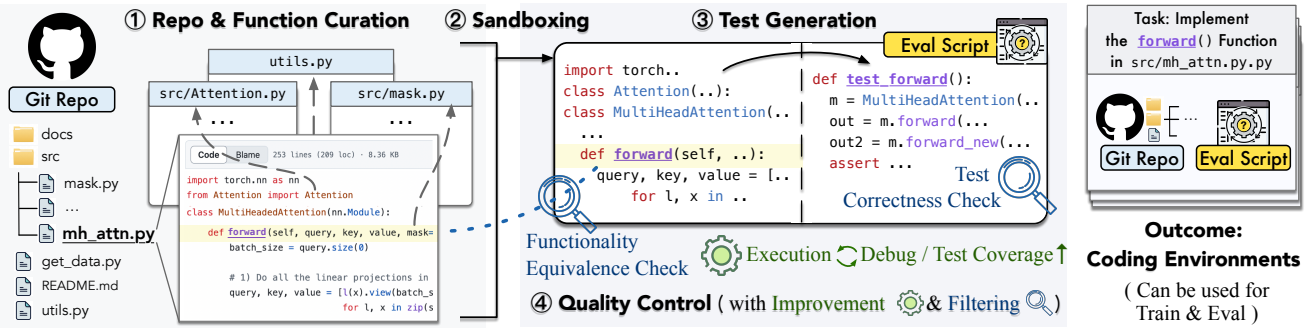


Figure 4.2: The REPOST coding environment construction framework. We sandbox the target function and its dependencies to a separate evaluation script for execution, which avoids building the entire repository. We design careful quality control strategies with iterative quality improvement and post-filtering. The outcome of REPOST is a set of executable repo-level coding environments, which can be used for training and evaluation.

4.2.1 Repository and Function Curation

We first randomly sample non-forked, MIT-licensed, Python GitHub Repositories with file sizes smaller than `10M`. With the sandbox testing method, we are able to set up environments for individual functions and do not need to build the entire repository. Hence, unlike previous works [40], our method does not need to filter out repositories without setup files (e.g., `setup.py`). The detailed repository statistics are provided in §4.3.

Then we extract functions from the repositories. To build the datasets in a docker with no access to GPUs or external services, we follow R2E [40] and filter out functions associated with GPUs, cloud tasks, etc. by keywords. To balance the distribution of examples, we keep at most 30 functions for each repository. After this step, for the train set, we started from 1,000 repositories and obtained 17,448 functions from 851 repositories. For the evaluation set, we started from 200 repositories and obtained 1,043 functions from 139 repositories. In theory, one can further scale up the dataset with more starting repositories.

4.2.2 Sandboxing: Key to Environment Setup

It is non-trivial to create coding environments that provide correct execution feedback for the implementation of GitHub functions. Existing methods typically provide execution feedback with integration testing, which creates test files that import the target function from the codebase. Executing such tests requires building the entire repository, which generally requires a complicated environment and is challenging for both human and LLMs.

With the observation that the external dependencies of a function are typically much sim-

Orig Function

```
def API_call(prompt: str) -> str:
    return API_client.generate(
        model=API_MODEL, messages=[...]
    )
```

Eval Script

```
class Mock_API(object):
  ...
  def generate(model, messages):
    return "mock_" + messages[0]['content']

API_MODEL = "model_name"
API_client = Mock_API("api_key")
def API_call(prompt: str) -> str:
    return API_client.generate(
        model=API_MODEL, messages=[...]
    )
...
```

Figure 4.3: An example where the LLM successfully creates a mock class, `Mock_API`, to replace real external API calls. In this way, while the function body of the target function `API_call` remains exactly the same as in the original codebase, it can be executed without making real API calls.

pler than a repository, We tackle this challenge with **sandbox testing**, where we create a separate script containing the target function with the **exact same functionality** as the original one and the context that supports its executability. By isolating the function and its local dependencies, we can execute the function by only installing the necessary packages.

Sandboxing based on Local Dependencies. The major challenge of sandboxing is to ensure the executability of the function. While a standalone function can be directly copied to a separate script and executed, a typical GitHub function may depend on other modules, classes, or global variables in the same repository. Thus, we leverage the call graph to extract such local dependencies that the target function directly or indirectly calls. Then we combine all the code fragments into the context and prompt an LLM (e.g., GPT-4o) to aggregate all the code fragments into a single script, with as little editing as possible.

Sandboxing External APIs and Files. Even if all the dependencies are presented, it is still nontrivial to execute the sandboxed script if it requires external API, databases, files, etc. We explicitly prompt the LLM to create mock connections for any external API and create strings or write example files to a specific directory for file reading. Figure 4.3 presents a successful case of sandboxing with mock APIs.

Functionality Equivalence Control. In addition to executability, another challenge of sandboxing is to ensure that the target function's functionality does not alter. We first conduct a list of sanity checks on the function name, length of scripts, etc., and re-generate examples that do not satisfy the requirements. We also have a final quality control step (in §4.2.4) that compares the functionality of the sandboxed and original functions. The precision of quality control is further verified in the human study in §4.3.1.

4.2.3 Test Generation

Equivalence Testing. We generate synthetic tests in the sandboxed script to evaluate the correctness of the generated code. Specifically, we prompt the LLM to (1) generate a set of test inputs to the target function and (2) conduct equivalence testing that checks whether the function generated in evaluation has the same behavior as the sandboxed function (i.e., the "ground-truth" implementation). Compared to traditional methods that specify the I/O examples in the test cases [11], equivalence testing does not need to predict the expected outputs and is more feasible for LLMs [40].

Test Generation with Mock APIs and Files. We observe that the LLM is able to generate tests with the mock classes created in the sandboxing step as context. As shown in Figure 4.3, we

create mock class instances as the test inputs and still ensure that the function body of the sandboxed function remains the same as the original function.

Test Quality Control. Similarly to the sandboxing step, for quality control purposes, we conduct a series of sanity checks, such as requiring the test function to call the target function and have at least 3 assertions. We further check the coverage and correctness of the tests in the final quality control step (§4.2.4),

4.2.4 Quality Control and Filtering

We conduct quality control for the executability of the evaluation script, the functionality of the sandboxed function, and the test quality.

Iterative Execution and Debugging. In principle, if the model-generated function is exactly the same as the ground truth, the evaluation scripts should be successfully executed, with all the tests passed. Hence, we execute the examples sequentially in a docker. If there are any execution errors or if the ground truth implementation cannot pass any test cases, we provide the error message as the context and prompt an LLM to debug the evaluation script. Examples that still have errors after k execution-debugging iterations are filtered out.

We also dynamically install external packages during execution. During execution, if there are any `ModuleNotFound` errors, we extract the package names from the error message, run a `pip install` command, and execute the script again. In case the package name differs from the import name or the code requires a specific version of packages, we also allow the LLM to output package installation commands during debugging. In this way, we are able to install external packages for functions extracted from repositories without setup files.

Iterative Test Coverage Improvement. To ensure that the test functions cover the major functionality of the target functions, we further compute the branch coverage rate of all the evaluation scripts. If the branch coverage rate is lower than some threshold (we set 80% for the train set and 100% for the evaluation set), We provide the LLM with the missing lines and prompt it to improve the test function by incorporating more tests.

Final-Step Quality Check & Filtering. As a final step, we conduct two quality checks: functionality equivalence check and test correctness check, and filter out unqualified examples.

To ensure the validity of sandbox testing, we examine the **functionality equivalence** between the sandboxed and original function. We first compare the AST of the function bodies, which is a sufficient condition of functionality equivalence. In our resulting dataset, 81.7% of the examples have the same AST. The remaining ones are filtered out from the evaluation set. To include more examples in the train set, since it is also possible that code with the

Dataset	#Ex	#Repo	Repo?	AutoTest?
HumanEval	164	–	✗	✗
DS1000	1,000	–	✗	✗
ClassEval	100	–	✗	✗
RepoEval-Func	455	6	✓	✗
SWE-Bench	2,294	12	✓	✗
CoderEval	230	43	✓	✗
DevEval	1,874	117	✓	✗
EvoCodeBench	275	25	✓	✗
SWE-Gym	2,438	11	✓	✗
R2E-Eval1	246	137	✓	✓
R2E (Reproduced)	744	123	✓	✓
REPOST-TRAIN	**7,415**	**824**	✓	✓
REPOST-EVAL	296	99	✓	✓

Table 4.1: Statistics of REPOST-TRAIN and REPOST-EVAL compared to existing **execution-based** code generation datasets. R2E (Reproduced) applies the R2E method to the same set of input repositories as REPOST-TRAIN, but results in a smaller number of repositories and examples. "Repo?" and "Auto Test?" refer to the repo-level setting and automatically generated tests.

same functionality has different ASTs (e.g., HTML tags can be parsed with `LexborHTMLParser` and `BeautifulSoup`), We prompt an LLM [1] to compare the functionality equivalence, which is shown to have high agreement with human (see our human study in §4.3.1 for details). We also include the examples that pass the LLM check for the train set.

In principle, a test that calls the target function without any assertion checks still has a 100% coverage rate. We hence conduct **test correctness** check and apply an LLM to check whether the tests are correct, reasonable, and complete the verification of the functionality. Human study in §4.3.1 demonstrates the high precision of the LLM test correctness checker.

4.3 Resulting Datasets: REPOST-TRAIN and REPOST-EVAL

Statistics. We build a train set, REPOST-TRAIN, and an evaluation set, REPOST-EVAL. To reduce contamination, we build REPOST-TRAIN from repositories created between `2023-01-31` and `2024-08-31`, and build REPOST-EVAL from repositories created after `2024-09-01`.

[1] The original code could be inexecutable due to package installation errors (e.g., `BeautifulSoup` in this case), so we do not execute the original code.

Avg Stats.	TRAIN	EVAL
Target # Tokens (Lines)	112.4 (12.8)	102.7 (9.9)
Eval Script # Tokens (Lines)	842.5 (75.7)	1217.5 (122.3)
# Test Cases	5.7	8.2
Test Branch Coverage	97.8%	100%
% Standalone Functions	28.1%	26.4%
# External Libraries	894	106

Table 4.2: Detailed statistics of our datasets.

Check	Func		Test	
Human →	Yes	No	Yes	No
GPT-4o: Yes	13	0	9	1
GPT-4o: No	1	6	4	6

Table 4.3: Agreement between human and GPT-4o on checking (1) the functionality equivalence between the sandboxed and original function, and (2) the test correctness. When GPT-4o predicts "Yes" for both quality checks, it has a high agreement with human.

We compare the statistics of the datasets and existing *execution-based* datasets in Table 4.1. By the time the paper was published, REPOST-TRAIN was the largest repo-level dataset with execution feedback. Prior works such as RepoEval [139] or SWE-Gym [84] require huge human effort to set up the environments. Automated frameworks such as R2E [40] apply LLMs to the complicated task of building the entire repositories and suffer from low success rates. In comparison, REPOST only requires necessary dependencies for each function, which is more feasible for LLMs and benefits scalability.

Table 4.2 shows detailed statistics of our datasets. Our datasets achieve high test numbers and branch coverage rates with iterative test coverage improvement and quality filtering. Results also show that REPOST can create relatively complex examples in terms of length and local and external dependencies. The percentage of standalone functions (i.e., functions without local dependencies) are 28.1% and 26.4% in our datasets. Both are very close to 27%, the percentage of standalone functions among all GitHub code estimated by Li et al. [54].

4.3.1 Quality Verification with Human Study

Agreement between LLM Checker and Human. We conduct a human study to verify the precision of our LLM-based quality check strategies introduced in §4.2.4. We ask 3 computer

% Solved	% Use-Tool	Easy / Med / Hard
81.5%	59.3%	29.6 / 51.9 / 18.5

Table 4.4: Human study results. We ask the participants to complete the function with the same context we use to evaluate code generation models in §4.4.

science graduate students to conduct functionality equivalence and test correctness checks for 20 examples sampled from REPOST-EVAL (before the final filtering), with the same instruction we use to prompt the LLM checkers. The Kappa agreement scores among human annotators are 0.9179 for the functionality check and 0.7750 for the test check.

Results in Table 4.4 show that all 13/20 examples that pass GPT-4o's functionality check are also predicted as "same functionality" by human. Among 10 examples that pass GPT-4o's test quality check, 9 of them are predicted as "high-quality tests" by human. It demonstrates that after applying our filtering strategies for quality control, the remaining examples have high quality. In principle, one can further enhance dataset quality by manually inspecting and selecting the examples.

Solvability Check. We conduct another human study to check whether the examples are reasonable and can be solved by human. We assigned 27 examples constructed by REPOST to 9 computer science students, with no overlaps, and asked them to complete the function and answer questions about the difficulties of the examples.

Results show that 81.5% of the examples were solved by human, indicating that most examples are reasonable and not too complicated. The remaining examples were not solved for various reasons, such as the participant is not familiar with the task (e.g., reading html data) or related libraries (e.g., `BeautifulSoup4`), the intent of the function cannot be fully entailed from the context, etc. We also observe that the generated examples have varying complexity levels based on the usage of external tools and the difficulty ratings. Furthermore, 33.3% of the examples are solved in the first submission and 37.0% require more than 5 submissions to solve.

4.4 Code Generation Training Experiments

Training with REPOST-TRAIN. In standard supervised fine-tuning (**SFT**), we can train the model with the code context c as the input and the ground truth target function f^* as the output. The execution feedback provided by our REPOST evaluation scripts further allows us to employ the **rejection sampling fine-tuning** (**RFT**) algorithm to generate additional valid

Model	HumanEval		RepoEval-Func		REPOST-EVAL	
	Pass@1	Δ	Pass@1	Δ	Pass@1	Δ
StarCoder2-7B [63]	34.76	–	32.98	–	26.35	–
+ SFT	37.20	↑2.44	33.78	↑0.80	27.70	↑1.35
+ RFT (Self)	39.63	↑4.87	34.58	↑1.61	28.38	↑2.03
+ RFT (Distill)	**40.24**	**↑5.49**	**35.12**	**↑2.14**	**29.05**	**↑2.70**
Qwen2.5-Coder-7B [34]	79.27	–	38.06	–	29.39	–
+ SFT	80.48	↑1.21	39.94	↑1.88	30.74	↑1.35
+ RFT (Self)	**84.76**	**↑5.49**	40.75	↑2.69	31.76	↑2.36
+ RFT (Distill)	**84.76**	**↑5.49**	**41.55**	**↑3.49**	**32.43**	**↑3.04**

Table 4.5: Code generation training results. We evaluate Pass@1 for all experiments. For RepoEval, we use the "Oracle" repo-level context as used in their original paper.

training targets. Specifically, we apply the model itself to our dataset, generating n candidate solutions for each function based on its code context: $(c, f_1), \ldots (c, f_n)$. The method is denoted as **RFT (Self)**. We can also apply other stronger models to generate candidates (denoted as **RFT (Distill)**). Only solutions that pass our test cases are retained. We then finetune the model on the successful functions $(c, f'_1), \ldots (c, f'_m)$ and the ground truth (c, f^*) using the standard negative log-likelihood loss.

To obtain more training pairs for both the "Self" and "Distill" settings, we further prompt the model itself or a stronger model to debug the failed solutions, with the error message in the context. We also train the model on successfully debugged solutions $(c, f''_1), \ldots (c, f''_k)$.

4.4.1 Experimental Setup

Datasets and Evaluation Metrics. We train two models: StarCoder2-7B [63] and Qwen2.5-Coder-7B [34] with REPOST-TRAIN and evaluate on two public benchmarks: HumanEval [11], an algorithm problem dataset, and RepoEval [139], a repo-level code generation dataset. We also evaluate on REPOST-EVAL. For RepoEval, we only evaluate on the "function" split, which supports execution. We use the "Oracle" context to mitigate the bias of context retrieval methods. We report the Pass@1 scores on all datasets.

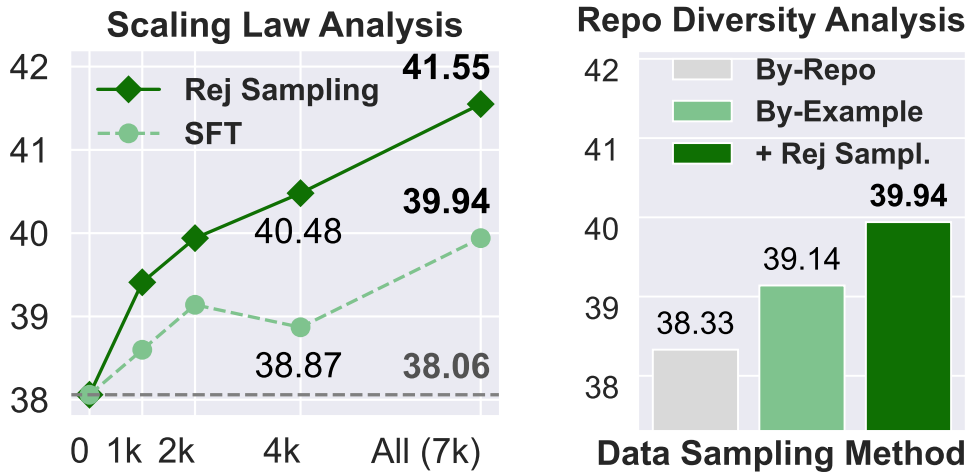
Training Details. We compare three training methods: SFT, RFT (Self), and RFT (Distill). For the "Distill" method, we apply GPT-4o and Claude-3.5-Sonnet to generate and debug candidate solutions, separately, and combine their successful candidates for training.

4.4.2 Experimental Results

Main Results. Table 4.5 demonstrates that models trained with REPOST-TRAIN can generalize well to other public benchmarks. Specifically, we improve Qwen2.5-Coder by 5.5% Pass@1 on HumanEval, 3.5% on RepoEval-Func, and 3.0% on REPOST-EVAL. Furthermore, in all experiments, training with RFT, even with self-training only, achieves better performance than finetuning on the original GitHub function only. For instance, RFT (Distill) outperforms SFT by 4.3% Pass@1 on HumanEval. This shows the benefit of training with environments that can provide execution feedback. We can also observe that RFT with self-training in general has lower performance than distilling from stronger models. Particularly, We can only obtain 1573 additional training targets with StarCoder2, but we obtained 3606 by combining GPT-4o and Claude-3.5. We hypothesize that one can obtain more examples and hence achieve better performance by sampling with more candidate solutions, generating from different types of context, etc., and we leave that to future work.

Scaling Law Analysis. In Figure 4.4a, we investigate how the scale of training data affects model performance. Specifically, we randomly sample different numbers of examples from REPOST-TRAIN to train the model. We can see that the performance of RFT (Distill) increases as we scale up the training data, which suggests the advantage of training data with high scalability. Furthermore, with different scales of data, RFT consistently achieves better performance than SFT, which further demonstrates the effectiveness of training environments with execution feedback.

Repository Diversity Experiment. Figure 4.4b examines whether broader repository coverage leads to better performance, given a fixed budget of training examples. We fix the number of examples to 2,000 and experiment with two example sampling methods: (1) Sample-by-Repo, where we keep sampling repositories and adding all the functions in the repository to the training set, until the data size reaches 2,000; (2) Sample-by-Example, which is the same setting as Figure 4.4a, where we randomly sample functions from REPOST-TRAIN. We observe that Sample-by-Example, covering 678 repositories, outperforms Sample-by-Repo, which only covers 221. The performance is further improved by RFT. Recall that our method only needs to set up individual functions, compared to existing methods, such as R2E, that need to build the entire repository, it is much easier to build datasets with broad repository coverage with our method, which benefits model training.



(a) Scaling law analysis.

(b) Repo diversity.

Figure 4.4: (a) Pass@1 scores on RepoEval with different numbers of training examples. (b) Pass@1 scores on RepoEval with different methods to sample 2,000 training examples. Sample-by-Example has a broader repository coverage and achieves better Pass@1. The performance is further enhanced with RFT (Distill).

4.5 Coding Agent Training Experiments

In this section, we show that in addition to static code generation models, REPOST-TRAIN can also be used to train coding agents.

4.5.1 Experimental Setup

Training Coding Agents with REPOST-TRAIN. We formulate the training task as follows: given a GitHub repository and the path to a function in it, whose body has been masked out, the agent needs to implement this function based on a piece of textual description.

To set up the task, we run a Docker container for each REPOST-TRAIN instance. We clone the GitHub repository and checkout to the specific commit. The target function would be masked properly before applying the agent to solve the task. Then we apply the agent to complete the masked function. After it finishes, we extract the function body it generates (if any), and run REPOST's evaluation script to verify its correctness.

Finally, similar to section 4.4, we train the agent model with rejection sampling finetuning: we apply the teacher model on a set of training instances, evaluate the results, and train the model on the collection of successful trajectories with supervised finetuning.

Training Details. In this experiment, we use Claude-3.7 [4] as the teacher model and train

Methods	**SWE-Bench** Resolved %	**SWT-Bench** Resolved %	**Commit-0** Resolved %
Qwen2.5Coder-7B	1.8	0.23	4.80
+ RepoST-Train (500) RFT	**7.8** ↑**6.0**	**2.54** ↑**2.31**	**9.25** ↑**4.45**

Table 4.6: Performance of the Qwen2.5Coder-7B agent on SWE-Bench Verified, SWT-Bench Verified, and Commit-0 Lite before and after training.

Qwen2.5-Coder-7B as the student model. We use OpenHands [106] (v0.40.0) as the agent scaffold. To limit the cost of generating training trajectories, we only generate 500 successful trajectories with Claude-3.7 for training.

Evaluation Datasets and Metrics. We report the resolved rates on three repo-level coding agent benchmarks: the SWE-Bench Verified issue-solving benchmark [44], the SWT-Bench Verified test generation benchmark [71], and the Commit-0 library construction benchmark [143]. Note that none of the benchmarks specifically targets function generation. Namely, the experiment is under the zero-shot task generalization setting.

4.5.2 Experimental Results

Table 4.6 presents the results of training the Qwen2.5-Coder-7B agent on 500 trajectories generated from RepoST-Train. Results show that although RepoST-Train does not target issue-solving, test generation, or library construction, the finetuned agent significantly outperforms the base model in all three benchmarks. Particularly, we improve the performance of Qwen2.5Coder-7B from 1.8% to 7.8% on SWE-Bench Verified, from 0.23% to 2.54% on SWT-Bench Verified, and from 4.80% to 9.25% on Commit-0 Lite. This demonstrates that the RepoST training environments are also effective in training coding agents. Note that we only train the agent on 500 successful trajectories (generated from around 1200 RepoST-Train instances) due to computational budget. It is possible to generate a larger training set based on the entire RepoST-Train dataset. We leave the exploration to future work.

Table 4.7 shows the performance of the Qwen2.5Coder-7B agent on SWE-Bench Verified after training with R2E and RepoST-Train. Both methods build function generation tasks for training from the same set of GitHub repositories, but R2E [40] yields only 74 successful trajectories and improves the resolved rate from 1.8% to 3.4%, whereas we collect 500 successful trajectories with RepoST-Train and reach 7.8%. In other words, compared to existing methods, the sandbox testing method scales to more training environments from a fixed repository pool and produce a more powerful coding agent.

Methods	|Train Set|	SWE-Bench Resolved %
Qwen2.5Coder-7B	—	1.8
+ R2E RFT	74	3.4 ↑1.6
+ RepoST-Train RFT	500	**7.8** ↑**6.0**

Table 4.7: Performance of the Qwen2.5Coder-7B agent on SWE-Bench Verified after training with R2E and RepoST-Train. For fair comparison, for both methods, we build function generation training data from the same set of repositories on GitHub.

4.6 Chapter Summary

In this chapter, we present RepoST, a scalable method to construct environments for code generation in real-world repositories that support sandbox testing. RepoST is fully automatic and enables the construction of scalable execution-based training environments and live benchmarks. Experiments demonstrate that training with the resulting train set, RepoST-Train, leads to performance gain on other public benchmarks. For instance, we improve Qwen2.5Coder by 5.49%/3.49% Pass@1 on HumanEval/RepoEval.

While both chapter 3 and chapter 4 focus on training and evaluation for one single task, we would like to note that the application of coding agents can be diverse in real-world usage. We will explore the generalizability to a wide range of coding agent tasks in Part II.

Part II

Analyses and Principles: What Makes Coding Agent Training Effective?

Overview of Part II. Part I focuses on constructing training environments for coding agents that are both scalable and effective. In Part I, we make two assumptions on the effectiveness of training: (1) we can rely on one single training task, and (2) it is sufficient to use task success as the only metric for selecting training demonstrations. However, both assumptions do not always hold. Our further analyses show that (1) training on a single task is NOT sufficient for generalizing to diverse downstream tasks, and (2) even successful demonstrations may have substantially different impacts in training.

In this part, we take a step back and **study the factors that make coding agent training effective**. Both chapters follow the same structure: we first investigate the abilities that the coding agents need to learn, and then derive training data design principles based on these abilities.

Chapter 5. In chapter 5, we decompose general coding agent tasks into multiple problem-solving stages (e.g., repository exploration and solution implementation) and we aim to improve coding agents' abilities to perfom each stage correctly by designing synthetic training tasks. We derive a set of principles for designing effective synthetic training tasks. For instance, the tasks must require non-trivial planning and repository exploration to resolve, and must require generating a code patch as the outcome for the solution implementation stage. Guided by these principles, we build our dataset, **Hybrid-Gym**, on a set of scalable synthetic tasks, for instance, function localization and dependency search. Experiments show that agents trained on our synthetic tasks effectively generalize to diverse real-world tasks (e.g., issue-solving, test generation, and library construction) that are not presented in the training data.

Chapter 6. While the task design principles derived in chapter 5 only require the presense of problem-solving stages (e.g., repository exploration and solution implementation) in the training demonstrations, we further observe that the effectiveness of training also substantially depends on **how** the stages are accomplished. For instance, we observe that for two sets of training demonstrations that both successfully locate target files in a codebase, the specific strategies to locate the file also substantially affect the final performance.

chapter 6 thus investigates teaching the coding agents what kind of fine-grained behaviors can lead to better performance, where "behavior" is defined as the pattern of accomplishing each problem-solving stage and may include the strategies, the selection of tools, the patterns of tool call command and reasoning text. Specifically, to find such influential behaviors in an efficient way, we present a **CATRadar**, an analysis method to automatically discover candidate behaviors by defining, extracting, and comparing the behaviors of different trained agents. **CATRadar** enables us to find behaviors that are influential to at least one model, including specific repo-exploration strategies, the planning and verification patterns, and the verbosity

of the reasoning text.

The insights obtained by **CATRadar** can improve the effectiveness of training data generated under the Hybrid-Gym synthetic tasks by various methods, including data selection, agent step synthesis, and behavior-conditioned rollout. For instance, results show that when training on the Hybrid-Gym function localization task, by inserting explicit planning steps into the training demonstrations that are synthesized by LLMs, we improve Qwen3-8B's resolved rate on SWE-Bench Verified from 11.6% to 17.6%.

Chapter 5

Constructing Effective Synthetic Tasks for Training Coding Agents

5.1 Introduction

Recent advances in large language models have enabled LM-based agents [106, 128] to tackle real-world coding tasks, with rapid progress in benchmarks such as the SWE-Bench issue resolution benchmark [44]. While most existing methods only focus on the issue-solving task [41, 84, 130], the application of coding agents can be diverse in real-world usage. A recent study show that more than 70% of the real-world prompts target other software engineering tasks [12].

As shown in a concurrent work [145], training on a single task can lead to overfitting and is not sufficient for generalizing to general coding agent tasks. In particular, agents trained on issue solving do not transfer reliably to tasks such as test generation or library implementation. This calls for training coding agents with stronger **task generalization** capabilities. Yet, in the current literature, task transferability for coding agents remains underexplored. In this work, we ask: what commonalities do real-world coding tasks share? How can we design training tasks that transfer effectively across a broad range of coding agent tasks?

To answer these questions, we start by decomposing real-world tasks into intermediate components (e.g., reasoning and repo-exploration) and analyzing the model's capabilities on each. These results are shown in Figure 5.1. We observe that a large number of actions in successful trajectories are spent on reasoning, repo-exploration, and implementation. These categories also remain challenging for current coding agents. For instance, even after finetuning on issue-solving [84], the agent still has file editing failures for the majority of the examples.

Fortunately, these intermediate components can be isolated from the construction of coding

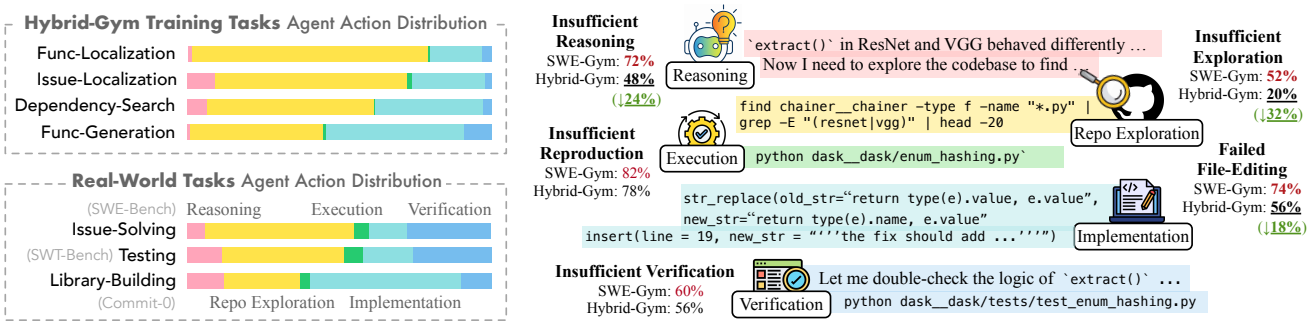


Figure 5.1: (***Left***) We decompose general coding agent tasks into a set of intermediate components and compute the percentage of agent actions spent on each component. Our training tasks partially cover verification and fully cover reasoning, repository exploration, and implementation, which consist of around 68% of the actions. (***Right***) Example actions for each component. Compared to a baseline training method, SWE-Gym [84], training with our Hybrid-Gym significantly reduces failures due to insufficient reasoning, insufficient exploration, and failed file editing, improving the final resolved rate on SWE-Bench Verified from 10.6% to 15.0%.

environments with executable repositories (e.g., with all Python packages required by the repository installed). In previous work, setting up executable repositories is often viewed as a prerequisite for constructing training examples. This typically involves a series of complex steps such as package installation and test generation, which is still challenging for agents [145] and time-consuming for human [41, 84, 130]. Based on the observation from our analysis, we ask: **can we design a set of tasks that cover the major components for coding practice: reasoning, repo-exploration, and implementation, but do not require executable repository setup?**

In this paper, we present Hybrid-Gym, a large-scale coding agent training dataset containing a suite of coding agent training tasks that only require simple environment setup. Based on our analysis summarized in Figure 5.1, we present a list of principles for training task design: the task needs to (1) match the output format of downstream tasks; (2) include a repository exploration process; and (3) require non-trivial reasoning. For scalability concerns, we further require that the task (4) must *not* require a complicated environment setup. Example tasks include function localization, issue localization, dependency search, and function generation. As shown in Figure 5.1, even without setting up executable repositories, all our tasks cover the major intermediate components of multiple downstream tasks, and such components are achieved with a similar set of actions across tasks. After training on Hybrid-Gym, the agent has significantly lower failure rates in reasoning, exploration, and file-editing compared to the baseline training method [84].

Command	Example	Func-Localize	Issue-Localize	Dep-Search	Func-Gen	SWE-Bench	SWT-Bench	Commit-0
grep	`grep -i "pipeline"`	31.3%	36.1%	48.4%	89.9%	35.4%	37.4%	5.3%
find	`find . -type f -name "least_angle.py"`	17.5%	11.1%	21.4%	8.7%	16.2%	21.5%	31.6%
cd	`cd /workspace/scikit-learn`	21.9%	31.5%	2.0%	0.0%	26.7%	0.0%	5.3%
ls	`ls -la`	6.6%	2.3%	3.7%	0.0%	2.4%	12.4%	36.8%

Table 5.1: The use of Linux commands for repository exploration in our proposed training tasks (left) and downstream tasks (right). We show the percentage of each command used in the agent steps targeting repository exploration.

Category	Func-Localize	Issue-Localize	Dep-Search	Func-Gen	SWE-Bench	SWT-Bench	Commit-0
Analysis and reasoning	1.6%	9.3%	6.3%	0.9%	6.0%	11.4%	1.3%
Repository exploration	77.9%	66.2%	55.5%	44.6%	49.1%	40.3%	27.3%
Execution of existing code	0.1%	0.3%	0.1%	0.2%	4.7%	6.3%	3.7%
Solution implementation	17.4%	23.9%	35.3%	45.3%	12.7%	16.2%	53.8%
Solution verification	3.0%	0.3%	2.8%	9.0%	27.6%	25.9%	10.9%

Table 5.2: We categorize the agent's actions based on the intermediate components of the task, where the categories are pre-defined.

We conduct experiments on three real-world coding agent tasks: SWE-Bench issue-solving [44], SWT-Bench test generation [71], and Commit-0 library generation [143]. Without training on any of the downstream tasks, Hybrid-Gym demonstrates strong task transferability on all three tasks, improving Qwen2.5Coder-32B by 25.4% / 7.9% / 5.1% on SWE-Bench Verified / SWT-Bench Verified / Commit-0 Lite, which achieves comparable performance to in-domain datasets (i.e., data built for the downstream tasks). Additionally, we also improve the performance of training on existing in-domain datasets by combining with Hybrid-Gym (e.g., we improve SWE-Play [145] by 2.4% / 4.9% on SWE-Bench Verified / SWT-Bench Verified).

To provide insights to future research on coding agent training, we conduct a series of controlled experiments to reveal what transfers and what does not for coding agents. In addition to validating our task design principles that require output format matching, repo-exploration, and non-trivial reasoning, we observe that even successful trajectories of the same set of instances can lead to markedly different impact. Repository diversity improves training effectiveness, but training on the same repositories used in evaluation does not inflate performance gains. The results emphasize the importance of teacher model selection and data selection.

5.2 Methodology

In this section, we first guide training task design by analyzing the commonalities shared among coding tasks and agent capabilities (§5.2.1). Based on the analysis results, we introduce a list of task design principles (§5.2.2) and present the four Hybrid-Gym tasks that satisfy

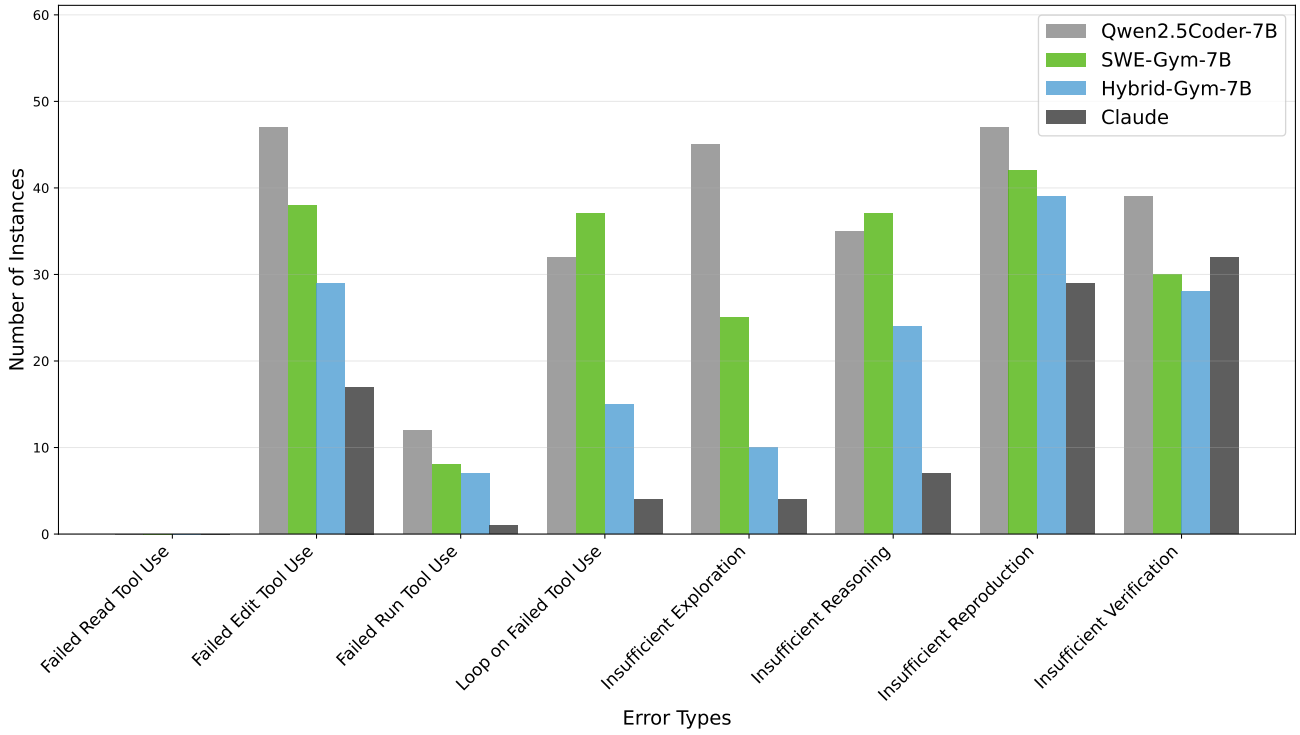


Figure 5.2: The error analysis of different methods on 50 instances sampled from SWE-Bench Verified.

all the principles (§5.2.3). Finally, we conduct a comparison between our training tasks and real-world coding tasks (§5.2.4).

5.2.1 Analysis of Coding Agent Task Transferability

With the ultimate goal of designing training tasks that transfer to diverse downstream tasks, we first investigate the commonalities shared by real-world tasks by answering: [**RQ1**] What are the commonalities of the components (i.e., high-level problem-solving steps) to complete real-world coding tasks? [**RQ2**] What are the commonalities of the concrete actions (i.e., commands) taken by the agents? Since effective training must target capabilities the agent has not yet learned, we also ask [**RQ3**] What are the abilities that the agent has not learned but are required for task completion?

Analysis Setup. Following prior works [41, 84, 145], we adopt the rejection sampling finetuning setting, where we finetune Qwen2.5-Coder-7B/32B on the successful trajectories generated for the training tasks. We conduct the analysis based on 200 successful trajectories sampled from each task, which are generated by Claude-4.5 [5] using OpenHands CodeAct as the agent scaffold [106]

Task Analysis: Decomposition of Coding Tasks. To answer **RQ1**, we first decompose general coding agent tasks into five intermediate components: reasoning, repo-exploration, execution of existing code, solution implementation, and verification. We then use o3-mini [76] to categorize each agent action by the component it targets. We manually examined 20 cases and made the same categorization as o3-mini in all the cases.

Results in Table 5.2 show that (1) The action distribution across different components is similar for the three downstream tasks, with repo-exploration accounting for the largest share. (2) We notice that around 70% of the actions fall into categories that do not require setting up executable repositories: reasoning, repo-exploration, and implementation. This is notable because executable repository setup is complicated and challenging for both humans and agents [130, 145] and is the bottleneck of constructing scalable training datasets. Our analysis hence suggests a key direction: training tasks that avoid executable repository setup can also be beneficial.

Task Analysis: Concrete Actions Used in Coding Tasks. To answer **RQ2**, we analyze the concrete agent tools used in each task. The tools built for the OpenHands agent [106] include bash command execution (`execute-bash`), file viewing (`view`), and file editing (`str-replace`). Results in Table 5.2 show that all three tools are heavily used across downstream tasks. This suggests that *the agents need to learn all the tools in training.*

For the bash execution tool, we further analyze the specific commands issued by the agent. Table 5.1 shows that different tasks rely on highly similar repo-exploration commands, such as grep, find, cd, and ls, suggesting that *repo-exploration skills could be broadly transferable across tasks.*

Agent Capability Analysis. To answer **RQ3**, under the setting of distillation-based training, we conduct an error analysis for the untrained student model (Qwen2.5Coder-7B, Hui et al. [34]), the student model trained on a baseline dataset (SWE-Gym, Pan et al. [84]), and the teacher model (Claude-4.5, Anthropic [5]). We define error categories along two axes: (1) failures in the intermediate components identified in our previous analysis, and (2) failures in the use of each tool. We then prompt an LLM to label whether each agent trajectory exhibits these failures.

Figure 5.2 presents the error analysis results. We observe several error categories where the fine-tuned model's failure rate remains far higher than the teacher model's: insufficient repo-exploration, insufficient reasoning, and failure/loop on the file-editing tool. This suggests that *these capabilities still have substantial room for improvement.*

5.2.2 Principles for Training Task Design

We summarize the analysis results in §5.2.1 as follows: (1) reasoning, repo-exploration, and implementation are the dominant intermediate components shared across coding-agent tasks; (2) The concrete types of commands to complete these components are also similar across tasks; and (3) The performance gaps between the student and teacher models are still large for the above categories and file editing.

Based on the task commonalities and the agents' capabilities, we present the following principles for training task design:

[**Principle 1**] The tasks should have the same output format as the downstream tasks. For the downstream tasks in our evaluation, the format is generating a code patch in the codebase, which requires file editing. A counterexample for training tasks is a variant of our issue-localization task, where we only need the agent to generate the plan to resolve the issue in the message, without making any code changes.

[**Principle 2**] The tasks should involve a meaningful repository-exploration phase. A counterexample is a script-level code generation task (e.g., HumanEval [11]). Although we can still provide a repository as context by putting the solution template file inside an empty repository, it does not involve any file localization.

[**Principle 3**] The tasks should require a non-trivial level of reasoning to complete. Namely, the agent must make substantive decisions, such as choosing the problem-solving strategy, generating the precise tool call arguments, and writing the code. A counterexample is a simple string-replacement task where we provide the exact edit location and the old/new strings. The agent can succeed by a single `str-replace` call without inspecting the codebase.

With the observation that all the major components: reasoning, exploration, and implementation, do not require an executable repository, we further require the training tasks to be scalable (i.e., easy to generate new instances):

[**Principle 4**] The task setup should not require substantial effort. For instance, the task should avoid installing packages for the entire repository or generating executable test files, which are challenging for both humans and agents.

In §5.3, we will show that all Hybrid-Gym tasks satisfy all four principles and generalize well to multiple downstream tasks. In contrast, §5.4 will show that all three counterexample tasks mentioned above do not yield effective task transfer.

5.2.3 Hybrid-Gym Tasks

In this section, we introduce four Hybrid-Gym tasks that satisfy all the principles above.

Dataset	`execute-bash`	`view`	`str-replace`
Func-Localize	5.55	4.43	1.16
Issue-Localize	9.37	7.76	5.15
Dep-Search	1.78	3.12	1.89
Func-Gen	0.53	1.75	1.47
SWE-Bench	10.65	6.88	3.65
SWT-Bench	16.54	6.09	5.16
Commit-0	18.40	21.20	36.40

Table 5.3: The average number of OpenHands tools called in each trajectory.

Model	Func-Localize		Issue-Localize		Dep-Search		Func-Gen	
	Success %	Non-Empty %	Success %	Non-Empty %	Success %	Non-Empty %	Success %	Non-Empty %
Qwen2.5Coder-7B	0.0%	50.0%	3.3%	16.7%	6.7%	60.0%	5.0%	70.0%
Qwen2.5Coder-32B	40.0%	95.0%	33.3%	83.3%	56.7%	86.7%	25.0%	95.0%
Claude-4.5	70.0%	100.0%	63.3%	100.0%	80.0%	100.0%	50.0%	100.0%

Table 5.4: Performance of the student and teacher models on a subset of instances sampled from each training task.

Function Localization. To boost repository exploration (principle 2), we design a task to localize a function in the entire codebase: given a short description, the agent needs to identify the corresponding function in the codebase. To satisfy Principle 1, which requires generating a code patch as the outcome, we require the agent to write a docstring containing the fix plan. We evaluate the task by checking whether a docstring is added for the correct function. This task requires substantial reasoning (principle 3), as the agent needs to understand the code content and conduct an effective and targeted search over the entire repository, which contains 2,925 functions on average in our training set.

Issue Localization. Function localization aims to locate the code based on the description, and issue localization locates the problematic code based on an issue. Given a GitHub issue, the task is to locate the relevant code in the repository and leave a comment containing a plan to fix the issue. We evaluate the task by checking whether a comment is generated in the same file where the actual fix is located.

Compared to the issue-solving task used in most prior work, constructing issue localization data does not need test execution for evaluation and can be instantiated from arbitrary GitHub issues. However, it still requires reasoning effort on analyzing the issue, exploring the repository, planning the fix, and making successful file-editing actions.

Dependency Search. We have the third task that targets code localization, which requires file-editing, repo-exploration, and reasoning: given a function, the agent must identify all func-

tions or classes it directly calls. Similar to the above two tasks, we require adding comments to all these dependent modules. To obtain the ground truth, we adopt `Jedi`, a static Python analysis tool, to resolve imported names to their defining modules. Solving this task requires the agent to locate the target function, extract the referenced names, resolve each name to the correct module, and successfully edit the relevant files. This is non-trivial because a codebase may contain multiple modules with the same name, and only one corresponds to the actual call.

Function Generation. Beyond the localization tasks that only aim to generate comments, we design the function generation task that requires actual code generation. Given a function in a codebase, we use an LLM to generate a description of its functionality. Then we remove the function body and ask the agent to re-implement it. To evaluate the implementation with test cases while avoiding repository installation, we adapt the RepoST [120] method to the coding agent setting, which extracts the target function and its dependencies to a separate script and generates tests.

Here we highlight that generating *executable test scripts* is substantially easier and more cost-efficient than setting up *executable repositories*. To execute the test script, we only need to install the packages needed by the extracted function and its dependencies (2.1 packages on average), while installing the entire repository requires 26.5 packages on average. Furthermore, compared to *generating a new test file* that requires understanding the repository structure and resolving relative imports, it is substantially easier to *produce tests for a separate script*, which only requires seq-to-seq generation, without any agentic frameworks, making the setup process cost-efficient.

We emphasize that Hybrid-Gym is not limited to the specific tasks introduced here: other task designs may also satisfy our principles. It is also possible to construct variants of our tasks, such as localizing multiple functions, or finding the empty function before generating the function body. We leave the exploration to future works.

5.2.4 Verification of Hybrid-Gym Tasks

In this section, we verify that our training tasks share commonalities with real-world tasks and remain challenging for the student models we use: Qwen2.5Coder 7B and 32B.

Hybrid-Gym tasks cover the majority of intermediate components in multiple real-world tasks. As shown in Table 5.2, all the training tasks cover reasoning, repo-exploration, and implementation, and the function-generation task additionally includes solution verification. We note that although our training tasks do not explicitly include executing existing code, it

Dataset	# Trajs	# Repos	Avg Steps	# Images	Cost
SWE-Gym	491	11	40.2	2.4k	unknown
R2E-Gym	3,321	10	33.2	4.5k	unknown
SWE-Smith	5,016	128	26.7	128	2.32¢
SWE-Play	704	28	72.9	28	unknown
Hybrid-Gym	4,470	762	39.1	2	0.07¢
Func-Localize	1,438	226	29.0	1	0.02¢
Issue-Localize	1,978	263	52.4	1	0.00¢
Dep-Search	502	120	26.9	1	0.00¢
Func-Gen	552	306	28.6	1	0.56¢

Table 5.5: Statistics of Hybrid-Gym and its subsets. Following SWE-Smith, we report the average per-instance cost of setting up the training environment for rollout. Compared to existing datasets, Hybrid-Gym covers more repositories and requires only 2 docker images to build all training instances.

typically accounts for only a small fraction of an agent's trajectory.

Hybrid-Gym trajectories contain similar actions with real-world tasks. As shown in Table 5.1, the training tasks use a similar set of bash commands for repo-exploration as the real-world coding tasks. Similarly, Table 5.3 shows that the training tasks cover the full set of OpenHands tools used in downstream tasks. Together, these results suggest that both the high-level problem-solving components and the concrete actions that achieve them can transfer across Hybrid-Gym tasks and downstream tasks.

Hybrid-Gym tasks are still challenging for models without finetuning. Under the setting of distillation learning, a training task is useful only if there is a performance gap between the student and teacher models As a sanity check, before generating training data, we sample 20-30 instances from each task and evaluate the teacher model (claude-4.5-Sonnet) and student models (Qwen2.5Coder, 7B and 32B).

Table 5.4 show that Claude-4.5 achieves substantially higher success rates than Qwen2.5Coder-32B on all four tasks, with gaps ranging from 23.3% to 30.0%. This confirms that all tasks provide sufficient room for effective distillation.

5.3 Results

5.3.1 Experimental Setup

Statistics of Hybrid-Gym. Table 5.5 presents the statistics of Hybrid-Gym and existing datasets. Based on the scalable training tasks, we collect 4.4k trajectories from 762 repositories using

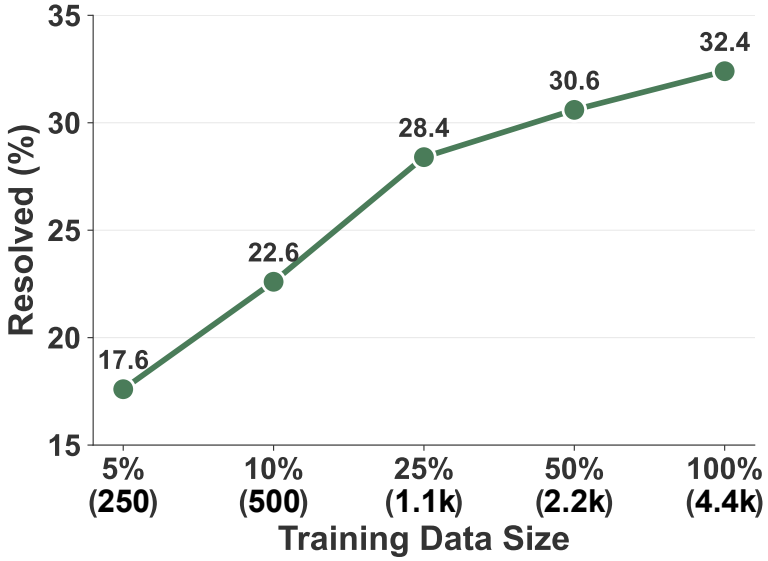


Figure 5.3: Scaling law analysis. Performance on SWE-bench Verified improves consistently as training data size increases from around 5% (250 trajectories) to 100% (4.4k trajectories).

only two Docker images. The environment construction cost is only 0.07¢per example, which is 16× lower than SWE-Smith [130]. Trajectory lengths vary by task, ranging from 26.9 to 52.4 agent steps, which are comparable to the issue-solving trajectories in prior datasets, suggesting that our tasks retain a meaningful level of complexity. The trajectories are obtained from the combination of Claude-4.5 [5], Claude-3.7 [4], and Qwen3-235b [126].

Evaluation Datasets and Metrics. We follow prior work [145] and report the resolved rates on SWE-Bench Verified [44], SWT-Bench Lite/Verified [71], and Commit-0 [143]. On SWE-Bench, we additionally report the localized rate: the percentage of code patches applied to the correct file, and the non-loop rate: the percentage of trajectories without any "loops", which is defined as repeating the same action three times consecutively [84].

5.3.2 Main Results

Hybrid-Gym Shows Strong Task Transfer Performance. Results in Table 5.6 show that Hybrid-Gym delivers strong task generalization performance across all three benchmarks. Relative to the base 32B model, it improves performance by 25.40 % / 7.85 % / 5.11 % on the three tasks, outperforming all existing task transfer training datasets. Notably, without training on the downstream tasks, Hybrid-Gym matches or even exceeds in-domain training. For example, SWE-Play-32B reaches 31.2 % / 16.17 % on SWE-Bench Verified / SWT-Bench Verified, whereas Hybrid-Gym-32B achieves 32.4 % / 16.86 %. This suggest that our tasks capture the

Training Tasks				Training Dataset	Issue Solving (swe) SWE-bench Verified			Test Generation (swt) SWT-Bench Lite & Verified		Library Build (cmt) Commit-0 Lite
swe	swt	cmt	syn		Resolved %	Localized %	Non-Loop %	Lite Resolved	Verified Resolved	Resolved %
				Base Model: Qwen2.5Coder-7B						
				(*base*) Qwen2.5Coder-7B	1.8	4.4	60.4	0.72	0.23	4.80
			✓	SWE-Play-general	8.6 +6.8	49.2 +44.8	93.4 +33.0	2.54 +1.82	1.62 +1.39	9.25 +4.45
✓				SWE-Gym	10.6 +8.8	48.2 +43.8	79.0 +18.6	1.45 +0.73	0.69 +0.46	8.61 +3.81
✓				R2E-Gym	**19.0 +17.2**	—	—	0.72 +0.00	0.46 +0.23	9.25 +4.45
✓				SWE-smith	15.2 +13.4	—	—	0.00 -0.72	0.00 -0.23	9.32 +4.52
✓	✓	✓	✓	SWE-Play	17.0 +15.2	60.4 +56.0	91.2 +30.8	3.26 +2.54	3.00 +2.77	10.42 +5.62
			✓	**Hybrid-Gym**	15.0 +13.2	**62.4 +58.0**	**97.6 +37.2**	2.90 +2.18	2.54 +2.31	10.54 +5.74
✓	✓	✓	✓	**Hybrid-Gym** + SWE-Play	17.6 +15.8	58.6 +54.2	97.2 +36.8	**5.07 +4.35**	**4.39 +4.16**	**11.02 +6.22**
				Base Model: Qwen2.5Coder-32B						
				(*base*) Qwen2.5Coder-32B	7.0	45.6	70.6	9.42	9.01	8.34
✓				SWE-Gym	20.6 +13.6	57.8 +12.2	76.2 +5.6	3.26 -6.16	2.77 -6.24	9.58 +1.24
✓				R2E-Gym	34.4 +27.4	—	—	3.26 -6.16	4.39 -4.62	11.56 +3.22
✓				SWE-smith	**40.2 +33.2**	—	—	13.77 +4.35	10.62 +1.61	12.38 +4.04
✓	✓	✓	✓	SWE-Play	31.2 +24.2	73.4 +27.8	85.2 +14.6	18.12 +8.70	16.17 +7.16	13.95 +5.61
			✓	**Hybrid-Gym**	32.4 +25.4	**75.4 +29.8**	98.2 +27.6	17.03 +7.61	16.86 +7.85	13.45 +5.11
✓	✓	✓	✓	**Hybrid-Gym** + SWE-Play	33.6 +26.6	75.2 +29.6	**99.2 +28.6**	**20.29 +10.87**	**21.02 +12.01**	**15.52 +7.18**

Table 5.6: Results on three real-world coding tasks. We show whether the datasets contain swe, swt, cmt, or synthetic tasks (syn) in training. Results with a white background are **task transfer** results. Results with a light grey background are trained with in-domain tasks. We highlight the best result for each dataset and show the relative gain and loss compared to the base model, Qwen2.5Coder. Hybrid-Gym achieves strong task transfer performances across all three benchmarks and further improves in-domain datasets (e.g., SWE-Play).

commonality shared among general coding agent tasks and transfer effectively across all three downstream tasks. These results are especially compelling because Hybrid-Gym requires substantially less effort to build new training instances than prior datasets (e.g., as shown in Table 5.5, SWE-Smith requires 128 docker images and costs 2.32¢per example, while Hybrid-Gym is only built with 2 images and an average cost of 0.07¢).

Hybrid-Gym Improves In-Domain Training Datasets. Table 5.6 further shows that we can further improve in-domain datasets by combining with Hybrid-Gym. For instance, we improve SWE-Play by 2.40% / 4.85% / 1.57% on SWE-Bench Verified / SWT-Bench Verified / Commit-0 Lite. The gains from mixing Hybrid-Gym with in-domain data suggest that the two sources are complementary. Hybrid-Gym primarily teaches general agent skills (e.g., reasoning, repo-exploration, and reliable tool use), while in-domain data contributes task-specific behaviors (e.g., generating tests and building libraries from scratch). Combining them both leads to additive improvements.

Training Tasks (# Instances)	**SWE-Bench** Resolved %	**SWT-Bench** Resolved %	**Commit-0** Resolved %
Qwen2.5Coder-7B	1.8	0.23	4.80
+ Issue-Solving (491)	10.6 +8.8	0.69 +0.46	8.61 +3.81
+ Func-Localize (500)	12.8 +11.0	**3.00 +2.77**	8.74 +3.94
+ Issue-Localize (500)	9.6 +7.8	2.31 +2.08	9.09 +4.29
+ Dep-Search (500)	6.4 +4.6	2.31 +2.08	9.25 +4.45
+ Func-Gen (500)	7.8 +6.0	2.54 +2.31	9.25 +4.45
+ **Hybrid-Gym**-full	**15.0 +13.2**	2.54 +2.31	**10.54 +5.74**

Table 5.7: Performance of training Qwen2.5Coder-7B on each of the Hybrid-Gym tasks and the issue-solving task. We sample 500 instances from each of our tasks. Even without finetuning on issue-solving, function localization achieves a larger performance gain than SWE-Gym, which has 491 issue-solving instances. We report the resolved rates on SWE-Bench Verified, SWT-Bench Verified, and Commit-0 Lite.

5.3.3 Detailed Results

Training on Individual Tasks. To assess the contribution of each task, we sample 500 instances per task and train the model on each subset separately. This training size is comparable to the SWE-Gym issue-solving dataset, which contains 491 examples. As shown in Table 5.7, each task yields a substantial improvement over the base model, with gains ranging from 4.6% to 11.0%. The tasks emphasize different capabilities (e.g., function localization performs best on SWE-Bench but worst on Commit-0), while the full Hybrid-Gym dataset—combining all four tasks—achieves the strongest overall performance.

Notably, at similar training scales, the function-localization and issue-localization tasks provide gains that are comparable to, or even exceed, in-domain training. For example, SWE-Gym achieves a 10.6% resolved rate and a 48.2% localized rate, whereas Func-Localize (500) reaches 12.8% and 53.6%. A possible explanation is that localization trajectories devote more steps to repo-exploration than issue-solving, which is one of the agent's primary bottlenecks.

Scaling Law Analysis. As shown in Figure 5.3, we train the 32B model on progressively larger subsets of Hybrid-Gym, where we proportionally sample training instances from each task. The performance continues to improve as the dataset grows and still shows improvements when we have 4.4k training trajectories. The results suggest the advantage of constructing highly scalable training datasets.

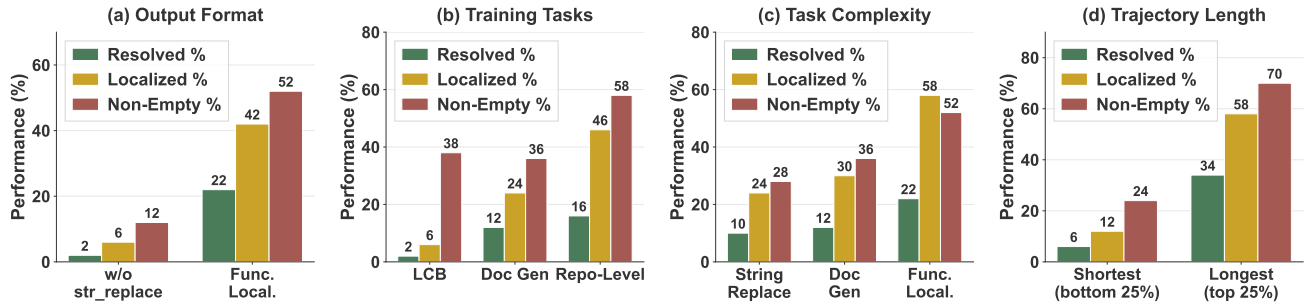


Figure 5.4: Controlled experiments on training data characteristics. (a) **Output Format:** Removing the file editing actions (`str_replace`) from function localization trajectories causes a large drop in SWE-bench resolution rate. (b) **Repo-Exploration:** script-level code generation (LCB) does not effectively transfer to repo-level issue-solving and even underperforms documentation generation, a simple repo-level task. (c) **Task Complexity:** Transfer improves as training tasks become more complex. (d) **Trajectory Complexity:** With a fixed data size, training on longer (more agent steps) trajectories substantially improves downstream performance.

5.4 Analysis of Task Transferability

We have established in §5.3 that the particular tasks included in Hybrid-Gym are effective for task generalization across diverse real-world coding benchmarks. In this section, we investigate: **what properties of these trajectories enable task transfer?** To answer this, we run controlled ablations that systematically modify trajectory structure and measure the resulting changes in performance.

As the ablations require repeated evaluation across many variations, we keep the analyses computationally tractable by evaluating on an easier subset of SWE-bench examples (denoted as "Easy 50"), which are solvable by Deepseek-v3, a strong reference model [17].

5.4.1 Output Format Must Match Downstream Tasks

While our tasks are designed around higher level tasks such as function localization and generation, we also find that *how* these behaviors are expressed in the training trajectories matters. Specifically, as stated in [**Principle 1**] in §5.2, we find that the output format of the tasks is a crucial factor.

All three downstream tasks in our experiments require the agent to generate a code patch as the output. To isolate the role of output format, we conduct a minimal ablation on our function localization task: instead of requiring the model to generate a comment in the codebase containing the fix plan, we simply ask it to generate the plan in a message. We accordingly

modify the training trajectories by removing the `str_replace` actions for writing comments and adding the fix plan in the final message generated by the agent.

As shown in Figure 5.4, performance on SWE-bench collapses when we remove the string replacement actions. These results suggest that for coding agent tasks, the production of **patch-like outputs** is not a superficial formatting choice but a crucial skill that the agent needs to learn.

5.4.2 Script-level Agentic Tasks Do NOT Generalize to Repo-level Tasks

We then validate [**Principle 2**], which requires the training tasks to have a repo-exploration stage. To test this, we implement a script-level task, LiveCodeBench (LCB) [39], which primarily evaluates function generation in a single file. We contrast it with two repo-level tasks: (1) the repo-level function generation task we use in Hybrid-Gym (introduced in §5.2), and (2) documentation generation, which requires the agent to generate the docstring for a given function. For fair comparison, we wrap the coding template and test script in LCB in a dummy repository so that the model interacts with the same harness and tool API.

Figure 5.4 shows that despite having the wrapper to make LCB superficially a repo-level task, LCB is not effective for transfer to SWE-bench. This negative result suggests that repository-level generalization requires more than superficial "agentic formatting." In particular, repo-level tasks induce behaviors that are largely absent from script-level problems: identifying and navigating relevant files, grounding edits in existing codebase structure, and producing concrete patch-like modifications that apply cleanly in context.

5.4.3 Task and Trajectory Complexity Matters

After identifying reasoning and planning as a crucial component in coding tasks in §5.2, we derive [**Principle 3**] that requires the task to involve a non-trivial level of reasoning. Here, we examine whether task generalizability improves as training supervision more closely matches the multi-step, tool-mediated interaction patterns required for coding tasks.

We first examine tasks with ascending complexity: (1) single-shot string-replacement edits in which the task is to refactor a function by simply renaming it without replacing other usages of it, (2) documentation generation for a fixed function (as in §5.4.2), and (3) the function localization task, which requires more complex reasoning than the previous two tasks. We see in Figure 5.4 that as tasks become more complex and interaction-heavy, downstream SWE-bench performance improves, and models are substantially more likely to produce non-empty patches.

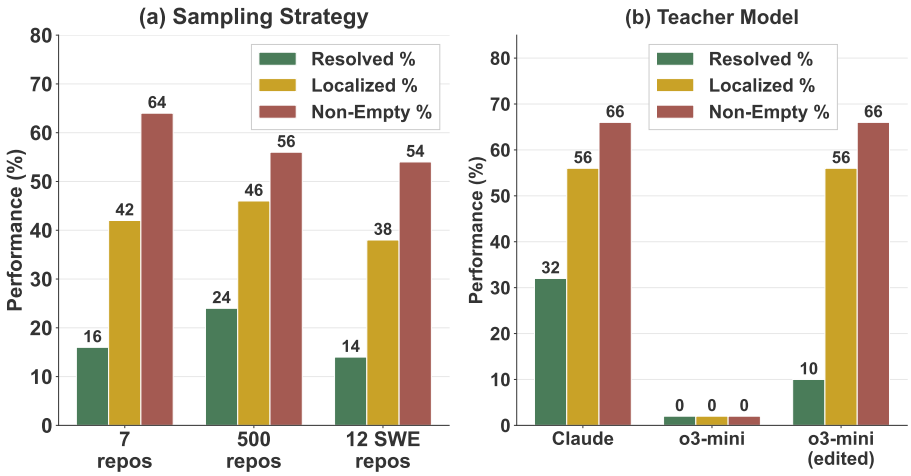


Figure 5.5: Effect of teacher model and data selection. (a) Effect of **sampling strategy** at fixed training budget. Repository diversity improves training but using the same repositories as in evaluation does not. (b) training on issue localization trajectories with different **teacher models**. o3-mini (edited) indicates the same set of trajectories as o3-mini, but with text and action steps combined.

To isolate complexity effects from task identity, we also investigate the effect of *trajectory complexity*, which is operationalized as the number of agent actions (tool calls and intermediate components) taken in successful trajectories. We pool trajectories from our repo-level task mixture and partition them by length, selecting the shortest and longest quartiles (25% each). As shown in Figure 5.4, although the training sizes are the same, training on longer trajectories yields a large improvement in SWE-bench resolution rate, alongside a sharp reduction in empty generations. These results suggest that trajectory complexity is an important driver of task performance.

5.4.4 Even Successful Trajectories of the Same Task Instances may have Different Impact

In addition to training task design, we observe that **the teacher model also plays a crucial role in effective training**. Specifically, we find that even successful trajectories for the same instances can yield very different distillation outcomes. We illustrate a particular case in Figure 5.5. On the issue localization task, we found that distilling from Qwen3-235B and Claude trajectories transfers well to SWE-Bench, but distilling from o3-mini trajectories degrades student performance. The key difference we found is that o3-mini frequently separates reasoning and actions (i.e., tool calls) into different turns. After training on such data, the agent collapses to the distribution of thinking-only conversational turns and does not learn to call tools at all.

We edit o3-mini trajectories by simply stitching adjacent rationale and action steps into a single example. The resulting data effectively transfers to SWE-Bench, indicating that model-specific behaviors such as separating reasoning and actions can strongly influence the effectiveness of distillation.

5.4.5 Sampling Strategies Affects Effectiveness

Finally, beyond task choice and trajectory structure, we also find that how training instances are sampled impacts downstream transfer. We examine two key factors: (1) repository diversity (i.e., the number of repositories covered by the training set), which is identified by previous works [120, 130], and (2) the domain of code. In the extreme case, we build training data from the exact same repositories we use in evaluation.

To investigate this, we keep the training budget fixed at 500 trajectories and sample from function localization trajectories. We compare: (i) *Min Repo-Diversity*, where we aggregate instances from a small number of large repositories (7 repositories), (ii) *Max Repo-Diversity*, where we sample one instance per repository (500 repositories) to maximize repository coverage, and (iii) *In-Domain Repos*, where we construct function localization trajectories based on the 12 SWE-Bench repositories.

Despite using the same number of training instances and the same SFT recipe, *Max Repo-Diversity* yields higher SWE-bench resolution (Figure 5.5). This suggests that exposure to a broader range of repository structures improves generalization to unseen repositories. Surprisingly, **training on the same repositories in evaluation does not improve the performance**, suggesting that learning general agent skills is more crucial than memorizing repo-specific information.

5.5 Chapter Summary

In this chapter, we improve the task transferability of coding agents. We conduct in-depth analyses of the transferability across coding agent tasks. We find that there are similar components shared by diverse real-world coding tasks (e.g., repo-exploration) that can be achieved by the agent with similar commands. Based on the discovery, we derive a set of principles for task design, including output format matching, requiring repo-exploration, non-trivial reasoning, and simple setup. We present four scalable and cost-efficient tasks that do not require setting up executable repositories. With the resulting dataset, Hybrid-Gym, we achieve strong task generalization results on three real-world benchmarks. For instance, even without issue-

solving or test-generation data in training, we improve Qwen2.5Coder-32B by 25.4% on SWE-Bench Verified and 7.9% on SWT-Bench Verified. We also improve existing in-domain datasets by combining them with Hybrid-Gym (e.g., improving SWE-Play by 4.9% on SWT-Bench Verified).

The principles introduced in this chapter focus on training task design. However, as studied in §5.4, there are other aspects that are also crucial for agent training, such as training instance selection and the concrete actions in successful training trajectories. We thus propose chapter 6 to investigate more training data construction principles, focusing on the effectiveness of training trajectories.

Chapter 6

Discovering Effective Demonstration Behaviors for Training Coding Agents

Chapter Overview. Most coding agent training methods judge the quality of demonstrations based on their final outcome, such as execution results or rubric scores. However, we observe that the effectiveness of training also substantially depends on *how* the demonstrations achieve the results. Previous work has manually made hypotheses about which demonstration behaviors influence the performance. To find such influential behaviors for **C**oding **A**gent **T**raining more efficiently, we present **CATRadar**, an analysis method to automatically discover candidate behaviors by defining, extracting, and comparing the behaviors of different trained agents. **CATRadar** enables us to find behaviors that are influential to at least one model, including specific repo-exploration strategies, the planning and verification patterns, and the verbosity of the reasoning text. Our insights also translate to performance gains by various methods, including data selection, agent step synthesis, and behavior-conditioned rollout. Results show that our agent step synthesis method improves Qwen3-8B's resolved rate from 11.6% to 17.6% on SWE-Bench Verified.

6.1 Introduction

Agents have been trained to be effective in diverse real-world coding applications [71, 143]. Existing research improves coding agent post-training by constructing scalable environments [84, 130], designing training methods [97, 113], and synthesizing training signals [98, 107]. Most of them still derive training signals based on the final outcome of the demonstrations, which is either obtained by execution in sandbox environments or simulated with manually-designed

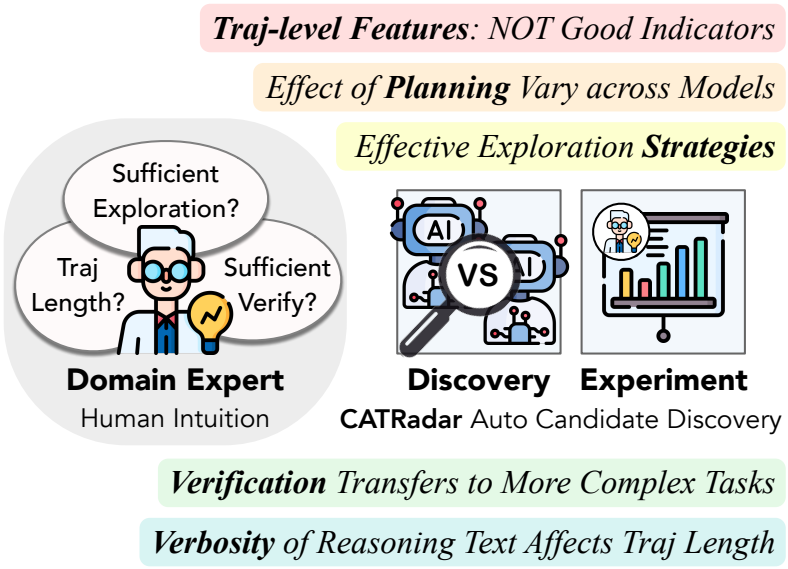


Figure 6.1: To find demonstration behaviors that affect the effectiveness of training, (*left*) prior works rely on human experts to manually make hypotheses, while (*right*) **CATRadar** narrows down the search space by automatically discovering candidate behaviors by extracting and comparing the behaviors of different trained agents.

rubrics [107] or LLM judges [98].

However, we observe that even when we only use successful rollouts as demonstrations in training, the exact **behaviors** reflected in the demonstrations also substantially affect the performance of the trained model. We conduct a preliminary study under the setting of distilling a student model from the teacher's successful rollouts. As shown in Figure 6.2, on the same set of training instances and for a fixed student model, distilling from different teachers leads to substantially different performance. Although they all successfully solve the task, the teacher models may apply different problem-solving strategies, use different tools, and generate concrete actions with different patterns.

We thus would like to answer the question: *what are the demonstration behaviors that affect coding agent training performance?* Existing research presents manually defined rubrics on agent behaviors and empirically examines their impact on training [107, 123, 132]. However, the space of coding agent behaviors is vast and open-ended, ranging from high-level problem-solving strategies to low-level tool-use patterns. The influence of a behavior could also vary across training configurations and base models. Manually enumerating behavioral hypotheses lacks scalability and human expertise may be limited to specific experimental settings.

In this paper, we present **CATRadar**, an automatic analysis method to efficiently discover candidate behaviors in training demonstrations that could potentially affect **C**oding **A**gent

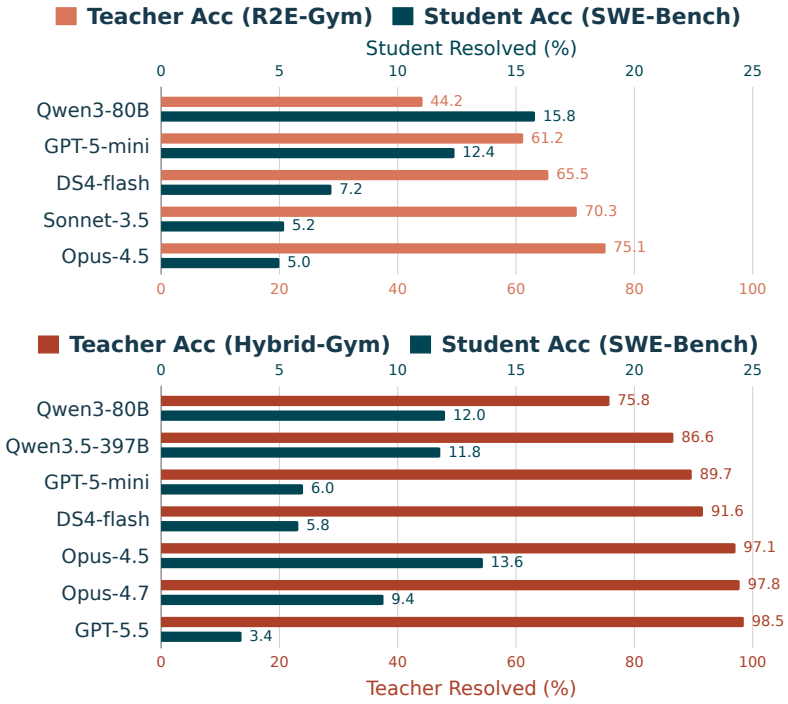


Figure 6.2: Across different training sets, on the same sets of task instances, successful demonstrations generated by different teacher models lead to substantially different performance, and higher teacher performance does not necessarily transfer to better student performance.

Training performance. We leverage an LLM agent to automatically define, extract, and compare the behaviors of coding agents trained on different data sources that have different performances. We only keep candidate behaviors that are well-defined and can distinguish the two agents we compare. After **CATRadar** narrows down the search space of behaviors, we examine the impact of those candidate behaviors through controlled interventions on the presence of each behavior in the training data.

We apply **CATRadar** to 8 teacher models and 3 student models for two training tasks: R2EGym [41] and Hybrid-Gym [123]. Guided by **CATRadar**, we examine the impact of 10 behaviors of coding agent demonstrations and 7 of them are shown to positively affect at least one model's performance. As shown in Figure 6.1, such influential behaviors include specific repo-exploration strategies, planning and verification patterns, and the verbosity of reasoning text.

Furthermore, we demonstrate that finding such influential behaviors can lead to actual performance gains by various methods, including data selection, agent step synthesis, and behavior-conditioned rollout. For instance, we demonstrate that data selection based on the repo-exploration strategies we discover improves all three student models on SWE-Bench Verified [44] (e.g., improving the resolved rate of Qwen3-8B from 4.8% to 11.6%). We also discover

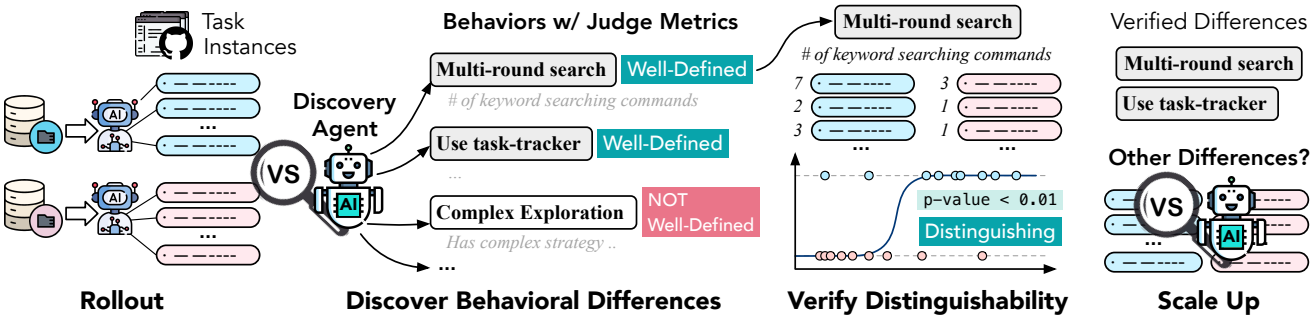


Figure 6.3: Overview of the **CATRadar** automatic behavior discovery framework (§6.2.1). We run two trained coding agents on the same task instances, use a discovery agent to propose behavioral differences with a well-defined judge metric, and verify the distinguishability of each behavior by fitting a logistic regression.

that simply adding 4 LLM-synthesized planning steps to each demonstration improves the resolved rates of all three student models (e.g., improving Qwen3-8B's resolved rate from 11.6% to 17.6%).

6.2 Analysis Method: Discovering Influential Behaviors of Demonstrations

Figure 6.2 shows that coding agents trained on demonstrations produced by different models can achieve different performances, but the success rates of the demonstrations cannot explain the performance gap. We hence hypothesize that there exist a set of properties of **how** the final results are achieved that account for the performance gap, which we denote as **influential behaviors**. Identifying such behaviors is challenging because the space of coding agent behaviors is vast, spanning from high-level problem-solving strategies to low-level implementation details. In this section, we first present **CATRadar** (§6.2.1) to automatically narrow down the space of candidate behaviors by discovering the different behaviors of coding agents with different performances. We then conduct controlled data intervention experiments (§6.2.2) to empirically examine the impact of the candidate behaviors we discovered. We released the code of the **CATRadar** framework and all the data intervention experiments we designed.

6.2.1 CATRadar: Automatic Candidate Behavior Discovery

Figure 6.3 shows the overview of **CATRadar**.

Rollout and Discover Distinguishing Behaviors. We run two coding agents on the same set

of task instances and provide their trajectories, paired by instance, to a discovery agent (e.g., Claude-Opus-4.7). We prompt the discovery agent to discover different *behaviors* between the two agents, where each behavior is defined by a judge metric that specifies a rule for computing either a numerical or a categorical value for each trajectory. Table 6.1 presents examples of behaviors and their associated judge metrics. We only retain the behaviors with a **well-defined** judge metric that can either be computed by a deterministic script or scored by an LLM judge with high consistency. Both are generated by the discovery agent and manually examined. We check consistency by applying the LLM judge to the same set of examples for multiple times.

Verify Distinguishability. Given a candidate behavior and its associated judge metric, we compute the metric score for each trajectory based on its definition. Then we treat each trajectory as one observation and fit a separate logistic regression, with agent identity (i.e., the agent that produces the trajectory) as the dependent variable and the judge metric score as the predictor, where we encode categorical values with indicator variables. We label $N = 50$ trajectories for each agent and consider the behavior to be **distinguishing** if the Wald test rejects the null hypothesis with p-value< 0.01.

Scale Up. We add all well-defined behaviors that pass the distinguishability test to the candidate behavior list and repeat the discovery-and-verification procedure for at most $k = 5$ iterations per task set and agent pair. In each subsequent iteration, we provide the discovery agent with all existing behaviors and prompt it to identify additional differences. We repeat this procedure across multiple training tasks and agent pairs. We manually review and prioritize the retained behaviors for the data intervention experiments (§6.2.2) to examine their impact on the training performance.

6.2.2 Data Intervention Methods

This section aims to isolate and measure the impact of each candidate behavior discovered in §6.2.1 on the training performance. We introduce two complementary intervention methods that control the presence of a target behavior while minimizing changes to other properties of the demonstrations. For both methods, we consider a behavior to be **influential** if the presence or absence of the behavior leads to a substantial performance change.

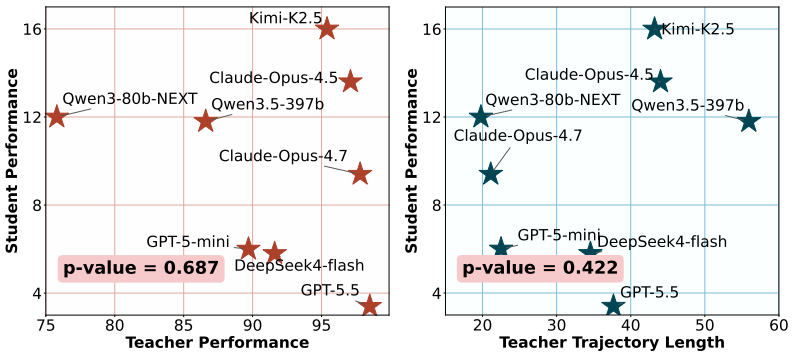
Trajectory Post-Editing. This method adds, modifies, or removes one or more trajectory steps to control the presence of a target behavior, which can be performed by deterministic rules or an LLM. One example is to remove the "verify after edit" behavior (as defined in Table 6.1) from a trajectory by identifying the final file-editing action by pattern matching and removing all subsequent steps.

Behavior	Judge Metric	LEGO	Hybrid	Others	Ours
Multi-round search	Run keyword searching commands for ≥ 2 times				✓
Has fine-grained keywords	Search for fine-grained keywords (e.g., specific filenames) instead of only broad keywords (e.g., `def` or `class`)				✓
Limit target files before read	Limit the candidates to ≤ 30 files before reading any file				✓
Use task-tracker	Use the `<function=task_tracker>` tool for explicit reasoning				✓
Verify after edit	Has verification steps after the final file-editing action	✓			✓
Use think	Use the `<function=think>` tool for explicit reasoning				✓
Reasoning text verbosity	The length of the text part outside tool calls		✓		✓
Stuck in loop	Try different commands after failing for 3 times	U			U
Exploration length	The number of actions before the first file-editing action (≥ 16: high; 6–15: medium; ≤ 5: low)				U
Trajectory length	Has ≤ 7 agent actions: short; ≥ 8 agent actions: long	U	✓	✓	U

Table 6.1: Some of the behaviors discovered and examined by our framework. We present the full list of candidate behaviors in §A.4. We use "✓" to indicate that the behavior is shown to be relevant to the final performance for some student models in related works and use "U" to indicate that the behavior is tested but not shown to be relevant. "LEGO" [132], "Hybrid" [123], and "Others" refer to existing works.

One major challenge is to only conduct edits that preserve *trajectory consistency*: every retained step must remain supported by its preceding context. Here we provide a non-exhaustive list of classes of edits that preserve consistency: (1) Interchanging two sequences of steps that produce the same intermediate results that are crucial for resolving the task (e.g., two search sequences that both locate the target file); (2) Removing steps whose outputs and state effects are not required by subsequent actions, such as unsuccessful searches whose results do not appear in subsequent actions; and (3) Modifying steps that neither change the environment state nor provide information to subsequent actions, including steps that contained only chain-of-thought reasoning text and actions that are not executed due to wrong syntax.

Behavior-Conditioned Trajectory Generation. For behaviors that are difficult to manipulate through local edits, we generate new demonstrations for the same task instances by prompting the agent to exhibit the target behavior. To limit differences unrelated to the target behavior, we retain an original-generated trajectory pair only if both trajectories successfully solve the task and reach the same set of essential intermediate results, which are pre-defined based on the task. For example, in the issue-solving task, we require the original and newly generated trajectories to identify the exact same set of target files and pass all the unit tests.



(a) Teacher performance (b) Trajectory length

Figure 6.4: Student performance (Qwen2.5Coder-7B) is not significantly correlated with trajectory-level features such as teacher success rate or trajectory length.

6.3 Analysis Results: What Behaviors Affect Coding Agent Training?

This section presents the analysis results of the **CATRadar** method. As a proof of concept, we focus on the distillation setting. In principle, **CATRadar** can also be applied to other settings such as self-training or reinforcement learning.

Experimental Setup. We conduct analysis on both in-domain training: the R2E-Gym issue-solving dataset [41], and transfer learning: the Hybrid-Gym function localization dataset [123], with SWE-Bench Verified [44] as the evaluation dataset. We adopt the distillation setting and apply **CATRadar** to 8 teacher models and 3 student models, as described in §A.2. We use the OpenHands agent harness [106] for all the experiments. Since the impact of one behavior may depend on other factors such as other teacher behaviors or the student's capabilities, when we conduct data intervention experiments and compare the performance with the original data (§6.2.2), we experiment on multiple training tasks, teacher models, and student models (if applicable).

6.3.1 CATRadar Discovers Diverse Candidates

Table 6.1 summarizes the candidate behaviors discovered by **CATRadar** and examined in our experiments, along with their definitions. Compared with the behaviors that are manually hypothesized and examined in prior work, **CATRadar** discovers a broader range of behaviors of diverse granularity, spanning trajectory-level features, problem-solving strategies, and tool-use patterns. Our experiments show that 7 of the 10 behaviors affect student performance in

[Case 1] Multi-Search: ✓ Target-Read: ✗ Suff-Explore: ✓
`grep -r ‘‘def ’’ -include=’*.py’ .` → **1771** files Search for keywords “model” → **105** files Read `core/optimizers/function_marker.py`...
[Case 2] Multi-Search: ✗ Target-Read: ✓ Suff-Explore: ✓
`grep -r ‘‘node’’ -include=’*.py’ .` → **6** files Read `core/optimizers/function_marker.py`...
[Case 3] Multi-Search: ✓ Target-Read: ✓ Suff-Explore: ✓
`grep -r ‘‘model’’ -include=’*.py’ .` → **105** files `grep -r ‘‘node’’ -include=’*.py’ .` → **6** files Read `core/optimizers/function_marker.py`...

Table 6.2: A case study showing that demonstrations with different repo-exploration strategies can all be labeled as “sufficient exploration” by the LLM judge.

at least one combination of student model, teacher model, and training dataset. Some of these findings are also consistent with prior work. For instance, Terminal-LEGO [132] shows that verification steps in training improve performance. **CATRadar** also discovers “verify after edit” as a potentially important behavior.

6.3.2 Trajectory-Level Features are Not Good Indicators of Training Effectiveness

Longer trajectories are commonly viewed as more useful training examples because they provide the model more information [145]. Under the setting of distillation, it is also a common belief that strong models often produce stronger students [84].

Our results challenge both assumptions. As shown in Figure 6.4 and Figure A.4, none of the trajectory-level features we examine is significantly correlated with student performance. Our hypothesis is that longer trajectories may reflect overly complicated problem-solving strategies, which may not be suitable for the student model. Similarly, a stronger teacher model’s strategies may rely on its sophisticated context understanding abilities, which are difficult to transfer to the student model, making the learned strategies less effective.

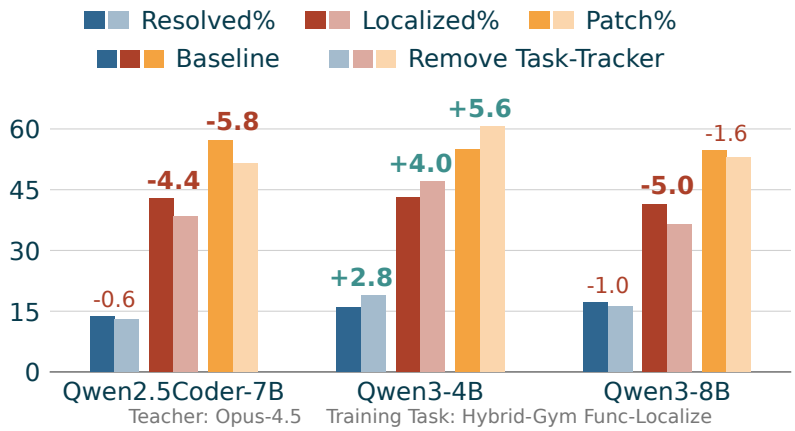


Figure 6.5: Planning steps in demonstrations have different effects on different models. We **highlight** significant performance changes after removing all task-tracker actions from demonstrations (with p-value < 0.05).

Teacher Model (Hybrid-Gym)	Teacher	Qwen2.5Coder-7B				Qwen3-4B				Qwen3-8B			
	# Verify	# Verify	Res%	Loc%	Patch%	# Verify	Res%	Loc%	Patch%	# Verify	Res%	Loc%	Patch%
Opus-4.5	**4.4**	**13.3**	**13.6**	**42.8**	**57.2**	**11.6**	16.0	43.0	55.0	**14.4**	**17.2**	**41.4**	**54.6**
Remove Verify	0.0	8.9	11.0 (-2.6)	39.6	53.0	7.0	**18.6** (+2.6)	**46.4**	**60.2**	7.7	15.8 (-1.4)	37.4	50.9
Opus-4.7	1.1	4.3	9.4	32.8	47.8	4.7	12.2	33.4	46.8	9.3	11.6	30.2	45.4
Add Verify	**4.2**	**11.2**	**10.8** (+1.4)	**34.0**	**54.2**	**9.9**	**13.4** (+1.2)	**38.4**	**50.8**	**15.2**	**15.0** (+3.4)	**36.2**	**50.8**

Table 6.3: Verification procedure in a simple training task (e.g., function localization) transfers to more complex tasks (e.g., issue-solving). However, the effect of verification steps depends on the student and teacher model. We highlight significant increases and decreases in resolved rates (with p-value < 0.05).

6.3.3 CATRadar Discovers Effective Problem-Solving Strategies

Prior work shows that demonstrations should include a non-trivial repository-exploration stage and leverage an LLM judge to check whether it conducts sufficient exploration [123]. However, it does not specify *how* the exploration is conducted. Instead, **CATRadar** goes a step further by identifying two concrete exploration strategies: **Multi-Search**, which searches for multiple keywords iteratively, and **Target-Read**, which only reads a file after narrowing down the target to a few files. As shown in Table 6.2, the two strategies we discovered capture more fine-grained information than the high-level rubric used in prior works.

We think these two strategies are potentially useful for training for the following reasons: by having multiple search actions in the demonstrations, the agent is trained to try different keywords if the previous ones are too broad or fail to match any files. "Target-Read" teaches the model not to blindly read files with low relevance, which improves the efficiency of localizing the target file. We will show in §6.4.1 and §6.4.3 that demonstrations containing these specific strategies improve training performance.

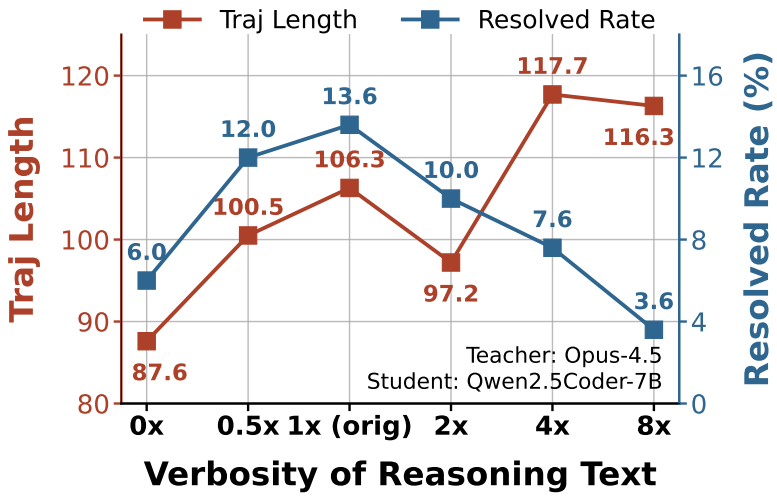


Figure 6.6: When the verbosity of reasoning text increases (including non-tool-call text and content of the "think" actions), the trajectory length in general increases, and the performance first increases and then decreases.

6.3.4 Planning Steps have Different Effects on Different Models

CATRadar discovers that the planning pattern also vary across different demonstrations. For instance, OpenHands has a "task-tracker" tool that creates and keeps a to-do list for the task, which is frequently used by some agents but not others. Intuitively, the task-tracker tool assists high-level planning and plays a similar role as chain-of-thought reasoning [111], which conducts explicit reasoning to guide the agent's decision-making process.

To examine whether the usage of task-tracker affects the performance, we directly remove all the task-tracker actions from the demonstrations and train the model. We additionally query an LLM to rewrite the text that explicitly refer to the removed steps to maintain the consistency of the trajectory.

However, results in Figure 6.5 show that having more planning steps in training does not always improve performance. On the demonstrations generated by Claude-Opus-4.5, the presence of task-tracker steps improves the performance of Qwen2.5Coder-7B and Qwen3-8B, while degrading the performance of Qwen3-4B. One possible explanation is that the 4B model has lower capacity and could benefit more from simpler and more straightforward demonstrations, while the larger models can better exploit complex reasoning patterns. We will further show in §6.4.2 that inserting LLM-synthesized task-tracker steps improves both the 7B and 8B models.

6.3.5 Verification Abilities Transfer from Simple Tasks to Complex Tasks

Verification is an important stage for coding agent tasks, where the agent examines its solution after it is generated. It can be complex for some tasks (e.g., issue-solving, where the agent needs to execute unit tests) but simple for others (e.g., function localization, where the agent only needs to read files and conduct reasoning). **CATRadar** discovers that the average number of verification steps varies across different teacher models.

To examine the effect of verification steps in training, we design the following data intervention experiments to control the presence of verification steps in training demonstrations: (1) "Remove Verify", where we remove all the steps after the last file-editing action, and (2) "Add Verify": for demonstrations without any verification steps, we copy the verification steps from the demonstration of the same task instance generated by another agent (e.g., Claude-Opus-4.5). Other demonstrations are kept unchanged.

Surprisingly, as shown in Table 6.3, we observe that the verification behavior transfers across different tasks. When training the agent on function localization demonstrations with more verification steps, it also learns to perform more verification steps on the issue-solving task.

However, similar to the planning behavior, we observe that verification improves the performance of the 7B and 8B models, but does not consistently improve Qwen3-4B. This can also be explained by the hypothesis that models with lower capacity could benefit more from simpler demonstrations.

Our results make a further step beyond the finding in Terminal-LEGO [132], which shows that the inspect-act-verify pattern in training trajectories improves coding agents' performance on terminal tasks. We further show that the impact of verification also depends on other factors such as properties of the student model.

6.3.6 Training on Reasoning Text Increases Trajectory Length

CATRadar discovers that the length of the text outside tool calls varies across different teacher models, which is on average 39 tokens for Claude-Opus-4.5, 98 for Qwen3-80B-NEXT, and 298 for GPT-5-mini on Hybrid-Gym.

To examine the effect of the verbosity of the reasoning text, we design an experiment that prompts an LLM to rewrite the reasoning text to reduce or increase the verbosity proportionally to its original length without adding new information. We define "reasoning text" as the combination of (1) the non-tool-call text and (2) the content of the "think" actions. We conduct post-checking with an LLM checker to ensure the rewritten text is natural and does not

Teacher Model	SWE-Bench Verified								
	Qwen2.5Coder-7B			Qwen3-4B			Qwen3-8B		
	Res%	Loc%	Patch%	Res%	Loc%	Patch%	Res%	Loc%	Patch%
***Training**: R2E-Gym Issue-Solving*									
GPT5-mini Subsets (250 examples)									
w/o Multi-Search	9.0	34.4	52.4	10.0	43.8	62.2	10.2	43.8	60.4
w/o Target-Read	8.6	36.2	52.0	13.2	48.8	62.4	10.4	42.4	56.6
Both Strategies (ours)	**11.4** (+2.8)	**49.4**	**63.6**	**14.2** (+4.2)	**58.4**	**75.4**	**13.8** (+3.6)	**48.2**	**63.2**
***Training**: Hybrid-Gym Func-Localize*									
Claude-Opus-4.5 Subsets (250 examples)									
w/o Multi-Search	2.8	13.6	31.6	11.8	**38.0**	**51.4**	4.8	15.2	29.2
w/o Target-Read	5.4	17.6	33.0	8.0	16.6	44.0	5.8	15.4	30.4
Both Strategies (ours)	**7.2** (+4.4)	**21.2**	**39.0**	**13.2** (+5.2)	35.4	50.8	**11.6** (+6.8)	**28.4**	**41.8**
Claude-Opus-4.7 Subsets (350 examples)									
w/o Multi-Search	2.8	15.8	42.8	7.2	**27.5**	**40.0**	8.6	26.8	**40.2**
w/o Target-Read	3.4	9.4	25.0	7.6	20.6	33.6	4.2	11.8	26.8
Both Strategies (ours)	**6.0** (+3.2)	**27.2**	**61.4**	**9.6** (+2.4)	25.8	36.8	**10.2** (+6.0)	**27.2**	39.4

Table 6.4: Data selection results with the repo-exploration strategies we discovered. Parenthesized results are absolute gains compared to the lower resolved rate of the two ablations. We highlight gains for resolved rates that are significantly better than both ablations and one of the ablations (with p-value < 0.05).

add new information. We also include a "0x" ablation that removes all reasoning text.

Surprisingly, Figure 6.6 shows that increasing the verbosity of the reasoning text increases the trajectory length. Our hypothesis is that more verbose reasoning encourages the model to articulate additional intermediate considerations, which in turn induces extra tool interactions. Similarly, we observe in Figure A.5 that when we completely remove the non-tool-call text but keep the "think" actions, the trajectory length still substantially decreases.

6.4 Practical Utility of Our Discoveries

We apply the discoveries in §6.3 through three methods, ordered by increasing data-construction cost: selecting demonstrations with beneficial behaviors (§6.4.1), augmenting existing demonstrations with synthesized steps (§6.4.2), and generating new demonstrations with influential behaviors (§6.4.3).

6.4.1 Method 1: Behavior-Aware Data Selection

Data Selection. Given a fixed training-data budget, we select demonstrations that exhibit beneficial behaviors identified by **CATRadar**. Following §6.3.3, we retain demonstrations that exhibit both the **Multi-Search** strategy (at least three keyword searches) and **Target-Read**

	Teacher (Hybrid-Gym Func-Loc)			Student (SWE-Bench Verified)				
Teacher Model	**Multi-Search%**	**Target-Read%**	**Res%**	**Multi-Search%**	**Target-Read%**	**Res%**	**Loc%**	**Patch%**
GPT-5-mini	58.8	1.5	**89.7**	74.9	53.2	6.0	44.6	66.2
+ prompt student	58.8	1.5	**89.7**	75.6	57.6	7.2 (+1.2)	**48.2**	**67.0**
+ prompt teacher	**72.5**	**10.4**	85.9	**78.2**	**64.3**	**9.2** (+3.2)	43.4	55.6

Table 6.5: Student performance (Qwen2.5Coder-7B) on SWE-Bench Verified under different teacher models and prompting strategies. We highlight significant changes in resolved rates.

Training Setting (Teacher → Student)	**SWE-Bench Verified**		
	Res%	**Loc%**	**Patch%**
***Training**: Hybrid-Gym Func-Localize*			
GPT-5-mini → Qwen2.5Coder-7B	6.0	**44.6**	**66.2**
Add task-tracker	**9.6** (+3.6)	33.2	48.4
GPT-5-mini → Qwen3-8B	10.8	**56.6**	**71.6**
Add task-tracker	**13.0** (+2.2)	49.2	61.6
Opus-4.7 → Qwen2.5Coder-7B	9.4	32.8	47.8
Add task-tracker	**12.2** (+2.8)	**38.6**	**55.2**
Opus-4.7 → Qwen3-8B	11.6	30.2	45.4
Add task-tracker	**17.6** (+6.0)	**40.6**	**53.0**

Table 6.6: We can improve performance by adding synthetic task-planning steps in training trajectories. We highlight significant improvements in resolved rates.

(narrow down to $\leq$ 50 candidate files before reading). We compare against equally sized subsets lacking either strategy: "w/o Multi-Search" and "w/o Target-Read".

Results. Table 6.4 shows that selecting demonstrations with both strategies consistently yields higher resolved rates than either ablation across the evaluated settings. For example, training Qwen3-8B on 250 Claude-Opus-4.5 function-localization demonstrations with both strategies achieves 11.6% on SWE-Bench Verified, compared with 4.8% without Multi-Search and 5.8% without Target-Read. A possible explanation is that agents trained on the "w/o Multi-Search" ablation do not learn to recover from unsuccessful searches, and the "w/o Target-Read" ablation does not teach the strategy to progressively narrow down the candidate files. One common failure case for agents trained on "w/o Target-Read" is to repeatedly read irrelevant files until exhausting their action budget, without a more efficient strategy to locate relevant files.

6.4.2 Method 2: Adding Synthesized Agent Steps to Training Trajectories

Agent Step Synthesis. Another method is to augment existing demonstrations by inserting LLM-synthesized steps that instantiate a beneficial behavior (e.g., has high-level planning). Particularly, we prompt an LLM to generate a task-tracker call for each function localization

demonstration that specifies a to-do list: identify the target file, locate the function, generate its docstring, and verify correctness. We insert subsequent task-tracker calls at completion points to update the corresponding items. Implementation details are provided in §A.6.

Results. As shown in Table 6.6, adding four synthesized task-tracker actions per demonstration improves the resolved rates of Qwen2.5Coder-7B and Qwen3-8B in all evaluated settings. For Qwen3-8B trained on Claude-Opus-4.7 demonstrations, the resolved rate increases from 11.6% to 17.6%. These steps may provide explicit planning structure that helps the student organize subsequent actions. Note that we train on the function localization task and improve issue-solving performances, suggesting that the planning behaviors transfer across tasks.

6.4.3 Method 3: Behavior-Conditioned Demonstration Generation

Prompt Generation. For behaviors that are difficult to incorporate through local edits, we generate new demonstrations using a behavior-conditioned teacher prompt (§6.2.2). We iteratively (1) ask an agent to write a prompt to elicit the target behaviors, (2) generate a small set of demonstrations, and (3) use an LLM judge to label their presence. We repeat this procedure until the generated prompt significantly increases the proportion of demonstrations exhibiting the target behaviors ($p < 0.05$), then use it to generate the training set.

Results. We apply this method to the two repo-exploration strategies we discovered in §6.3.3: Multi-Search and Target-Read. Table 6.5 shows that by generating demonstrations with the LLM-generated prompt, we increase the prevalence of the strategies for both the teacher and the student models. Furthermore, training Qwen2.5Coder-7B on the resulting demonstrations significantly improves its resolved rate on SWE-Bench Verified from 6.0% to 9.2%. We also observe that directly prompting the student to follow the same strategies ("prompt student") yields a smaller gain, from 6.0% to 7.2%, suggesting that the student model has limited instruction-following capabilities to incorporate the strategies as prompted, and prompting the teacher model is more effective in this setting.

6.5 Conclusions

We present **CATRadar**, an efficient analysis method, to discover potentially influential behaviors for training coding agents. Guided by **CATRadar**, we conduct data intervention experiments and find influential behaviors including specific repo-exploration strategies, the planning and verification patterns, and the verbosity of reasoning text. Our insights further

lead to practical improvements through data selection, agent step synthesis, and behavior-conditioned rollout, with step synthesis improving Qwen3-8B's resolved rate from 11.6% to 17.6% on SWE-Bench Verified. Beyond the supervised finetuning setting studied here, we envision **CATRadar** supporting behavior discovery in other training settings including self-training and reinforcement learning, providing insights for why coding agent training fails or succeeds and guiding the design of the training process.

Part III

Training Auxiliary Models to Assist Coding Agents

Overview of Part III. In Part I and Part II, we aim to construct synthetic data to improve coding agent effectiveness. However, empirically, we still observe that the performance satuates with respect to data scale after a certain point, and the high-level planning ability remains a major bottleneck.

To address this limitation, in this part, we develop a critic model to provide guidance to the coding agent at inference time. While the main coding agent is trained to jointly balance multiple abilities, including high-level planning and low-level implementation, the critic model is specialized for planning. Since the optimum for jointly optimizing multiple abilities does not necessarily align with the optimum for planning, training a specialized critic model may lead to stronger planning performance and, in turn, more effective guidance for the coding agent.

To demonstrate the utility of critic model, in chapter 7, we show that a critic model can improve the factuality of a generator model by either interacting with it in inference or providing feedback to assist training. We further improve the effectiveness of the critic model by constructing tailored training data.

When it comes to the domain of coding agent, in chapter 8, we first show that using a frontier model (e.g., Claude) to provide critique during inference substantially improves the correctness of a coding agent. Then we distill a more cost-efficient critic model by constructing synthetic supervised-finetuning training data. To further improve its effectiveness, we propose a method to construct direct preference optimization (DPO) training data, focusing on its gap between the frontier critic model.

Chapter 7

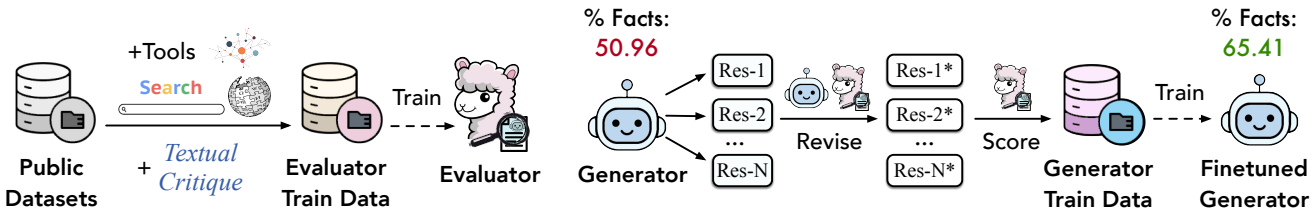
Improving Generator Factuality by Training a Critic Model

7.1 Introduction

Hallucination is one of the persistent challenges for large language models (LLMs), where models generate plausible-sounding but incorrect information, even if they are shown factual information during pretraining [56, 142]. One hypothesis is that LLMs fail to distinguish the boundary between memorized facts and other plausible-sounding information and do not learn to only output memorized facts, especially on their unfamiliar topics [27, 29, 47]. Although it is possible to reduce hallucination in inference with decoding strategies [15, 57] or post-editing [46, 70], they introduce severe latency issues and hurts efficiency in real-time applications.

Alternatively, prior studies train the generator to output more factual responses, by preferring (1) generation candidates with higher factuality scores [103], which is limited by the generator's capabilities, or (2) responses with false information corrected [46], which is prone to introducing lesser-known facts. As shown in recent work [27, 29], such preference training reinforces the model to generate information not well memorized during pretraining, and hence could even hurt factuality. Furthermore, the methods either leverage proprietary models that have restricted terms of use, or prompt the generator to evaluate its own factuality, which suffers from self-bias and leads to inaccurate judgments [125].

Recent work trains evaluator models that could potentially be used to provide training signals for generators. One category of work relies on proprietary models (e.g., GPT-4) to generate training data in various formats [49, 55]. In contrast, Vu et al. [104] leverage public datasets containing judgments of whether a claim is factual against certain source documents.



(a) Evaluator Training

(b) Using Evaluator to Improve Generator Factuality

Figure 7.1: (*Left*) The framework to train an evaluator, FENCE, by augmenting public datasets with textual critiques and more diverse knowledge sources. We show the details in Figure 7.2. (*Right*) The framework to improve model factuality with FENCE. We construct training data by leveraging FENCE to revise and score the generator's responses. Details of response revision are shown in Figure 7.3.

However, such documents are generally sampled from very **restricted sources** such as news corpora or Wikipedia, while an evaluator could potentially benefit from knowledge obtained by a multiplicity of tools (e.g., search engines) [112]. Furthermore, the judgment label in most datasets is a single binary or numeric score, providing **limited feedback** to the generator model.

In this paper, we present FENCE, a **Fin**e-grai**n**ed **C**ritic-based **E**valuator that aims to provide textual critiques for each model-generated claim based on diverse knowledge sources. We start with a set of public datasets with human judgments on the factuality of model-generated claims. As shown in Figure 7.1(a), we augment the judgment labels with textual critiques, which provide more informative and explainable feedback to the generator. In addition, we augment the source documents by invoking multiple tools, including a search engine, knowledge base, and knowledge graph, with the goal of training the evaluator to leverage more diverse knowledge sources.

We further demonstrate how to leverage FENCE to improve the generator's factuality with finetuning. To construct training data (Figure 7.1(b)), we generate multiple responses for each prompt, use FENCE to judge and critique every claim in the responses, and replace false information with facts or remove it from the response, depending on whether the corresponding fact is within the generator's knowledge (i.e., whether the generator outputs *"unknown"* when prompted about claim correctness). This mitigates introducing lesser-known facts into training data. Finally, unlike existing work that use the generator itself as the evaluator, we use FENCE to score each original and revised response and construct preference data, which reduces self-bias and produces more accurate judgments.

Experiments show that FENCE outperforms large open-source models such as Mistral-123B

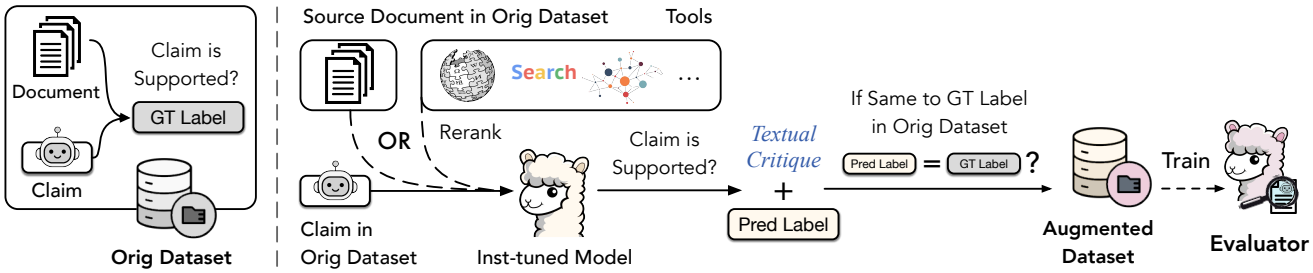


Figure 7.2: Framework of evaluator training. (*left*) Existing public datasets for evaluator training. Each example contains a claim, a source document, and the ground truth (GT) label of whether the claim is supported by the document. (*Right*) We augment the datasets with textual critiques and more diverse source documents obtained by tools. We use zero-shot Llama-3-70B-chat as the instruction-tuned model in our experiments.

and strong proprietary models such as Claude-3 on LLM-Aggrefact. With FENCE, we improve Llama2-7B-chat's factuality by 16.86% on FActScore [68] and 17.64% on TruthfulQA [59], outperforming existing factuality finetuning methods by 8.83% and 3.99%, respectively. Analyses further show that after training with our recipe, the generator outputs less information for unfamiliar entities and more information for popular ones, suggesting that it learns to only generate information that is likely to be factual.

Contributions. (1) We train a fine-grained critique-based evaluator, FENCE, by augmenting public datasets with textual critiques and more diverse source documents. (2) We propose a training recipe to improve the generator's factuality by leveraging FENCE to improve and score its responses. (3) We conduct extensive experiments to validate both the judgment accuracy of FENCE and the factuality of the generator trained with FENCE, outperforming state-of-the-art factuality training methods by 8.83% on FActScore and 3.99% on TruthfulQA.

7.2 Methodology

7.2.1 FENCE: Factuality Evaluator Training

Preliminary. To train the evaluator to recognize hallucinations, previous work [104] incorporates a combination of datasets with human judgments on factuality. As shown in Figure 7.2, in most datasets, each example contains a claim, a source document, and the ground-truth judgement of whether the claim is (1) fully supported by, (2) contradicted with, or (3) contains information that cannot be verified by the document.

We formally define the problem as follows: Given a claim $c \in \mathcal{C}$, a source document $d \in \mathcal{D}$, a

factuality evaluator aims to learn a mapping $f : \mathcal{C} \times \mathcal{D} \rightarrow \mathcal{L}$, which maps each claim-document pair (c, d) to one of the labels: $f(c, d) = l \in \mathcal{L} = \{$*Supported*, *Contradictory*, *Unverified*$\}$.

Augmenting Labels with Textual Critiques. In addition to the classification label, we aim to train the evaluator to generate a *textual critique* that explains the judgement, which provides more informative feedback such as which part of the document supports or contradicts the claim. We will leverage such feedback to revise generator responses and use them to train a more factual generator (see §7.2.2 for details). Formally, we aim to learn the mapping $f : \mathcal{C} \times \mathcal{D} \rightarrow \mathcal{R} \times \mathcal{L}$, which maps each claim-document pair (c, d) to both the textual critique $r \in \mathcal{R}$ and the label $l \in \mathcal{L}$.

As shown in Figure 7.2, we prompt an instruction-tuned model $\mathcal{M}$ (e.g., Llama3-70B-chat) to generate both the critique $r_{\mathcal{M}}$ and label $l_{\mathcal{M}}$ for whether a claim c is supported. The critique and label are likely to be consistent because the label is generated conditioned on the critique. As a result, if the predicted label $l_{\mathcal{M}}$ aligns with the ground truth label l_{GT} in the original dataset, the critique is also likely to be aligned and we hence use both the critique $r_{\mathcal{M}}$ and label $l_{\mathcal{M}}$ as the new training target. Otherwise, if the predicted label does not align with the ground truth, we discard the whole example.

Augmenting Source Documents using Tools. To judge the factuality of an arbitrary model-generated claim, we could potentially benefit from *a multiplicity of tools* such as search engines or online knowledge bases. However, the source documents in existing judgment datasets typically come from very restricted sources, such as news corpora or Wikipedia. To bridge this gap, we obtain additional source documents for each claim by calling the following tools: a search engine (Bing Search API), knowledge base (Wikipedia), and knowledge graph (Google Knowledge Graph API).

As shown in Figure 7.2, given the claim c in the original dataset, we prompt an instruction-tuned model to call multiple tools to verify the factuality of the claim (i.e., by generating tool calls such as search queries). Then we rerank the returned results to obtain a combination of tool-extracted documents d_t. Similar to critique generation, we prompt the instruction-tuned model $\mathcal{M}$ to predict whether claim c is supported by the tool-extracted documents d_t. We add d_t to the train set if the predicted label $f_{\mathcal{M}}(c, d_t)$ is the same as the ground truth label $f_{GT}(c, d)$ in the original dataset.

The intuition is that if a claim can be supported by some documents, it is likely that we can obtain other supporting sources by calling the tools. If a claim is hallucinated, it is very unlikely to find any knowledge that supports it with any tools. In both cases, we have $f_{GT}(c, d_t) = f_{GT}(c, d)$.

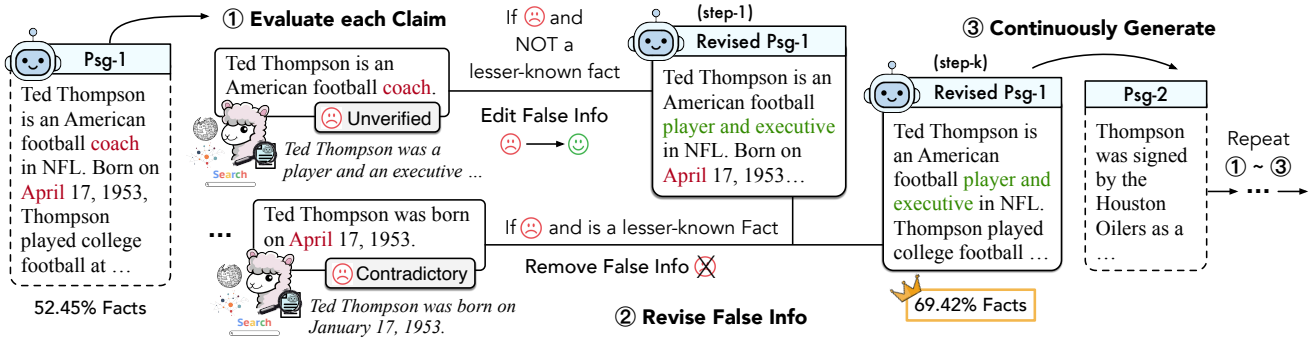


Figure 7.3: The framework to revise model responses without introducing lesser-known facts. We iteratively (1) use FENCE to evaluate the factuality of each claim, (2) replace false information (if any) with the correct fact or remove it from the response, depending on whether it corresponds to a lesser-known fact, and (3) continue generating the next passage. For every claim, we prompt the generator *"Is this claim factual?"* without providing any source documents. If the generator outputs *"unknown"*, we assume that the claim corresponds to a lesser-known fact.

Training Objective. After obtaining the augmented training data $\mathcal{TR}_{Eval} = \{(c, d), (r, l)\}$, where each example contains (claim c, source document d, critique r, label l), we initialize the evaluator $\mathcal{E}$ with a instruction-tuned model and train it with a standard conditional language modeling objective, maximizing likelihood:

$$\max_{\mathcal{E}} \mathbb{E}_{(c,d),(r,l)\sim\mathcal{TR}_{Eval}} \log \mathbb{P}_{\mathcal{E}}(r, l \mid c, d). \tag{7.1}$$

7.2.2 Improving Generator Factuality with FENCE

In this section, we use our evaluator, FENCE, to improve a generator model's factuality, where we construct training data by revising and scoring the generator's own responses. Compared to directly training the generator on factuality datasets, our method only requires a prompt set as inputs and hence enjoys much better scalability.

Overview. As shown in Figure 7.1, given a prompt, we use the generator to generate N candidate responses. Then we improve the factuality of each response by using FENCE to evaluate the factuality of each piece of generated information and editing or removing the false information, depending on whether the corresponding fact is rare. Finally, we use FENCE again to score all original and revised responses and construct training data.

Response Revision. We aim to improve the factuality of the generator's responses *without*

introducing lesser-known facts to training data. The motivation is that as shown in recent research [27, 29], forcing the model to generate lesser-known facts that are poorly memorized during pretraining will blur the boundary with memorized facts and other plausible sounding information, which may lead to even more hallucination. As shown in Figure 7.3, we iteratively revise each passage with the following three steps:

[**Step-① Evaluate**] we prompt an instruction-tuned model (e.g., Llama3-70B-chat) to decompose each response into claims. Then for each claim, we call tools to obtain related documents and apply FENCE to evaluate its factuality with critique.

[**Step-② Revise**] If there are any claims that are judged as *"unverified"* or *"contradictory,"* we further check whether it corresponds to a lesser-known fact. Specifically, we prompt the generator *"Is this claim factual?"* and output *"true," "false,"* or *"unknown,"* without providing it any external knowledge. We regard the claim as a lesser-known fact if the generator outputs *"unknown."*

If the claim does not correspond to a lesser-known fact, we prompt the generator to correct the false information based on the critique generated by FENCE. Otherwise, we prompt the generator to remove the false information from the passage.

[**Step-③ Generate**] To reduce error propagation, we use the revised passages as the prefix and continuously generate the next passage.

Generator Training. We use FENCE to score each original and revised response by computing the percentage of factual claims. Then we train the generator with first supervised finetuning (SFT) and then direct preference optimization (DPO) [90]. In the SFT stage, we train the generator with the top-k responses as targets, where the responses are ranked by the percentage of factual claims, optimizing with the conditional language modeling objective, which is similar to Equation 7.1.

In the DPO stage [90], we construct preference data $\mathcal{TR}_{Gen} = \{x, y_w, y_l\}$ as follows: for each prompt x, we choose the preferred response y_w from the top-k responses and choose any responses with lower scores than y_w as the rejected response y_l. Suppose we have N original and N revised responses, this gives us $\binom{2N}{2} - \binom{2N-k}{2}$ preference pairs. We initialize the generator from the SFT checkpoint and optimize the following classification loss:

$$\max_{\mathcal{G}} \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{TR}_{Gen}} \left[\log \sigma \left(\beta \log \frac{\pi_{\mathcal{G}}\left(y_w \mid x\right)}{\pi_{\text{ref}}\left(y_w \mid x\right)} - \beta \log \frac{\pi_{\mathcal{G}}\left(y_l \mid x\right)}{\pi_{\text{ref}}\left(y_l \mid x\right)} \right) \right], \tag{7.2}$$

where σ is the Sigmoid function. The reference policy π_{ref} is computed by the SFT check-

point.

7.3 Experiments on Evaluator Training

In this section, we aim to answer the research question: (**RQ1**) Can FENCE correctly judge the factuality of model-generated claims?

7.3.1 Experimental Setup

Training Details. We initialize FENCE from Llama3-8B-chat and train it on a set of public factuality judgment datasets, which Vu et al. [104] is trained on. To ensure label accuracy, we focus on datasets with human judgment on model responses, including summarization datasets: XSum Hallucination [66], QAGS [105], FRANK [83], question-answering datasets: RAGTruth [73], FActScore [68], and dialogue datasets: Q^2 [33], FaithDial [20], BEGIN [21].

Evaluation Dataset and Metric. We evaluate the evaluators on the LLM-AggreFact benchmark [99], a combination of 10 datasets covering three tasks: fact verification, summarization, and long-form QA. All datasets contain human-annotated (document, claim, label) tuples.

We follow Tang et al. [99] and use balanced accuracy (BAcc) as the evaluation metric: $\text{BAcc} = \frac{1}{2}\left(\frac{\text{TP}}{\text{TP+FN}} + \frac{\text{TN}}{\text{TN+FP}}\right)$, where TP, TN, FP, and FN represent true/false positives/negatives.

Baselines and Ablations. We compare to the LLM-based fact-checkers reported by Tang et al. [99]. We do not include results reported in Vu et al. [104] because they use a different metric.

In addition, we compare to two ablations: (1) FENCE (*Vanilla SFT*), which is trained on the original public datasets with no augmentation, and (2) FENCE (*Critique Only*), where we only generate textual critiques and not source documents.

7.3.2 Main Results

As shown in Table 7.1, after training on our augmented datasets, FENCE improves Llama3-8B-chat by 8.3% BAcc and outperforms all the open-source LLMs, including models with significantly more parameters (e.g., Mistral-123B). It also outperforms strong proprietary models such as Claude-3 Opus. When compared with its ablations, FENCE consistently outperforms the Vanilla SFT model on 8 out of 10 datasets, with average gain of 2.9% BAcc. The performance of FENCE (*Critique Only*) is between Vanilla SFT and the full model, which indicates the utility of both augmentation methods.

Model Name	LLM-AggreFact (*without* threshold tuning)										
	AggreFact		TofuEval		Wice	Reveal	Claim Verify	Fact Check	Expert QA	Lfqa	Avg
	CNN	XSum	MediaS	MeetB							
Open-source Models (Llama3-8B based)											
Llama3-8B-chat	50.9	58.2	63.9	72.4	65.1	85.2	63.8	76.5	55.9	72.3	66.4
FENCE (*Vanilla SFT*)	63.2	73.4	66.6	77.7	64.6	86.4	72.5	73.0	57.9	83.1	71.8
FENCE (*Critique Only*)	59.5	74.7	68.4	80.0	71.7	88.0	74.3	74.2	59.6	**87.0**	73.7
FENCE (*Full*)	62.1	72.4	**70.9**	**80.3**	76.0	**88.6**	**74.9**	74.4	**60.3**	86.9	**74.7**
Other Open-source Models (47B-123B)											
Mistral-8x7B†	55.0	65.5	68.5	73.3	63.8	80.8	64.3	75.1	56.3	70.8	67.3
Llama3-70B-chat	**63.3**	71.3	67.9	75.2	74.8	86.7	67.3	78.3	58.4	82.9	72.6
Mistral-123B†	58.4	**76.3**	67.3	78.9	**76.6**	88.4	67.6	**79.0**	60.0	81.7	73.4
Proprietary Models or Distilled Models											
Gemini-Pro†	49.4	60.6	63.8	65.8	65.8	85.5	61.8	76.8	56.8	75.9	66.2
GPT-3.5†	63.2	72.4	66.8	73.4	68.5	84.7	65.2	70.8	57.2	73.8	69.6
Claude-2.1†	59.9	66.4	69.2	72.3	64.3	88.2	69.7	79.3	59.8	78.2	70.7
Claude-3 Opus†	65.2	72.4	74.1	82.4	75.0	83.8	69.3	78.8	58.8	81.6	74.1
MiniCheck-FT5†	69.9	74.3	73.6	77.3	72.2	86.2	74.6	74.7	59.0	85.2	74.7
GPT-4†	66.7	76.5	71.4	79.9	80.4	87.8	67.6	79.9	59.2	83.1	75.3

Table 7.1: Performance (BAcc) of evaluator models on the test split of LLM-AggreFact. We separate Llama3-8B-chat based models, larger open-source models, and proprietary models into different blocks. We highlight the best-performing open-source model for each dataset. Results with † are reported in Tang et al. [99].

Among all the datasets, we observe that the performance on Wice decreases after Vanilla SFT, but is largely improved by FENCE. One possible reason is that some training datasets such as Q^2 contain claims that are labeled as "factual" but are only partially supported by the documents. We hypothesize that such noisy examples strongly affect the performance on Wice, which contains as high as 54.7% of such "partially supported" examples. However, by filtering out examples where Llama3-70B-chat cannot generate explanations, we filter out a large percent of such noisy examples and hence improve the final judgment accuracy.

7.3.3 Result Analyses

Accuracy of Augmented Critique and Source Documents. With our data augmentation methods, we equipped 77.2% of the training examples with textual critiques, and generate new source documents for 54.1% of the examples with the combination of our three tools. To verify the data quality, we randomly sampled 45 examples where we successfully obtain critiques or source documents (15 for each label) and manually inspect the accuracy. Specifically, we check whether both the critique and the label correctly reflect the relationship between the claim and the source documents.

Pred Label (→)	Supported	Contradictory	Unverified
Critique Acc	15/15	14/15	14/15
Tool-ext Doc Acc	14/15	15/15	15/15

Table 7.2: The accuracy of the critiques and the tool-extracted source documents we obtained. We randomly sampled 15 examples for each predicted label and manually check the accuracy of each example.

As shown in Table 7.2, 95.6% of the augmented critiques and 97.8% of the tool-extracted documents are accurate. Although in one of the wrongly-labeled examples, our method mislabels "unverified" as "contradictory", it does not affect the final conclusion that the claim is not factual.

Case Studies. We further show three concrete examples of our augmented critiques and source documents. In the first example (Table 7.5), our generated critique correctly explain the judgment label: the claim mentions a call for a national project, which does not appear in the documents.

In the second example (Table 7.6), the original document is a CNN news report. By calling the search engine, we obtain other news articles written by diverse news agencies on the same event (i.e., Kenneth Morgan's murder). Such tool-extracted documents increase the diversity of documents in the train set, while still having high label accuracy.

In the third example (Table 7.7), we obtain Chadwick Boseman's birthday information from all three tools: knowledge graph, Wikipedia, an search engine, where the knowledge graph provides more structured and concise knowledge, while Wikipedia returns a long paragraph containing the information. Compared to the original documents, our tool-extracted documents have more diverse formats, improving the evaluator's generalizability.

7.4 Experiments on Generator Factuality

We aim to answer the research questions: (**RQ2**) Can we leverage FENCE to improve the generator's factuality? (**RQ3**) How well can our training recipe improve the generator's factuality?

7.4.1 Experimental Setup

Datasets and Evaluation Metric. Following prior works [46, 140], we conduct experiments on FActScore [68] and TruthfulQA [59]. For FActScore, we randomly split the unlabeled split into 400 training and 100 test prompts. We compute the "% Facts" metric by extracting correct and

Model Name	FActScore			TruthfulQA
	# Facts	# Errors	% Facts	% True*Info
Llama2-7B-chat	10.70	17.04	38.57	38.83
+ SFT	10.76	15.59	40.83	45.52
+ Self-Eval-SKT	11.02	14.18	43.73	48.65
+ EVER-Pref	**11.24**	15.11	42.66	51.07
+ FactTune-FS	11.23	12.87	46.60	52.48
(Our Method)				
+ E/R + Coarse	10.84	**8.72**	**55.43**	**56.47**
Llama3-8B-chat	17.83	17.16	50.96	58.89
+ SFT	20.05	18.13	52.52	59.17
+ Self-Eval-SKT	18.69	14.22	56.80	61.88
+ EVER-Pref	20.25	15.16	57.18	63.01
+ FactTune-FS	18.77	13.34	58.45	64.58
(Our Method)				
+ E/R + Coarse	**20.40**	**10.79**	**65.41**	**67.14**

Table 7.3: Comparison between our method and baselines on FActScore and TruthfulQA. "E/R" stands for "Edit/Remove". All baselines use the zero-shot models as the evaluator in training and our method uses FENCE.

incorrect facts in each response, where we use Llama3-70B-chat to decompose the responses and use the same three tools in training to obtain source documents. For TruthfulQA, we randomly select 3 examples from each of the 38 categories as the training set and use the remaining 703 examples as the test set. We follow the "generation" setting and use the finetuned evaluator in the original paper to compute "% True*Info", the percentage of responses that are both truthful and informative.

Baselines and Ablations. We implement four baselines: SFT, FactTune-FS [103], Self-Eval-SKT [140], and EVER-Pref [46]. All methods sample N candidate responses for each training prompt. SFT first uses the generator itself to score responses by computing the percentage of factual claims and finetunes with the best response. FactTune-FS first finetunes on all candidates and then uses all $\binom{N}{2}$ candidate pairs as DPO pairs, preferring the one with a higher score based on retrieved context. Self-Eval-SKT self-trains an evaluator using the model's own knowledge and uses the evaluator to score responses with no external context, also resulting in $\binom{N}{2}$ preference pairs. EVER-Pref uses all N candidates as rejected responses and constructs a preferred response by iteratively evaluating and correcting false information in each passage.

For ablations, we first equip SFT, FactTune, and EVER with our evaluator, FENCE, and the three external tools, denoted as "SFT + FENCE", "Coarse", and "Edit" in Table 7.4, respectively.

Model Name	FActScore		
	# Facts	# Errors	% Facts
Llama3-8B-chat	17.83	17.16	50.96
(*Ablations with Llama3-8B-chat as the evaluator*)			
+ SFT	20.05	18.13	52.52
+ EVER-Pref	20.25	15.16	57.18
+ FactTune-FS	18.77	13.34	58.45
(*Ablations with FENCE as the evaluator*)			
+ SFT (FENCE)	21.19	16.47	56.26
+ EVER-Pref (FENCE)	**20.68**	14.42	58.91
+ FactTune-FS (FENCE)	20.07	12.89	60.89
(*Our Full Method*)			
+ E/R + Coarse	20.40	**10.79**	**65.41**

Table 7.4: Ablation study that compares different training recipes when equipped with FENCE as the evaluator. "SFT + FENCE", "Edit", and "Coarse" denote equipping "SFT", "EVER-Pref", and "FactTune-FS" in Table 7.3 with the FENCE evaluator.

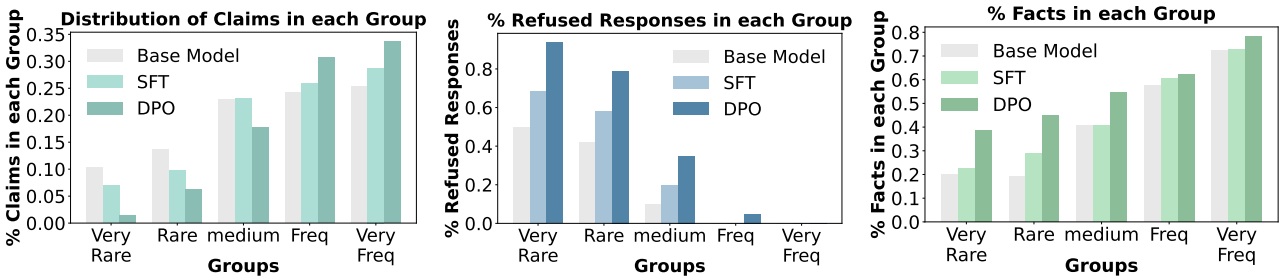
Then we implement "Edit + Coarse", which corrects all false information without checking whether it corresponds to a lesser-known fact. We denote our full method as "E/R + Coarse" (Edit/Remove + Coarse).

Note that we do not compare to retrieval-augmented (RAG) methods [51] because it is not a fair comparison. RAG-based methods require access to external knowledge sources or tools in inference, while in our experiments, we only used "`tell me a bio of <entity>`" as the prompt. Furthermore, while most RAG-based methods aim to improve the understanding of the long retrieved context, our main focus is to train the model to better utilize the knowledge it has seen during pretraining.

We also do not compare to inference-time methods [15, 57] because these methods need to call the model multiple times in inference, introducing heavy latency issues. In the real world, such latency is unacceptable in production. In comparison, our method uses the standard random decoding in inference and does not introduce extra inference-time cost. Furthermore, in principle, our method is orthogonal to those inference-time methods and can be combined.

7.4.2 Main Results

Results in Table 7.3 answer (**RQ2**) and show that our method significantly improves Llama2/Llama3's factuality by 16.85/14.45% on FActScore and 17.64%/8.25% on TruthfulQA. Results also show



(a) Distribution of claims (b) % Refused responses (c) % Facts

Figure 7.4: Statistics of generated responses, where we group the prompts by the popularity of the person to write biography for, as labeled in the FActScore dataset. We compare zero-shot Llama3-8B-chat (denoted as *"Base Model"*), SFT + FENCE (*"SFT"*), and our method, Edit/Remove + Coarse (*"DPO"*).

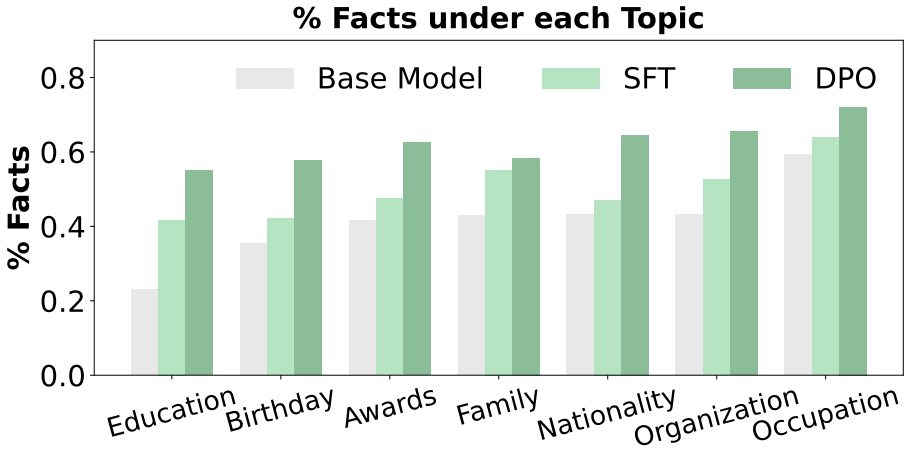
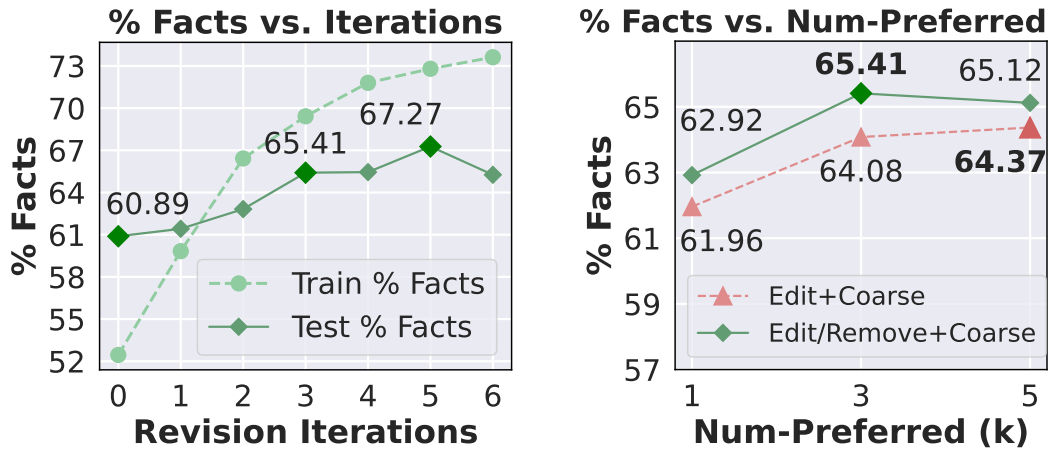


Figure 7.5: The factuality of claims with different topics, where we pre-define the topics and use Llama3-70B-chat to predict whether each claim covers any topic(s).

that our method significantly outperforms the best baseline on factuality training (e.g., by 8.83/6.96% on FActScore for Llama2/Llama3).

Table 7.4 presents the ablations with FENCE as the evaluator. The comparison between using FENCE and Llama3-8B-chat shows that FENCE can improve the performance of SFT, EVER-Pref, and FactTune-FS, which emphasizes the significance of training factuality evaluator models. For example, SFT with FENCE as the evaluator outperforms SFT with Llama3 as the evaluator by 3.74%. In addition, we also observe that our method of combining response editing/removing with coarse-level scoring achieves significantly better performance. The above results answer (**RQ3**), demonstrating the effectiveness of our training recipe.



(a) Iterations of Revision

(b) top-k

Figure 7.6: Hyper-parameter analysis. We investigate how percentage of facts changes (a) as number of iterations to revise candidate responses increases, and (b) with different numbers of preferred responses for each prompt.

7.4.3 Result Analyses

Distribution of Generated Claims. We first group the prompts by the popularity of the person to write biography for, which is provided as meta data in the FActScore dataset, and then compare the distribution of generated claims in each group before and after training. As shown in Figure 7.4a, after training, the generator outputs less information for unfamiliar people and outputs more information for popular ones, suggesting that it learns to only generate information that is likely to be factual.

In addition, as shown in Figure 7.4b, we observe that the generator refuses to generate responses more frequently for rare entities (e.g., by generating "I apologize, but I'm not familiar with this person."), and almost never refuses for frequent ones. This aligns with previous research's conclusion [47] that training the model to say "I don't know" to unfamiliar prompts reduces hallucination.

Performance Breakdown. We further check the generator's performance (i.e., percentage of factual claims) on different groups of prompts. As shown in Figure 7.4c, we observe that our method achieves consistent performance gain over all groups of prompts, which is another reason why our method obtains higher overall performance.

Similarly, we check the performance on claims describing different topics, where we first come up with a list of topics and then prompts Llama3-70B-chat to determine whether each claim covers any of the topics. In Figure 7.5, our method generates more factual claims under

all the topics, and the performance gain is larger on unfamiliar topics (i.e., topics where the generator has lower scores).

Hyper-parameter Analysis. We first alternate the number of iterations to revise the responses and inspect the testing and training accuracy (i.e., the average % of facts of the best preferred responses). As shown in Figure 7.6a, with more revision iterations, we can always obtain preferred responses with better factuality, but the test performance converges after the third iteration. In other words, training data with fewer factual errors does not always transfer to better test performance.

In our experiments, we only choose the top-k candidates (ranked by the percentage of facts) as preferred responses. We investigate the effect of k in Figure 7.6b. We observe that training on top-3 and top-5 responses leads to similar performance. With all the ks in our experiments, our method consistently outperforms the "Edit + Coarse" ablation.

7.5 Chapter Summary

In this chapter, we improve LM generators' factuality by training an open-source evaluator model, FENCE. To train FENCE to leverage diverse knowledge sources and to generate more informative feedback, we equip a combination of public datasets with textual critique along with judgment scores and obtain additional source documents by calling a multiplicity of tools. We then present a training recipe that leverages FENCE to finetune LM generators for better factuality, where we construct preference data by prompting FENCE to revise and score the generator's responses, without introducing lesser-known facts in training. Experiments show that FENCE outperforms strong proprietary models such as Claude-3 on LLM-AggreFact. Our factuality training method improves Llama3-8B-chat's factuality performance by 14.45% on FActScore and 8.25% on TruthfulQA.

chapter 8 adapts the high-level idea of auxiliary model training to coding agent, where we first show that the coding agent performs better in inference if it receives critique from an auxiliary model every k steps. Then, we propose a data construction method to train an open-source critic model.

Source Document (*Dataset: FRANK-xsum*)
Title: News Articles
Text: They believe ministers are placing too much emphasis on the environment at the expense of trees grown for timber. Britain is currently the world's third largest importer of wood. Ministers said they were encouraging commercial forestry organisations to invest in woodland creation. Conifer forests have been a familiar sight for half a century in Wales and have helped the timber industry grow. But Confor, which promotes the forestry industry, warns that at least 16,000 hectares - or 40,000 acres - of commercial forest have been lost since 2001 and need to be re-planted to meet needs. Half of the woodland is managed by Natural Resources Wales with the the other half by private companies. As an industry, it is estimated to be worth more than Â£450m a year to the Welsh economy. But there is a conflict with environmental policy. **Successive ministers have wanted more native, broadleaf species of trees and better access for walkers and bike riders while conservationists claim commercial forests are bad for wildlife.** However, some forests in Wales are difficult to reach and hard to harvest while transporting the timber through rural communities can bring companies into conflict with local residents. The Welsh government wants a balance between the environmental, the economic and the social in how forestry is developed. It points to its Glastir scheme which encourages farmers and other landowners to plant woodland. It said its Woodlands for Wales forestry strategy "sets out how we want Wales to be known for its high quality woodlands that enhance the landscape, provide real social and community benefits, support thriving woodland-based industries and contribute to a better quality environment". Martin Bishop, of Confor, which represents 200 businesses in Wales, believes the emphasis on environmental aspects is seen as "sexier" than commercial forestry. He said the current system is also too bureaucratic for creating new woodland areas. Mr Bishop said 90 hectares were planted last year but there needed to be 20,000 hectares a year to meet long-term targets set by ministers. "There's a huge market for our timber. Every saw miller tells me he would double or treble production if the timber was there to feed that mill," he added.

Claim
Conservationists are calling for a national project to protect forests in wales.

Ground Truth Label
Non-Factual

Our Generated Critique and Label
Critique: The document does not mention conservationists calling for a national project to protect forests in Wales. It only mentions that conservationists claim commercial forests are bad for wildlife, but it does not mention a specific call for a national project.

Label: **Unverified**

Table 7.5: An example of the textual critique we generate. We highlight the most relevant sentence in the document. In this case, our generated critique aligns with the judgment label.

Original Source Document (*Dataset: FRANK-CNN/DM*)
Title: News Articles
Text: (CNN) Deputies rushed Kenneth Morgan Stancil III from court Thursday after the 20-year-old murder suspect swore at a judge and tried to flip over a table. Stancil is accused of killing an employee Monday at Wayne Community College in Goldsboro, North Carolina. Relatives have said victim Ron Lane was gay, CNN affiliate WNCN reported, and investigators are looking into whether the shooting was a hate crime. Authorities arrested Stancil after he was found sleeping on a Florida beach on Tuesday. Just a few minutes into Thursday's hearing on the first-degree murder charge he faces, Stancil snapped back at the judge after he was offered a court-appointed lawyer. N̈o, I don't need one,s̈aid Stancil, who stood before the judge with his legs shackled and his arms handcuffed in front of him. Ÿou know what I'm saying? I knew I would get life anyway.S̈uperior Court Judge Arnold O. Jones interjected, pointing out that the maximum sentence Stancil faces is the death penalty. Ÿes, I know that,S̈tancil fired back. B̈ut when I knew what I had to do and I knew when I got caught, you know, I knew in my mind that I could get life, I could get the death penalty. You know what I'm saying? Do you follow my topic? I would have killed you, you know what I'm saying, if you're a f—ing child molester.T̈he judge told him not to swear. Ï don't give a f— what you want,S̈tancil said, lunging forward and lifting up the table in front of him. Deputies quickly corralled him and hustled him from the courtroom. The hearing resumed about 25 minutes later, when Stancil was brought back into the courtroom, this time with his arms handcuffed behind him. When asked again by Jones whether he wanted a lawyer, his response was quick – and calm. Ÿes, sir,ḧe said. In an interview with CNN affiliate WRAL, Stancil described himself as a neo-Nazi and said he hates gay people ẅith a passion.S̈tancil had worked for Lane, the school's print shop operator, as part of a work-study program, but was let go from the program in early March because of poor attendance, college officials said.

Claim
Kenneth Morgan Stancil is accused of killing an employee at Wayne Community College in Goldsboro, North Carolina.

Ground Truth Label
Factual

Generated Queries to Call the Tools
Search query for Bing Search API: Kenneth Morgan Stancil Wayne Community College killing
Queries for Wikipedia: Kenneth Morgan Stancil; List of school shootings in the United States; Wayne Community College;
Entities for Google Knowledge Graph API: Kenneth Morgan Stancil

Tool-extracted Source Documents (*after Reranking*)
Title: Wayne Community College shooter gets life sentence without parole
Text: GOLDSBORO, North Carolina (WTVD) – **Kenneth Morgan Stancil III was sentenced Tuesday to life in prison without parole for the murder of 44-year-old Ron Lane on the campus of Wayne Community College in Goldsboro on April 13, 2015.** Stancil entered the campus print shop on the third floor of the same building that houses the school library and cafeteria shortly after Lane arrived for work that day and shot him once with a pistol-grip 12-gauge shotgun. ...

Title: Stancil guilty in Wayne Community College murder trial - CBS17.com
Text: GOLDSBORO, N.C. (WNCN) – After deliberating for an hour and a half, **a Wayne County jury found Kenneth Morgan Stancil III guilty of first-degree murder.** He will spend the rest of his life in prison. During his closing argument Tuesday in a Wayne County court, the district attorney relied largely on Stancil's own words, **painting him as confident and proud about his decision to kill a man at Wayne Community College.** In his counterargument, Stancil's attorney said the prosecution tried to scare the jury with Stancil's possession of knives, guns, and white supremacist memorabilia, as well as his numerous tattoos. None of that proved he planned to commit the murder, the defense said. **Neither side disputed that Stancil killed Ron Lane at Wayne Community College in 2015**. The prosecution contends it was first-degree murder, but the defense argues ...

Title: Man convicted in fatal Wayne Community College shooting - WRAL ...
Text: Man convicted in fatal Wayne Community College shooting A white supremacist was sentenced Tuesday to life in prison without parole for gunning down a print shop operator at Wayne Community College two years ago. **Jurors deliberated for about 90 minutes before convicting Kenneth Morgan Stancil III, 22, of first-degree murder in the April 13, 2015, death of Ron Lane in the school's library.** Authorities said Stancil shot Lane with a 12-gauge pump shotgun before fleeing the area on a motorcycle. He was captured the next morning when police found him asleep on a beach in Volusia County, Fla. ...

Title: Wayne Community College killing suspect makes shocking allegations in ...
...

Our Generated Critique and Label
Critique: The statement is supported by the document. In all the documents, it is mentioned that Kenneth Morgan Stancil III is accused of killing Ron Lane, an employee at Wayne Community College in Goldsboro, North Carolina.

Label: **Supported**

Table 7.6: An example where we obtain multiple relevant source documents by calling a multiplicity of tools.

Original Source Document (*Dataset: FActScore*)
Title: Chadwick Boseman
Text: Chadwick Boseman Chadwick Aaron Boseman (**; November 29, 1976August 28, 2020**) was an American actor. During his two-decade career, Boseman received multiple accolades, including two Screen Actors Guild Awards, a Golden Globe Award, a Critics' Choice Movie Award, a Primetime Emmy Award, and an Academy Award nomination. After studying directing at Howard University, Boseman began his career in theatre, winning a Drama League Directing Fellowship and an acting AUDELCO ...

Claim
Chadwick Boseman was born on November 29, 1977.

Ground Truth Label
Non-Factual

Generated Queries to Call the Tools
Search query for Bing Search API: Chadwick Boseman date of birth
Queries for Wikipedia: Chadwick Boseman
Entities for Google Knowledge Graph API: Chadwick Boseman Stancil

Tool-extracted Source Documents (*after Reranking*)
Title: knowledge graph result
Text: Chadwick Boseman: American actor. Chadwick Aaron Boseman was an American actor, known for portraying African-American historical figures and the fictional African superhero Black Panther. **Chadwick Boseman Born: November 29, 1976, Anderson, SC.** Chadwick Boseman Died: August 28, 2020 (age 43ŏ0a0years), Los Angeles, CA. Chadwick Boseman Spouse: Taylor Simone Ledward. Chadwick Boseman Height: 6ž032 0ž033. Chadwick Boseman Parents: Leroy Boseman and Carolyn Boseman. Chadwick Boseman Alma mater: Howard University (BFA). ...

Title: Chadwick Boseman | Biography, Movies, Black Panther, & Death text: Chadwick Boseman
Text: Our editors will review what you've submitted and determine whether to revise the article.
- Chadwick Aaron Boseman
- Born: November 29, 1976, Anderson, South Carolina, U.S.
- Died: August 28, 2020, Los Angeles, California (aged 43)
- Notable Family Members: son of Leroy Boseman ...
Chadwick Boseman (born November 29, 1976, Anderson, South Carolina, U.S.—died August 28, 2020, Los Angeles, California) was an American actor and playwright who became a highly respected movie star with several iconic roles, notably that of T'Challa/Black Panther in the groundbreaking film Black Panther (2018). ...

Title: Chadwick Boseman - Wikipedia
Text: Chadwick Boseman Chadwick Aaron Boseman (**; November 29, 1976August 28, 2020**) was an American actor. During his two-decade career, Boseman received multiple accolades, including two Screen Actors Guild Awards, a Golden Globe Award, a Critics' Choice Movie Award, a Primetime Emmy Award, and an Academy Award nomination. After studying directing at Howard University, Boseman began his career in theatre, winning a Drama League Directing Fellowship and an acting AUDELCO ...

Title: Chadwick Boseman | Biography, Movies, Black Panther, & Death ...
...

Our Generated Critique and Label
Critique: The statement claims that Chadwick Boseman was born on November 29, 1977. However, all four documents consistently state that Chadwick Boseman was born on November 29, 1976, not 1977.

Label: **Contradictory**

Table 7.7: An example where we obtain multiple relevant source documents by calling a multiplicity of tools. The tool-extracted documents all contain the information about Chadwick's birthday, but have diverse formats.

Chapter 8

Improving Coding Agent Effectiveness by Training a Critic Model

8.1 Introduction

Coding agents have been widely used in real-world programming tasks [43, 127], and post-training has proven effective for coding agent training [85, 124, 131], where the model is optimized for the combination of multiple abilities, including high-level reasoning (e.g., troubleshooting, search, planning) and low-level implementation (executing tool calls correctly).

Theoretically, the optima for the joint capabilities could be different from the optima for each individual capability, such as high-level planning. Empirically, during post-training, multiple studies have shown that the planning ability of a coding agent is still far from perfect after reaching the plateau on the training data [108, 124]. For example, about 65% of coding agent failures come from flawed reasoning, not wrong actions [61]. A possible explanation is that integrating both high-level and low-level knowledge in training makes the learned patterns overly complex, which undercuts the coding agent's ability to learn generalizable patterns and limits the overall performance.

To address this challenge, we shift part of the high-level planning role to a separate, specialized **critic model**, which steers the main coding agent by providing feedback during inference. The idea is inspired by research on critic and adviser models in other NLP domains such as question-answering [30], factuality [122], and mathematical problems [58]. Compared to training the coding agent itself, we highlight the advantages of training a critic model as follows: First, the critic model improves the overall performance by complementing the planning ability of the coding agent. Second, critic model training is cost-efficient in terms of both data curation and computational resources. §8.2 shows that we can re-use existing coding-agent

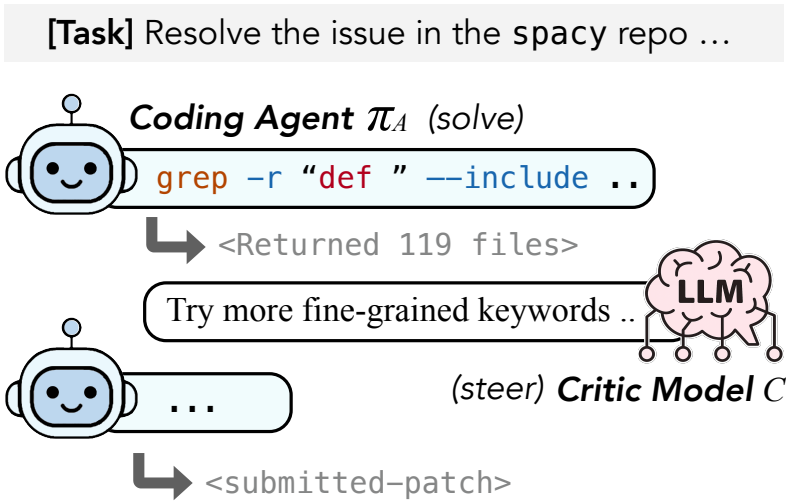


Figure 8.1: Inference setup. A trained critic model C provides guidance to a frozen coding agent π_A for every k steps. The agent is responsible for all concrete actions and the critic model inspects the current trajectory and generates high-level feedback for its next steps.

environments for critic model training, without building new training tasks or environments. §8.3.3 shows that to improve the overall performance, we can train a small critic model instead of training the large coding agent, which requires fewer computational resources. Finally, the use of critic models is highly adaptive and flexible. We can provide critiques to any frozen coding agent at any step with any frequency.

In this paper, we design a two-stage training framework to train the critic models. The first training stage (Figure 8.2) uses supervised fine-tuning (SFT), where we distill from a strong teacher model (e.g., `Claude-Opus-4.6`). We explicitly require the target critiques to detect the major errors made by the coding agent and provide corresponding high-level guidance to the next steps, without containing any concrete implementation such as specific tool-call commands. As shown in our analyses, there is still a substantial gap between the overall guidance generated by the SFT student model and the teacher critic model. Therefore, we present the second training stage with direct preference optimization (DPO), where we construct preference pairs based on the quality of the guidance in the critiques. Specifically, we design two metrics: correctness and clarity. Correctness requires the critiques to ground to the previous error categorization results and suggest the correct direction for future steps. Clarity checks whether the critiques provide clear instructions without ambiguity or under-specification.

Experiments on SWE-Bench Verified [43] show that our trained critic model improves the resolved rate of six different coding agent models by 5.6%–16.0%, without re-training the coding agents. We further demonstrate that our training can transfer across different program-

ming languages and coding agents in training and evaluation. For instance, we improve the performance on Multi-SWE-Bench [136] by 2.6%–3.6% on 7 unseen languages. We also conduct detailed analyses to demonstrate the effectiveness of the SFT stage, DPO stage, and the design choice of training on high-level critiques only. Finally, we provide qualitative analyses and case studies on the critiques generated by our models.

8.2 Methodology

Long-horizon coding agents drift: they over-commit to early hypotheses, repeat failed commands, or allocate steps to subgoals in a suboptimal manner [26, 108]. To overcome this challenge, we leverage a critic as a second model to inspect the trajectory at runtime in order to alert the model to such drift. Empirically, pairing a frozen coding agent with a strong proprietary model such as `Claude-Opus-4.6` [6] as a critic substantially improves the performance but also requires a higher total cost. For instance, it improves the resolved rate of `Qwen3-32B` on SWE-Bench-Verified from 8.8% to 26.8%, but also increases the per-instance cost from $0.05 to $0.28.

To achieve a better balance between cost and performance, in this section, we train a smaller open-source critic model to guide the coding agent during inference. Specifically, we first introduce the formal definitions (§8.2.1) and the design of the critique (§8.2.2). Then we design training data for the SFT (§8.2.3) and DPO stages (§8.2.4).

8.2.1 Formal Definitions

Coding Agent. Given a coding task and a docker environment in which to perform the task, the coding agent has access to a set of tools (e.g., the bash command-line tool) that can be called through tool-call actions. In problem-solving, the agent generates a trajectory as the solution $\tau_{1:t} = (a_1, o_1, \ldots, a_t, o_t)$, where a_i is the agent's action and o_i the corresponding environment observation. We limit the number of steps with a fixed budget $t \leq B$. In our setting, π_A is held frozen throughout and we only train the critic model C_θ.

Critic Models for Coding Agents. Formally, we pair the coding agent π_A with a critic model C at inference time. For every k agent steps (we use $k = \{5, 10\}$ in our experiments), the critic model inspects the task and the current trajectory prefix $\tau_{1:t}$. To preserve continuity across interventions, we also include the feedback the critic produced at earlier intervention points. Let $h_t = (f_k, f_{2k}, \ldots, f_{t-k})$ denote the critic's prior feedback history before step t. We form the critic input $z_t = \mathrm{Serialize}(\tau_{1:t}, h_t, x)$, which fits the trajectory to the context window limit

x by preserving the start of the trajectory (system prompt and task context) and the most recent steps, including the final user message, and finally truncating the intermediate steps that exceed the context limit. We sample feedback $f_t \sim C(\cdot \mid z_t)$, which is appended to the agent's context for the next step.

8.2.2 Designing the Critique Structure

We design the structure of the critique as follows: it first outputs the major errors made by the coding agent and the recovery action, where the error categories are pre-defined. Then it outputs the overall guidance, which contains only high-level feedback instead of the concrete tool call actions. Finally, it outputs a reminder when the agent is close to using up the step limit. We provide the prompt for critique generation in App. B.4.1.

Error Detection for the Coding Agent. Multiple existing studies show that coding agents' capability to recover from errors is still limited [26, 108]. One example is the "stuck-in-loop" behavior, where the agent keeps generating the same tool call action even if it fails every time. To improve error recovery, we prompt the critic model to detect major errors made by the coding agent. Inspired by SWE-PRM [26], we pre-define 12 trajectory-level error categories covering *Specification*, *Reasoning*, and *Coordination* errors. The complete list is provided in App. B.4. For each category, the critic model reports whether the error is detected, the supporting evidence, recovery strategies, and a task-status verdict showing whether the error is not significant and the agent is still on track, or the error is serious and needs correction or critical intervention.

High-level Feedback Only, NOT Concrete Actions. The motivation for critic model training is to separate the high-level planning ability to a **specialized** model. As a result, we explicitly prompt the critic model NOT to suggest specific technical implementation or code-level changes, such as outputting the exact tool call command. To improve correctness, we also require the suggested recovery strategies or overall guidance to be grounded in previous error detection results. We additionally prompt the critic model to keep the guidance high-level (1-2 sentences per detected error), without giving too complicated instructions to coding agents that cannot be completed in a few steps. The prompt thereby restricts the teacher to high-level signals such as whether the agent is making progress, looping, relying on an unsupported assumption, or pursuing the wrong subgoal (App. B.4).

Budget Awareness. A high-level critic that ignores the budget will keep recommending exploration even when the agent is close to running out of steps, leaving useful in-progress work unsubmitted. Therefore, in inference, once the agent passes step T (e.g., we use $T{=}100$ of the $B{=}150$-step budget), we append short budget-aware instructions that state the steps remain-

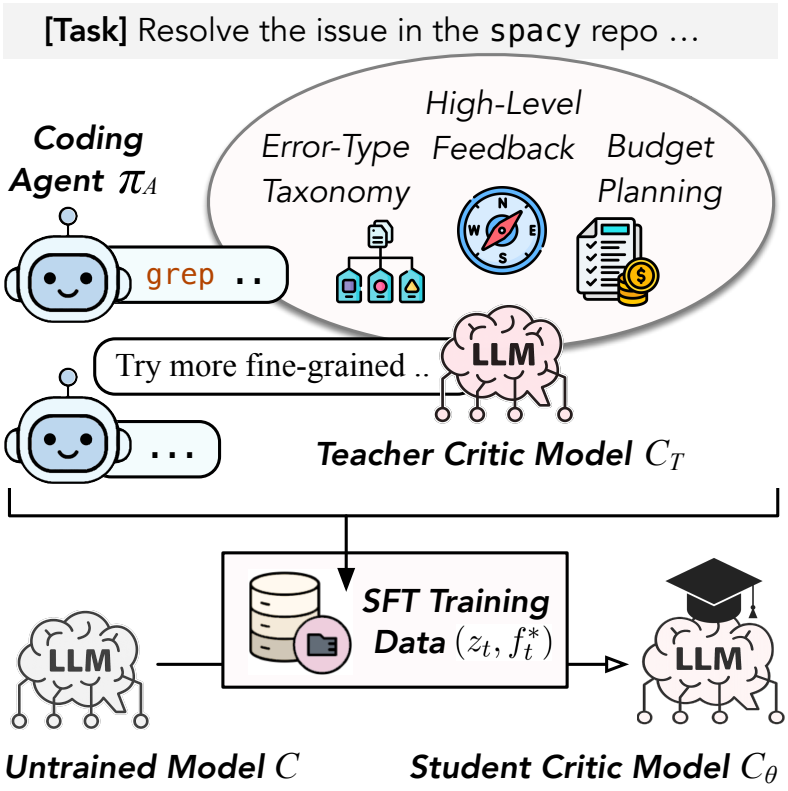


Figure 8.2: SFT training pipeline for critic models. Based on our analysis (§8.2.2), the critic contains error detection, budget planning, and only provides high-level feedback instead of concrete actions. We run the frozen coding agent π_A on disjoint training tasks and query the teacher critic C_T every k steps. We collect (trajectory prefix, feedback) pairs (z_t, f_t^*) to train a critic C_θ by supervised fine-tuning (SFT).

ing and recommend the coding agent to make a submission as soon as possible (App. B.4.3).

8.2.3 Distilling Critic Models with SFT

SFT Data Construction. As show in Figure 8.2, we prompt the teacher critic model C_T to provide guidance to a frozen coding agent π_A every k steps based on the current trajectory z_t on the training dataset $\mathcal{TR}_{\mathcal{SFT}}$ (as described in §8.3.1) and fine-tune a student critic model C_θ with the teacher's outputs as targets.

Based on the prompt design introduced in §8.2.2, we prompt the teacher model to first detect errors made by the coding agent and then provide overall guidance, with budget-aware alerts. We additionally require it to only generate high-level feedback rather than action-level prescriptions. We filter out malformed teacher responses (empty completions or responses that drop out of the critique format) before training (§8.3.1). Note that we do NOT filter

training critiques based on whether the task is successfully completed, which depends more on the coding agent itself rather than the critic model.

Critic Model Training with SFT. As shown in Figure 8.2, we distill C_θ from C_T by supervised fine-tuning (SFT) with a standard conditional language modeling objective, maximizing likelihood:

$$\max_\theta \mathbb{E}_{(z_t, f_t^*) \sim \mathcal{TR}_{\mathcal{SFT}}} \log \mathbb{P}_\theta(f_t^* \mid z_t). \tag{8.1}$$

The trained critic model C_θ is then paired with π_A at inference..

8.2.4 Improving Critic Models with DPO

The Remaining Gap after Distillation. We empirically study the gap between the student and teacher critic models after SFT (§8.2.3). As shown in App. B.1, the major gap exists in the **overall guidance** provided by the critic, instead of the error categorization. Among the 132 critiques we sampled, 67% of the teacher model's critiques both point to the correct direction and contain specific guidance for the next steps. In contrast, only 29% of the SFT student model's critiques are both correct and specific. 53% point to the correct direction but are underspecified. 18% suggest the wrong steps to take. We use an LLM judge for the judgment. We choose a different model from the teacher model to avoid self-bias [144]. We thus train the critic model for the second stage, focusing on generating high-quality overall guidance.

DPO Data Construction. We design two metrics to judge the quality of the guidance: (1) correctness, where the guidance fixes the serious error made by the coding agent (if any) and assists problem-solving; and (2) clarity, where the guidance is not ambiguous or underspecified. We adopt direct preference optimization (DPO) to improve these two metrics.

As shown in Figure 8.3, we apply the coding agent π_A on the training dataset TR_{DPO}. For every k steps, we use the SFT critic model $C_{SFT,\theta}$ to generate $N = 10$ critiques, with the previous trajectory z_t as context. Then we prompt an LLM judge to select the best and worst ones based on the correctness and clarity of the overall guidance. This forms one (chosen, rejected) training pair (f_c, f_r) for every set of critiques. We only keep the training pairs where the error categorization does not have major error, the chosen critique is both correct and clear, and the rejected critique is either not correct or not clear.

Critic Model Training with DPO. As shown in Figure 8.3, we initialize from the SFT critic

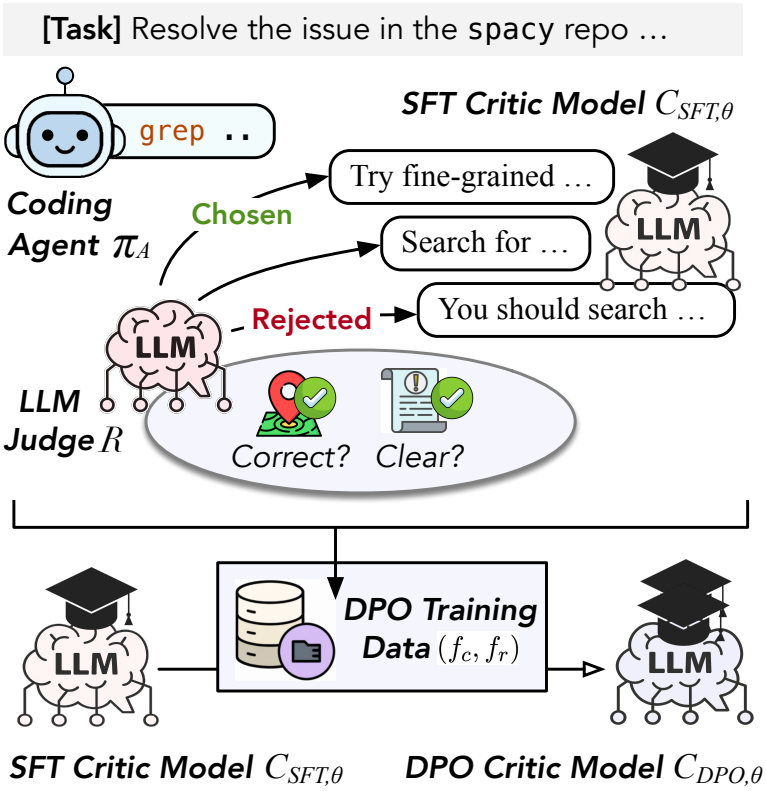


Figure 8.3: DPO training pipeline for critic models. Given a critic model checkpoint $C_{SFT,\theta}$, for every k steps produced by the coding agent π_A, we use $C_{SFT,\theta}$ to generate $N = 10$ critiques, and prompt an LLM judge model R to select the best and worse critique, which forms (chosen, rejected) pairs (f_c, f_r) to train $C_{SFT,\theta}$ by direct preference optimization (DPO). We only keep a (f_c, f_r) pair if the chosen critique is both correct and provides clear instruction and the rejected critique misses any of the criteria.

model $C_{SFT,\theta}$ and conduct DPO training with the following classification loss:

$$\max_{\theta} \mathbb{E}_{(z_t, f_c, f_r) \sim \mathcal{TR}_{DPO}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta\left(f_c \mid z_t\right)}{\pi_{\text{ref}}\left(f_c \mid z_t\right)} - \beta \log \frac{\pi_\theta\left(f_r \mid z_t\right)}{\pi_{\text{ref}}\left(f_r \mid z_t\right)} \right) \right], \tag{8.2}$$

where σ is the Sigmoid function. The reference policy π_{ref} is computed by the SFT checkpoint $C_{SFT,\theta}$ and is frozen throughout training.

8.3 Experiments

This section evaluates the performance of our trained critic models on downstream coding tasks. §8.3.1 and §8.3.2 introduces the experimental setup and implementation details. §8.3.3 demonstrates the effectiveness of our critic models of different sizes when combined with six different coding agent models. §8.3.4 presents detailed analyses, including the transferability of our critic models across programming languages and training corpus, the cost-performance trade-off of critic models, and the verification of our design choices that the critiques should only contain high-level guidance, not concrete actions. §8.3.5 provides qualitative analyses and case studies for the generated critiques.

8.3.1 Experimental Setup

Benchmarks. We primarily evaluate on SWE-Bench Verified [43], a benchmark of 500 human-verified GitHub issues drawn from 12 open-source Python repositories. We also conduct evaluation on its multilingual variant, Multi-SWE-Bench [136], to demonstrate the transferability of our method across programming languages. For each instance, the agent is given the issue's problem statement and access to the pre-issue repository snapshot, and must edit the codebase to resolve the issue. An instance is counted as resolved only if the resulting patch satisfies two hidden test sets: `FAIL_TO_PASS` tests that target the reported bug, and `PASS_TO_PASS` tests that guard against regressions in unrelated code.

Evaluation Metrics. We report evaluation metrics on both the output quality and the cost: (i) Resolve Rate, the fraction of the 500 instances whose submitted patch passes both the `FAIL_TO_PASS` and `PASS_TO_PASS` test sets; (ii) Localized Rate, the percentage of evaluated instances whose patch edits at least one file also touched by the gold patch [117]. (iii) Non-Empty Rate, the fraction of runs that exit through the agent's submit command within the $B{=}150$ budget. (iv) Non-Stuck-in-Loop Rate, the percentage of trajectories that do not contain the same command in three consecutive assistant steps. (v) Total Cost per Instance, the mean combined agent and critic cost in USD per instance, computed from token usage and the rates in App. B.6.

Coding Agents. We evaluate the critic model with six different coding agents: `Qwen3-32B`, `Qwen3-Next-80B-A3B-Instruct` [126], `GLM-4.7-Flash` [134], `GPT-OSS-20B`, `GPT-OSS-120B` [77], and `o3-mini` [78]. The coding agents are frozen throughout. We use the same critic model checkpoint for all coding agents.

Critic Models. We first use a strong proprietary LLM as the critic model, `Claude-Opus-4.6` [6], which is also used as the teacher model in the SFT stage and the LLM-judge in the DPO

Coding Agent	Critic Model	Resolved %	Localized %	Non-Empty %	Non-Loop %
Qwen3-32B	No Critic	8.8	49.2	74.8	92.0
	Qwen3-4B	10.2 (↑1.4)	**55.4** (↑6.2)	**90.0** (↑15.2)	**95.2** (↑3.2)
	+ SFT	11.4 (↑2.6)	52.4 (↑3.2)	74.6 (↓0.2)	90.4 (↓1.6)
	+ SFT + DPO	**14.4** (↑5.6)	52.2 (↑3.0)	69.2 (↓5.6)	95.0 (↑3.0)
	Qwen3-8B	10.0 (↑1.2)	49.4 (↑0.2)	73.0 (↓1.8)	93.0 (↑1.0)
	+ SFT	13.8 (↑5.0)	49.8 (↑0.6)	65.8 (↓9.0)	93.0 (↑1.0)
	Opus 4.6 (upper bound)	26.8 (↑18.0)	67.8 (↑18.6)	82.4 (↑7.6)	92.4 (↑0.4)
Qwen3-Next-80B-A3B	No Critic	20.0	69.8	92.6	95.0
	Qwen3-4B	20.2 (↑0.2)	70.4 (↑0.6)	**99.8** (↑7.2)	97.8 (↑2.8)
	+ SFT	24.2 (↑4.2)	76.8 (↑7.0)	99.2 (↑6.6)	98.2 (↑3.2)
	+ SFT + DPO	**26.2** (↑6.2)	**78.4** (↑8.6)	99.2 (↑6.6)	**98.4** (↑3.4)
	Qwen3-8B	20.6 (↑0.6)	69.4 (↓0.4)	97.6 (↑5.0)	94.8 (↓0.2)
	+ SFT	25.2 (↑5.2)	75.6 (↑5.8)	**99.8** (↑7.2)	96.2 (↑1.2)
	Opus 4.6 (upper bound)	38.2 (↑18.2)	81.4 (↑11.6)	100.0 (↑7.4)	99.4 (↑4.4)
GPT-OSS-20B	No Critic	3.0	11.8	58.4	1.4
	Qwen3-4B	6.8 (↑3.8)	36.6 (↑24.8)	98.8 (↑40.4)	3.0 (↑1.6)
	+ SFT	9.8 (↑6.8)	52.6 (↑40.8)	**99.6** (↑41.2)	5.0 (↑3.6)
	+ SFT + DPO	**14.8** (↑11.8)	**55.0** (↑43.2)	**99.6** (↑41.2)	**5.6** (↑4.2)
	Qwen3-8B + SFT	13.0 (↑10.0)	52.0 (↑40.2)	99.4 (↑41.0)	5.4 (↑4.0)
GLM-4.7-Flash-30B-A3B	No Critic	21.6	34.8	36.0	54.8
	Qwen3-4B	29.8 (↑8.2)	62.2 (↑27.4)	70.4 (↑34.4)	67.2 (↑12.4)
	+ SFT	35.2 (↑13.6)	69.6 (↑34.8)	79.6 (↑43.6)	74.2 (↑19.4)
	+ SFT + DPO	35.8 (↑14.2)	71.0 (↑36.2)	79.2 (↑43.2)	**75.0** (↑20.2)
	Qwen3-8B + SFT	**37.6** (↑16.0)	**73.4** (↑38.6)	**83.4** (↑47.4)	70.6 (↑15.8)
GPT-OSS-120B (medium reasoning)	No Critic	20.4	57.4	95.2	94.2
	Qwen3-4B	16.8 (↓3.6)	46.6 (↓10.8)	**100.0** (↑4.8)	**99.2** (↑5.0)
	+ SFT	31.2 (↑10.8)	75.2 (↑17.8)	99.8 (↑4.6)	98.0 (↑3.8)
	+ SFT + DPO	**34.8** (↑14.4)	**75.8** (↑18.4)	99.2 (↑4.0)	97.2 (↑3.0)
	Qwen3-8B + SFT	31.4 (↑11.0)	74.8 (↑17.4)	99.2 (↑4.0)	96.8 (↑2.6)
o3-mini (200B)	No Critic	19.0	69.4	99.4	**100.0**
	Qwen3-4B	20.4 (↑1.4)	64.4 (↓5.0)	**100.0** (↑0.6)	**100.0** (0.0)
	+ SFT	27.6 (↑8.6)	73.2 (↑3.8)	**100.0** (↑0.6)	99.6 (↓0.4)
	+ SFT + DPO	28.2 (↑9.2)	**75.2** (↑5.8)	**100.0** (↑0.6)	99.4 (↓0.6)
	Qwen3-8B + SFT	**29.4** (↑10.4)	71.0 (↑1.6)	**100.0** (↑0.6)	98.8 (↓1.2)

Table 8.1: Results on SWE-Bench Verified. We report the better result from $k \in \{5, 10\}$ for every configuration. *Non-Loop %* is the fraction of trajectories that do *not* repeat a Bash command in three consecutive steps. "+ SFT" is a critic trained on teacher critiques generated with the high-level prompt (a `CWM-32B` + `Qwen3-Next-80B-A3B` mix). "+ SFT + DPO" further optimizes the 4B SFT critic with DPO on critique preference pairs. All arrows compare to "No Critic" setting. **Bold** highlights the best non-Opus value in each column.

stage. Then we evaluate our trained critic models. We conduct SFT training on both `Qwen3-4B` and `Qwen3-8B` [126]. We conduct SFT+DPO training only on the 4B model due to constraints on computational resources. We additionally evaluate the untrained 4B and 8B models as the critic models. The baseline is denoted as "No critic", where the coding agents complete the task without any guidance. We design and compare with other ablations in §8.3.4.

8.3.2 Implementation Details

Inference Details. All agent uses the mini-SWE-agent scaffold [127], with a budget of 150 steps, $\$6.0$, and 60-seconds per-command. To reduce variance, every critic-guided run shares its first k agent steps with the "No critic" run. This is based on our observation that differences in early steps could cascade into divergent trajectories.

Training Details. We collect training trajectories from 500 instances randomly sampled from R2E-Gym tasks in SWE-Bench repositories (disjoint from SWE-Bench Verified at the instance level). We collect training corpora using the combination of `CWM-32B` and `Qwen3-Next-80B-A3B` as the coding agents. Additional details can be found in section B.2.

8.3.3 Main Results

Table 8.1 reports our main results on SWE-Bench Verified across six agent models.

Our Trained Critic Models Significantly Improve All Six Coding Agents. Across the six agent models in Table 8.1, our critic model improves the resolved rate over the "No Critic" baseline by 5.6%–16.0% absolute gain. All six gains are statistically significant (App. B.7). We can also see that the SFT+DPO method achieves better performance than the SFT-only method for all six coding agents and the SFT-only method always outperforms the untrained critic model and the "No Critic" baseline, which demonstrates the effectiveness of both the SFT and the DPO training stages. Namely, the performance gain mainly comes from training rather than merely interleaving another model into the agent's loop.

Training Improves Performance Across Different Critic Model Sizes. Relative to the matched untrained 4B critic, SFT improves the resolved rate for all six coding agents by 1.2%–14.4%, and DPO adds additional gain of 0.6%–5.0%. Compared to the untrained 8B critic, SFT improves the resolved rate for the two Qwen agents by 3.8% and 4.6%, respectively. The results show that our training data is effective across different critic model sizes.

8.3.4 Detailed Ablations and Analyses

Critic Models Transfer across Programming Languages. As shown in Table 8.2, on Multi-SWE-Bench, the 8B SFT critic model improves the resolved rate by 2.0 points on both `Qwen3-Next-80B-A3B` and `Qwen3-32B`. Its SFT data contain only Python trajectories, whereas this 300-instance benchmark spans nine non-Python languages, showing that the trained critic model's guidance transfers beyond its training language distribution.

Size	Critic Model	Qwen3-Next	Qwen3-32B
8B	No Critic	9.7	1.3
	+ SFT	11.7 (↑2.0)	**3.3** (↑2.0)

Table 8.2: Resolve rate (%) on 300 Multi-SWE-Bench instances. The 8B CWM+Qwen critic uses k=5 and is trained only on Python trajectories. Arrows show gains over the "No Critic" baseline.

Size	Corpus	Qwen3-Next	Qwen3-32B
–	–	20.0	8.8
4B	Qwen-only	**24.6** (↑4.6)	10.8 (↑2.0)
	Qwen+CWM	24.2 (↑4.2)	**11.4** (↑2.6)
8B	CWM-only	23.6 (↑3.6)	11.4 (↑2.6)
	Qwen-only	**26.2** (↑6.2)	10.6 (↑1.8)
	Qwen+CWM	25.2 (↑5.2)	**13.8** (↑5.0)

Table 8.3: Resolved rate (%) of critic models trained with different SFT corpus on SWE-Bench Verified. All critic models use k=5; arrows show gains over no critic.

Critic Models Transfer Across Different Coding Agents in Training and Evaluation. Table 8.3 varies the SFT corpus, where we generate critiques on the trajectories generated by the `CWM-32B` model only (denoted as "CWM-only"), the `Qwen3-8B` model only ("Qwen-only"), and our default SFT corpus that combines both coding agents ("Qwen+CWM"). We can see that the 8B critic model trained on CWM trajectories alone also improves `Qwen3-Next-80B-A3B` and `Qwen3-32B` by $+3.6$ and $+2.6$ points, respectively, showing that the learned steering signal transfers beyond coding agents in training and evaluation. Furthermore, `Qwen3-Next-80B-A3B` favors Qwen-only training at both critic sizes, while Qwen+CWM training produces more balanced gains across the two agents (e.g., with an average gain of 3.4% for the 4B critic model and 5.1% for the 8B model). This indicates that training on the corpora with multiple coding agents yields better overall gain across diverse coding agents.

Our Trained Critic Models Achieve Balance between the Coding Agents' Performance and Cost. As shown in Figure 8.4, on the two Qwen agents, the entire critic-guided system costs \$0.03–\$0.04 per instance with the 8B SFT critic model, versus \$0.17–\$0.27 with `Claude-Opus-4.6`. Namely, our trained critic model achieves a 5.2–$6.0\times$ reduction in total cost. Specifically, on `Qwen3-Next-80B-A3B`, the 8B SFT critic model Pareto-dominates the "No Critic" baseline, which

Size	Critic Model	Qwen3-Next	Qwen3-32B
8B	No Critic	20.0	8.8
	+ SFT (Detailed-Prompt)	24.6 (↑4.6)	12.6 (↑3.8)
	+ SFT (High-Level, Ours)	**25.2** (↑5.2)	**13.8** (↑5.0)

Table 8.4: Resolve rate (%) on SWE-Bench Verified for 8B critic models trained on SFT data with the detailed critiques vs. the high-level teacher critiques (our data).

is strictly better on both resolved rate (25.2% vs. 20.0%) and total per-instance cost ($0.03 vs. $0.08). The reason is that with the critic model, the `Qwen3-Next-80B-A3B` coding agent requires on average fewer steps to resolve a task, which reduces the overall inference cost.

Having High-level Critique Only Improves Training Performance. We compare to an ablation called "Detailed-Prompt" in Table B.1. When we generate SFT data, we do NOT require the teacher critic model to only generate high-level guidance. Namely, the SFT critiques contain both high-level guidance and low-level implementation details such as the concrete tool call commands. Results show that on both Qwen agents, high-level supervision outperforms detailed supervision: 13.8 vs. 12.6% on `Qwen3-32B` and 25.2 vs. 24.6% on `Qwen3-Next-80B-A3B`. This result is consistent with the hypothesis in §8.2.2, that the `Claude-Opus-4.6` teacher's detailed critiques may contain low-level implementation knowledge that the Qwen agents could in principle exploit, but the smaller student critic model cannot reliably learn. The high-level prompt (i.e., our design choice) instead restricts supervision to signals that the trained critic model can more faithfully inherit.

8.3.5 Qualitative Analysis and Case Studies

We conduct qualitative analyses with concrete cases to answer the following research questions: (1) What does the critic model learn from training? And (2) The "Detailed-Prompt" ablation achieves better performance than our "High-level" prompt for the teacher critic model `Claude-Opus-4.6`, is this expected?

What Critic Model Training Removes, and What it Adds. As analyzed in App. B.8.1, the untrained `Qwen3-8B` critic model fails in two ways: only 43.4% of its outputs correctly follow the format in the prompt (vs. 85.8% after SFT) and 19.3% contain hallucinated bash command actions that are not grounded in the coding agent's trajectory (vs. 0% after SFT). In addition, when the untrained critic model does raise an error, it almost always flags only Verification Failures (330/354 firings) with a "run the test script" recovery, pulling the agent off self-review even on instances the base agent would otherwise have salvaged.

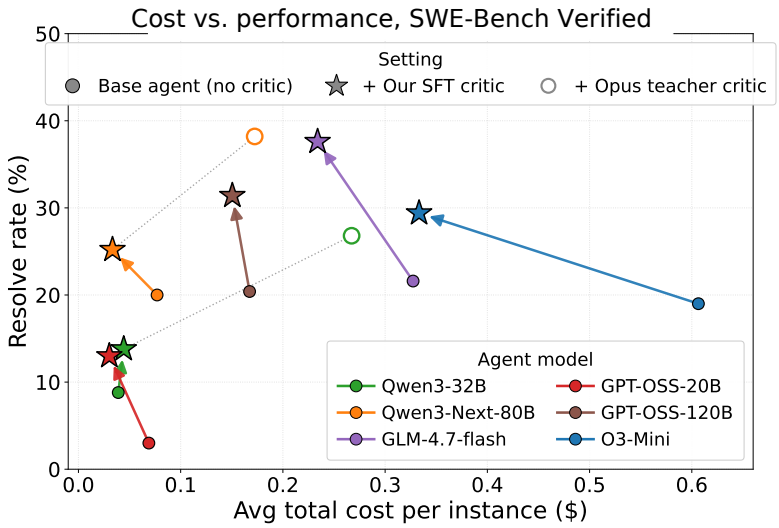


Figure 8.4: Cost versus performance on SWE-Bench Verified across six coding agents. Each agent shows its base (●) and the 8B SFT critic (★); the two Qwen agents additionally show the `Claude-Opus-4.6` teacher critic (○). The cost is for the agent **plus** the critic model.

We also check the evaluation instances where the trained critic model helps the coding agent succeed, but the coding agent alone cannot resolve. The common pattern is a strategy-level redirect: the critic identifies the failure category the agent is in and points to a different class of approach, without dictating the specific code. We provide two case studies that illustrate the pattern in App. B.8.1 and App. B.8.2. On instance `django__django-9296`, `sed` append commands silently duplicate a method each time the pattern matches. The critic jointly flags Step Repetition and Tool Selection Errors and redirects the agent to a heredoc rewrite, which resolves the task. On `sympy__sympy-22714`, a `ModuleNotFoundError: mpmath` blocks execution-based verification; the critic names the import as the root cause and tells the agent to bypass it and read the source directly.

Detailed Critiques Work Better than High-level Ones for Strong Teacher Models Because of Backseat Driving. We show in Table B.1 that the teacher critic model performs overall better when we allow it to generate detailed critiques instead of requiring it to only generate high-level guidance (e.g., `Qwen3-32B` achieves a resolved rate of 40.6% when combined with detailed critiques, but only achieves 26.8% with high-level critiques).

We manually examine the examples and observe that if the critic model suggests a concrete action (e.g., the tool call command), the coding agent is likely to directly copy it in its next action. Table 8.5 presents an analyses on a 50-instance subsample with the `Claude-Opus-4.6` teacher. If we use the "detailed" prompt for the critic model, for 23 out of 50 instances, more than 30% of the lines in the final submitted code patch are directly copied from the critic model.

Prompt	k	Avg. Critique Lines	Copy%	≥ 0.30%
Detailed	5	6.7	0.32	23/50
High-Level	5	1.3	0.01	1/50
Detailed	10	7.1	0.27	10/49
High-Level	10	1.9	0.00	0/49

Table 8.5: `Claude-Opus-4.6` critic model backseat driving on the 50 SWE-Bench Verified mini examples. Avg. Critique Lines denotes the average number of lines in the critiques. Copy% denotes the fraction of lines in the submitted code patch that are directly copied from the critic model. $\geq 0.30\%$ counts the instances where more than 30% of the coding agent's final submission lines are directly copied from the critic model. We test critique frequencies of $k = 5$ and 10.

In comparison, when using the high-level prompt, the coding agent copies more than 30% of the code lines in only $1/50$ of the evaluation instances. The detailed prompt also increases the average number of lines in the generated critiques (e.g., from 1.3 to 6.7 for the $k = 5$ experiments). The results suggest that the detailed critiques' higher resolved rate ($32/50$ vs. $22/50$) is driven by direct copying rather than agent reasoning: when the teacher dictates a single patch, the agent transcribes it and resolves; when the teacher offers multiple candidate fixes without committing, the agent picks the wrong action and writes a confidently incorrect patch.

While the strong proprietary critic model such as `Claude-Opus-4.6` has the capability to generate correct actions, the student critic model has a much smaller model size, even much smaller than the coding agents. Therefore, it is unlikely that the distilled 8B critic model is strong at generating both high-level guidance and concrete actions, and training with high-level critiques only yields better overall performance.

8.4 Conclusion and Future Works

We study whether a small, trained critic model can provide helpful strategy-level signals for a larger coding agent. We construct SFT and DPO training data using a strong proprietary model (`Claude-Opus-4.6`) as the teacher model and the LLM judge. Specifically, for the SFT stage, we design the structure of the critiques, which includes error detection for the coding agent, high-level feedback, and a budget alert. We also design preference data for the DPO stage, focusing on the correctness and clarity of the guidance to future steps. Experiments show that our trained critic models significantly improve six coding agents by 5.6%–16.0%

resolved rates on SWE-Bench Verified. We also demonstrate that our training data transfers across programming languages and coding agents and works for critic models with different sizes. Our detailed analyses further demonstrate the effectiveness of both the SFT and DPO stages and verify the important design choice of training on high-level critiques only.

Future Works. We encourage future researchers to keep working on the direction of critic model training. Specifically, while we observe performance gains in the resolved rate for all coding agents, we do not always improve intermediate metrics such as the non-empty rate or the non-loop rate, and we leave it to future work to have a deep analysis on this phenomenon. Furthermore, in this work, we restrict the frequency of giving critiques to every $k = 5$ or 10 steps. However, it is possible to train a critic model to dynamically select the step to provide feedback. Finally, the usage of critic models is just one example of multi-model collaboration frameworks in inference. We hope there could be more research that studies the inference settings involving multiple agents or models, with the aim of either improving the overall performance or reducing the inference cost.

Chapter 9

Conclusion

This thesis aims to build synthetic training data for coding agents to resolve the three fundamental challenges: the high cost of constructing training data, the difficulty of handling diverse use cases simultaneously, and the performance saturation with respect to data scale.

In Part I, we begin by improving the scalability of training data by finding an appropriate level of abstraction for training environments, with the goal of simplifying the data construction process while preserving effectiveness. We provide a successful approach that simplifies the process of external package installation while remaining effective. Building on this lightweight environment construction process, in Part II, we extend the scope to general-purpose coding agent training and we aim to conduct systematic analyses on the generalizability of coding agents across different tasks. We start with developing the principles for designing synthetic training tasks for task generalization. Finally, in Part III, towards the empirical observation that the performance satuates with respect to data scale after a certain point, with high-level planning ability remaining a major bottleneck, we aim to raise the performance ceiling by training a critic model that specializes in planning to assist the coding agent. We motivate this approach by showing that critic model training can be effective in general NLP tasks such as factuality generation.

For the next steps, we will continue the research on Part II and Part III by completing the two proposed works in chapter 6 and chapter 8. Specifically, for Proposed Work I (Part III), we will verify that the characteristics we find indeed affect the overall performance (**??**) and construct SFT data targeting the beneficial or distracting characteristics we find (**??**). Meanwhile, we will write the research paper and we aim to submit the paper to EMNLP 2026, where the deadline is in May 25, 2026. After we submit the paper for Proposed Work II, we will keep exploring the potential to improve the critic model by constructing DPO data (**??**), which approximately takes one month.

As for Proposed Work II (Part II), we will start with discovering the key agent behaviors by comparing the training trajectories generated by a frontier model, Claude-Sonnet-3.7, and a weaker model, Qwen2.5Coder-32B, with our categorization of agent behaviors proposed in subsection 6.3.1. The effectiveness of the key behaviors will be verified with controlled experiments proposed in subsection 6.2.2. We aim to finish this stage by the end of July 2026. We will iteratively discover the key agent behaviors and verify their effectiveness. The discovery process is automated with an LLM agent, and the verification process is conducted with manually designed controlled experiments. We approximately need 3-4 days to complete the controlled experiment design, conduct the experiment, run training, and run evaluation for each candidate behavior. Then we will extend the analysis to multiple training and evaluation tasks and multiple teacher model pairs as proposed in section 6.3. We aim to finish this stage by the end of August 2026. We will then wrap up the experiment results and write a research paper. We aim to submit the paper to ICLR 2027, where the deadline is September 24, 2026.

After submitting the two papers, I will spend the remaining time on thesis writing and job search. I aim to defend the thesis in May 2027.

Part IV

Appendices

Appendix A

Appendix for Chapter 6

A.1 Additional Related Work

Analyses on the Effectiveness of General Agent Training. Outside SWE, Agent-FLAN [14] shows that agent corpora entangle format following with reasoning and redesigns them to separate the two. With regard to reasoning, the structure and behaviors of a trace matter more than its correctness: with verification, backtracking, and related behaviors, RL would be able to gain even from incorrect solutions [24], and long-CoT distillation tolerates wrong answers but not shuffled or deleted steps [52]. Furthermore, Jin et al. [45] find the same for agentic search, where SFT on trajectories exhibiting beneficial behaviors before RL outperforms SFT on outcome-correct ones.

A.2 Experimental Setup

Datasets and Models. We conduct evaluation on the SWE-bench Verified dataset [44] and analysis on both in-domain training: the R2E-Gym issue-solving dataset [41], and transfer learning: the Hybrid-Gym function localization dataset [123]. We experiment on 3 student models: Qwen2.5Coder-7B [34], Qwen3-4B, and Qwen3-8B [126] and 8 teacher models: Claude-Opus-4.5 [5], Claude-Opus-4.7 [7], Claude-Sonnet-3.5 [3], GPT-5-mini [75], GPT-5.5 [80], DeepSeek4-flash [16], Qwen3-80B [126], and Qwen3.5-397B [102].

Training Details. We fine-tune each student on the teacher demonstrations with supervised fine-tuning: we use 8 A6000 GPUs and search hyper-parameters in the range of learning-rate={5e-5, 1e-4}, num-epochs={3, 5}, and batch-size={8, 16}.

Training and Evaluation Prompts. All Hybrid-Gym teacher rollouts use the function local-

ization prompt in Figure A.1, which asks for a single function or class, except the strategy-prompted rollouts in §A.6. The students are evaluated on SWE-Bench Verified [44] with the issue-solving prompt in Figure A.3. On R2E-Gym, whose images come with a ready virtual environment instead of conda, the Claude-Sonnet-3.5 rollouts come from the original R2EGym dataset and use the original R2E-Gym prompt [41] (Figure A.2). Other models use the same prompt as our SWE-Bench Verified evaluation.

Function Localization Prompt

```
I've uploaded a python code repository in the directory {{ workspace_dir_name }}.
Your goal is to locate the {{ module_type }} described below and write its docstring.

<{{ module_type }}_description>
{{ description }}
</{{ module_type }}_description>

Known details:
- Target type:  {{ module_type }}
- The original docstring was removed; please restore/author it.
- Note:  The target name is not provided.  You must identify the correct {{ module_type }} based on the
functionality description above.

Follow these steps:
1.  EXPLORATION: Search the codebase (no file path provided) to find where this {{ module_type }} is
defined.
2.  UNDERSTANDING: Read the implementation to fully understand its arguments, return values, and side
effects.
3.  RECHECK: Verify that the {{ module_type }} you located actually matches the given description.  Compare
the implementation behavior with the description to ensure you found the correct target.
4.  GENERATION: Insert a clear, concise Python docstring inside the definition using triple quotes.  Cover
purpose, parameters, return value(s), and important behaviors or exceptions.
5.  REVIEW: Double-check formatting and accuracy; avoid changing code outside the docstring unless strictly
necessary.
Be thorough in your exploration and reasoning.  It's fine if your thinking process is lengthy - quality and
completeness are more important than brevity.
```

Figure A.1: The Hybrid-Gym function localization prompt. `module_type` is `function` or `class`.

Original R2E-Gym Prompt

```
Consider the following github issue:
<github_issue>
{{ instance.problem_statement }}
</github_issue>

Can you help me implement the necessary changes to the repository to fix the <github_issue>?
```

```
I've already taken care of all changes to any of the test files described in the <github_issue>.  This means
you DON'T have to modify the testing logic or any of the tests in any way!
Your task is to make the minimal changes to non-tests files in the /testbed directory to ensure the
<github_issue> is satisfied.

IMPORTANT TIP:
Follow these steps to resolve the issue:
1.  As a first step, it might be a good idea to explore the repo to familiarize yourself with its structure.
2.  Create a script ('reproduce_issue.py') to reproduce the error and execute it to confirm the error
3.  Edit the sourcecode of the repo to resolve the issue
4.  Rerun your reproduce script and confirm that the error is fixed!
5.  Think about edgecases and make sure your fix handles them as well
```

Figure A.2: The R2E-Gym prompt [41], used by some of the R2E-Gym rollouts.

Issue-Solving Prompt

```
I have access to a python code repository in the directory {{ instance.repo_path }} .  You can explore and
modify files using the available tools.  Consider the following issue description:

<issue_description>
{{ instance.problem_statement }}
</issue_description>

Can you help me implement the necessary changes to the repository so that the requirements specified in the
<issue_description> are met?
I've already taken care of all changes to any of the test files described in the <issue_description>.  This
means you DON'T have to modify the testing logic or any of the tests in any way!
Also the development Python environment is already set up for you (i.e., all dependencies already
installed), so you don't need to install other packages.
Your task is to make the minimal changes to non-test files in the {{ instance.repo_path }} directory to
ensure the <issue_description> is satisfied.

Follow these phases to resolve the issue:

Phase 1.  READING: read the problem and reword it in clearer terms
1.1 If there are code or config snippets.  Express in words any best practices or conventions in them.
1.2 Hightlight message errors, method names, variables, file names, stack traces, and technical details.
1.3 Explain the problem in clear terms.
1.4 Enumerate the steps to reproduce the problem.
1.5 Hightlight any best practices to take into account when testing and fixing the issue

Phase 2.  RUNNING: install and run the tests on the repository
2.1 Activate the environment by running
./opt/miniconda3/etc/profile.d/conda.sh ; conda activate testbed
2.2 Follow the readme
2.3 Install the environment and anything needed
2.4 Iterate and figure out how to run the tests

Phase 3.  EXPLORATION: find the files that are related to the problem and possible solutions
3.1 Use `grep` to search for relevant methods, classes, keywords and error messages.
3.2 Identify all files related to the problem statement.
3.3 Propose the methods and files to fix the issue and explain why.
3.4 From the possible file locations, select the most likely location to fix the issue.

Phase 4.  TEST CREATION: before implementing any fix, create a script to reproduce and verify the issue.
4.1 Look at existing test files in the repository to understand the test format/structure.
4.2 Create a minimal reproduction script that reproduces the located issue.
4.3 Run the reproduction script to confirm you are reproducing the issue.
4.4 Adjust the reproduction script as necessary.

Phase 5.  FIX ANALYSIS: state clearly the problem and how to fix it
5.1 State clearly what the problem is.
5.2 State clearly where the problem is located.
5.3 State clearly how the test reproduces the issue.
5.4 State clearly the best practices to take into account in the fix.
5.5 State clearly how to fix the problem.
```

```
Phase 6.  FIX IMPLEMENTATION: Edit the source code to implement your chosen solution.
6.1 Make minimal, focused changes to fix the issue.

Phase 7.  VERIFICATION: Test your implementation thoroughly.
7.1 Run your reproduction script to verify the fix works.
7.2 Add edge cases to your test script to ensure comprehensive coverage.
7.3 Run existing tests related to the modified code to ensure you haven't broken anything.

8.  FINAL REVIEW: Carefully re-read the problem description and compare your changes with the base commit {{
instance.base_commit }}.
8.1 Ensure you've fully addressed all requirements.
8.2 Run any tests in the repository related to:
8.2.1 The issue you are fixing
8.2.2 The files you modified
8.2.3 The functions you changed
8.3 If any tests fail, revise your implementation until all tests pass

Be thorough in your exploration, testing, and reasoning.  It's fine if your thinking process is lengthy -
quality and completeness are more important than brevity.
```

Figure A.3: The issue-solving prompt used to evaluate the students on SWE-Bench Verified and used in some of the R2E-Gym rollouts.

A.3 Computation of Significance

With the paired bootstrap test of Berg-Kirkpatrick et al. [9], we test whether two students differ in resolved rate on SWE-Bench Verified. Let $\delta(x)$ be the gain in resolved rate of the better student over the other on the 500 instances x. We draw $b = 10^6$ samples $x^{(i)}$ of 500 instances from x with replacement and estimate the one-sided p-value as the fraction of samples with $\delta(x^{(i)}) \geq 2\delta(x)$; the threshold is $2\delta(x)$ rather than $\delta(x)$ because the samples are centered at $\delta(x)$ instead of at 0. We mark a change as significant if $p < 0.05$.

A.4 List of Candidate Behaviors

In Table A.1, we provide a list of candidate behaviors on which we do not conduct further experiments. The reason is that we prioritize the behaviors that are more likely to be useful for training.

A.5 Detailed Analysis Results

A.5.1 Exploration Length is Not a Good Indicator of Trajectory Effectiveness

Following §6.3.2, we show that the exploration length of demonstrations is, however, not a good indicator of trajectory effectiveness.

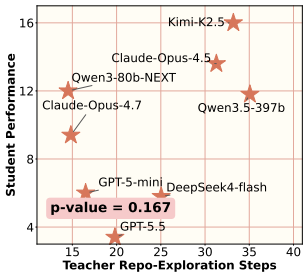


Figure A.4: The exploration length of demonstrations is also not a good indicator of demonstration effectiveness.

A.5.2 Strong Teacher Does Not Always Translate to Strong Student

Some Claude-Opus-4.7 demonstrations open the target file without saying why they chose it, as is shown in Table A.2. On an instance that asks for a class with "attributes [...] such

Behavior	Judge Metric
Search frequency	The frequency of search (≥ 15: high; 7–14: medium; < 7: low)
Search-view ratio	The ratio of keyword searching to file-viewing commands is > 3
Multi-round file view	Run file-viewing actions for ≥ 3 times
Start with thinking	Use the `<function=think>` tool as the first action
Start with task-tracker	Use the `<function=task_tracker>` tool as the first action
Frequent command chain	Use command chaining on 50% or more of terminal commands
View file before edit	Has a file-viewing action before the first file-editing action
Use Git commands	Use `git log`, `git diff`, or other Git commands

Table A.1: The candidate behaviors discovered by **CATRadar** that we do not conduct further experiments on.

as their nickname, username, and hostname", Claude-Opus-4.7 lists the repository's Python files, cut at 50 by `head`, and then opens the 9th entry, the target file, saying only "The file `sopel/tools/target.py` looks promising."; it runs no content search such as `grep` and does not narrow the listing in any form. In other words, the trajectory itself does not show any reason why it picks the target file out of the 50 searched results. In comparison, Claude-Opus-4.5 has a clearer strategy for file localization. It first names four candidate files with a reason each, and it checks where the class is used before editing it. Similarly, for function localization, on `open-mmlab__mmengine-304_415`, Claude-Opus-4.7 likewise opens the 44th entry of a 50-file listing ("The class description matches `base_model.py`."), while Claude-Opus-4.5 first lists the six requirements of the description and browses two directories.

A.5.3 Non-Tool-Call Text Increases Trajectory Length

Similar to §6.3.6, we observe that when we only increase the verbosity of non-tool call text, the trajectory length in general increases, but the performance doesn't change much.

A.6 Concrete Data Intervention Implementation for each Candidate Behavior

In this section, we introduce how we design experiments to control the presence or absence of each candidate behavior in training demonstrations. We manually design different experi-

Claude-Opus-4.7 (5 actions)	**Claude-Opus-4.5 (16 actions)**
1 `find <repo> -type f -name "*.py" | head -50 && ls <repo>` → 50 file paths (listing cut by `head`)	1 `view /workspace` → directory tree
2 "The file `sopel/tools/target.py` looks promising. Let me check it." `view sopel/tools/target.py` → **target class** `User`	2 `view sopel/` → directory tree
3 `str_replace`: add the docstring to `User`	3 `think`: "[...] this sounds like a User class in an IRC context. Let me look at relevant files: 1. `sopel/tools/target.py` - the name 'target' could represent users/channels 2. `sopel/trigger.py` - might contain user-related classes 3. `sopel/bot.py` - could contain user tracking classes 4. `sopel/irc.py` - IRC-specific classes"
4 `python -c "from sopel.tools.target import User; print(User.__doc__)"`	4 `view sopel/tools/target.py` → **target class** `User`
5 `finish`	5 `think`: "I found the `User` class [...]"
	6 `grep` for the modules that import `User`
	7–8 `view sopel/bot.py`, `view sopel/coretasks.py`
	9 `think`: "Now I'm certain the `User` class [...] is the correct target."
	10 `str_replace`: add the docstring to `User`
	11–16 view the edited file, import check, run the test suite (3 commands), `finish`

Table A.2: Demonstrations of Claude-Opus-4.7 and Claude-Opus-4.5 on the Hybrid-Gym instance `sopel-irc__sopel-1344_101` (both successful). `<repo>` abbreviates the repository path, and file paths are shown relative to it. Claude-Opus-4.7 opens the target file directly from a listing cut at 50 files, while Claude-Opus-4.5 takes a more comprehensive strategy: picking several candidate files with detailed reasoning, and then reading them one-by-one (action 3).

ments for different behaviors.

A.6.1 Label Demonstrations with the Presence of Repo-Exploration Strategies

We label the two strategies with rule-based judges.

Multi-Search. We define keyword search as a `terminal` command that contains `grep` or `rg`,

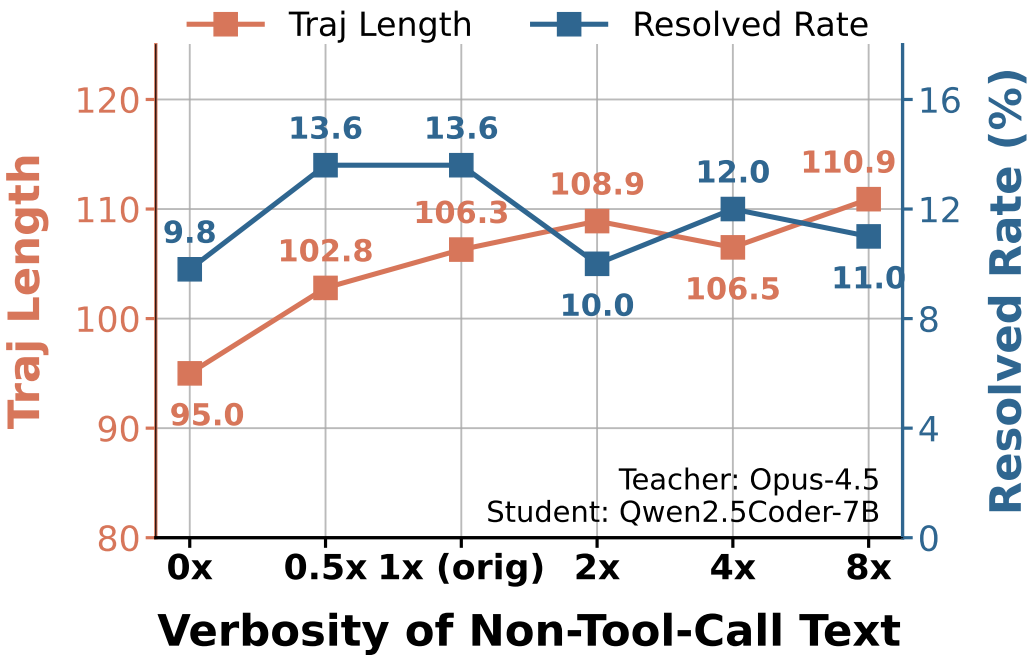


Figure A.5: When we increase the verbosity of non-tool call text, the trajectory length in general increases. However, the performance does not change much.

or `find` with `-name`. A trajectory exhibits the "Multi-Search" behavior if it runs at least two keyword searches before its first file-editing action (`file_editor` with `create`, `str_replace`, or `insert`, or a `sed -i` or `git apply` command).

Target-Read. We take the first action that is a `file_editor view` (of a file or a directory), a `terminal` command that contains `cat`, `head`, `tail`, `less`, or `more`, or a file-editing action. Its path is the `file_editor` path, or the first argument after `cat`, `head`, `tail`, `less`, or `more` in the command. The trajectory exhibits Target-Read if an earlier keyword search lists at most 30 distinct files and its output contains this path: a listed path equals it, ends with it, or is a suffix of it, has the same file name, or the path or its file name occurs in the output text.

A.6.2 Prompt the Models to Adopt our Repo-Exploration Strategies

This section introduces the experimental settings for the "prompt teacher" and "prompt student" ablations in Table 6.5.

Prompt Teacher. To instruct the teacher to follow the strategies for the Hybrid-Gym function localization task, we add the `<localization_strategy>` block in Figure A.6 after the steps of the function localization prompt (Figure A.1) and end step 1 with "Use the localization strat-

egy below." GPT-5-mini produces 1,289 successful demonstrations with this prompt, which keep the strategy block in their task prompt. It produces 1,346 successful demonstrations with the original prompt (i.e., without the strategy block). We train the student on all 1,289 demonstrations and evaluate it with the unchanged issue-solving prompt (Figure A.3).

Prompt Student. As for the "prompt student" ablation, we train the student model on the original 1,346 GPT-5-mini demonstrations. In inference, we insert the `<repo_exploration_strategy>` block in Figure A.6 in the prompt for the SWE-Bench Verified issue-solving task, which states the same principles for issue solving, after Phase 3 (EXPLORATION) of the issue-solving prompt at evaluation time.

A.6.3 Add Task-Tracker Steps to Training Demonstrations

Neither Claude-Opus-4.7 nor GPT-5-mini uses any task-tracker tool calls in its original Hybrid-Gym demonstrations. To implement the "Add Task-Tracker" ablation, we insert LLM-synthesized task-tracker plans at four points of each demonstration: (1) before the first action; (2) once the target file is found, i.e., before the last action prior to the first edit that mentions the target file (the file of the last successful edit); (3) before the first file-editing action; and (4) right after the output of the first file-editing action. If two points coincide, only the earlier point's plan is inserted. Each plan is a task-tracker action with `command=plan`, followed by the tool's usual output, "Task list has been updated with N item(s)."
We use an LLM (DeepSeek4-flash, DeepSeek-AI and et al. [16]) to write the plans at temperature 0.3 without seeing the demonstration: the first plan from the task description, asked to list 4–6 tasks with the first one in progress, and each later plan from the task description and the first plan, asked to keep the task titles and update their status. For GPT-5-mini, 1,325 demonstrations receive four plan turns and 21 receive three, and every planning step is consistent with the remaining part of the trajectory. For Claude-Opus-4.7, 1,466 demonstrations receive four plan turns and one receives three. However, we cut off 354 of these turns because they change task titles in a later plan item, or generate too few or too many plans in the first task-planner tool call.

A.6.4 Add Verification Steps to Training Demonstrations

As introduced in §6.3.5, we take the verification steps from Claude-Opus-4.5, whose demonstrations contain more of them (Table 6.3). For each Claude-Opus-4.7 demonstration whose instance also has a successful Claude-Opus-4.5 demonstration that first edits the same file (the same path relative to the repository root), we keep the Claude-Opus-4.7 trajectory before its

Strategy Block for the Teacher

```
<localization_strategy>
When searching for the target {{ module_type }}, follow these two principles
(summarized from successful trajectories):

1.  MULTI-ROUND REFINEMENT. Treat localization as iterative.  After each
search, look at the matches as a whole, pick the most promising signal,
and run a follow-up search refining on it (different keyword, restricted
path, different surrounding pattern).  Do not commit to a single candidate
on the first search - expect 2-3 rounds of refinement before you have a
short list.

2.  READ AFTER NARROWING. Do not open and read full file contents until you
have narrowed to a small handful (<=3) of plausible candidates.  Opening
files too early wastes context on irrelevant code.  Once you have a short
list, then read each candidate's definition and compare against the
description to pick the correct one.
</localization_strategy>
```

Strategy Block for the Student

```
<repo_exploration_strategy>
When searching the repository for the file(s)/function(s) relevant to the issue,
follow these two principles (summarized from successful trajectories):

1.  MULTI-ROUND REFINEMENT. Treat localization as iterative.  After each search,
look at the matches as a whole, pick the most promising signal, and run a
follow-up search refining on it (different keyword, restricted path, different
surrounding pattern).  Do not commit to a single candidate on the first search -
expect 2-3 rounds of refinement before you have a short list.

2.  READ AFTER NARROWING. Do not open and read full file contents until you have
narrowed to a small handful (<=3) of plausible candidates.  Opening files too
early wastes context on irrelevant code.  Once you have a short list, read each
candidate's definition and compare against the issue description to pick the
correct one.
</repo_exploration_strategy>
```

Figure A.6: The strategy blocks added to the teacher's function localization prompt (first box; `module_type` is `function` or `class`) and to the student's issue-solving prompt (second box).

first file-editing action and append the Claude-Opus-4.5 trajectory from its first file-editing action on, with the workspace directory renamed. That is to say, we copy both the file-editing

steps and verification steps from Claude-Opus-4.5's trajectories for the consistency of trajectories. This applies to 1,389 of the 1,467 demonstrations. We keep the remaining 78 unchanged.

A.6.5 Verbosity Experiments

We use DeepSeek4-flash (with thinking disabled) [16] to rewrite the Claude-Opus-4.5 demonstrations of Hybrid-Gym, which have 30,615 assistant turns, including 3,738 `think` and 683 task-tracker actions.

For every multiple $k > 0$, only the reasoning text changes: every tool call apart from the `thought` parameter of `think` calls, and every tool output, stays byte-identical, which we verify on the released datasets.

Non-tool-call text. For Figure A.5, we rewrite the text before the tool call to k times its own length in Qwen3 tokens, for $k \in \{0.5, 2, 4, 8\}$, in every turn except `think` and task-tracker turns. 10,513 of these 26,194 turns have text; the others, including 4 with a malformed tool call, stay unchanged. The rewriter sees only the turn's text and its tool call and is told to keep the meaning and intent, not to invent facts, findings, file contents, or results that the text and the call do not imply, and to write as the agent at this step: longer ablations elaborate on what the agent is looking for, why, and what it expects to learn, without adding steps or discoveries; shorter ablations condense the same content.

Different ablations of a turn are written as a ladder in one request: the $0.5\times$ ablation condenses the original, the $2\times$ ablation elaborates it, and the $4\times$ and $8\times$ ablations each elaborate the previous one. Furthermore, an ablation is accepted if its length is within $\pm 25\%$ of the target; otherwise it is requested again with its measured length as feedback, for at most three rounds, after which the turn keeps its original text, as do turns whose target is below 4 or above 2,000 tokens.

On average, the rewritten texts are 0.5, 1.9, 3.9, and 8.1 times as long as the originals. Out of the 10,513 turns with text, 0.8%, 1.3%, 3.0%, and 4.1% keep their original text in the first release described below. The second release has the same rounded rates except at $8\times$ (4.2%).

The $0\times$ variant removes the text of all turns except the `think` and task-tracker turns.

Thinking Content. For Figure 6.6, we also rewrite the `thought` parameter of each `think` call, and the text before the call, to the same multiple of its own length; the other turns are as in the non-tool-call setting with the same k. The rules for thoughts are the same, adapted to the agent's reasoning: keep its reasoning, conclusions, and concrete references (files, functions, line numbers), and reach no new conclusions or plan different steps. Thoughts in the trajectories are long (median 304 tokens), so every ablation above 600 tokens is written in several

parts, each continuing the text so far, up to 8,000 tokens. The $0\times$ variant removes all `think` actions with their outputs and the text of all other turns except the task-tracker turns, which stay whole (99 of them have text).

Post-Checks. We do not check the meaning of the rewrites. Instead, we check: (1) whether the rewriting describes the rewriting task instead of the agent's reasoning (e.g., "the thought doesn't mention testing, so I won't plan that"; 4.6%, 10.0%, and 19.8% of the thoughts at $2\times$, $4\times$, and $8\times$) or ended with JSON left from the rewriter's reply (3.3% of the rewritten $2\times$ and $4\times$ texts); (2) whether the written reasoning text is natural and consistent with the remaining part of the trajectory. We keep the original reasoning text if the rewriting does not meet any of the requirements.

Appendix B

Appendix for Chapter 8

B.1 Analyses on the Gap between Teacher and Student Critic Models

Error Categorization Results. We first compute the agreement of error categorization between the student and teacher critic models. As shown in Figure B.1, the agreement rate is high for major error categories such as step repetition (F1=0.91) and task derailment (F1=0.73). The overall average Micro-F1 score is 0.74.

Overall Guidance Quality. Then we prompt an LLM judge to check whether the guidance generated by both the teacher and student models is both correct and well-specified. We use a different model (`DeepSeek V4 Pro`) as the judge to avoid self-bias. The results are shown in Figure B.2, the SFT critic model has a significantly lower rate of generating correct and well-specified guidance compared to the teacher critic model.

B.2 Training and Inference Details

B.2.1 Inference Details

Scaffold. All agent runs use the mini-SWE-agent scaffold [127], with a budget of 150 steps, $\$6.0$ and 60-seconds per-command.

Sampling and serving. Open-weight models are served with vLLM[1] on $8\times$L40S; `Claude-Opus-4.6` is accessed via its hosted API. We sample at `temperature` 0, `top-p` 1, with a per-turn output

[1] `https://github.com/vllm-project/vllm`

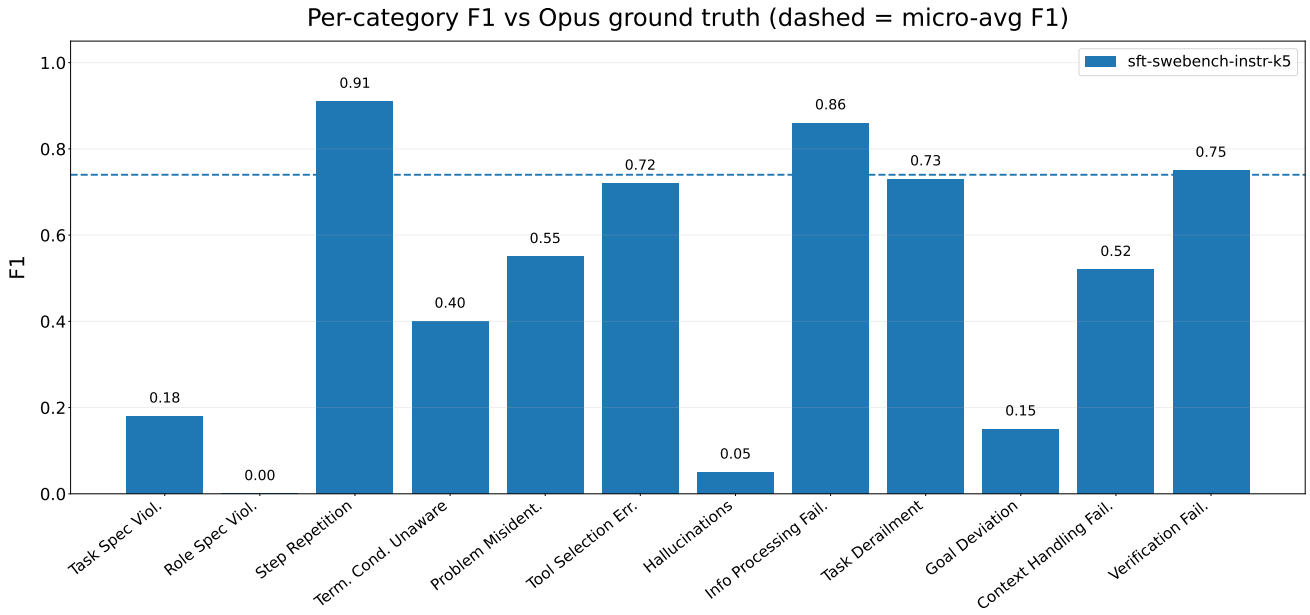


Figure B.1: Error categorization results.

cap of 2048 tokens. For `GPT-OSS`, we use `medium` reasoning effort.

Shared prefix. Every critic-guided run shares its first k agent steps with the base run. This isolates the critic's contribution from rollout variance: small differences in early actions otherwise cascade into divergent observations (e.g., reading lines 50-100 instead of 100-150 of a file).

Default settings. Our trained critic uses the **high-level** prompt and budget-aware instructions (§8.2.2).

B.2.2 Training Details

Training trajectories are collected on 500 instances randomly sampled from R2E-Gym tasks in SWE-Bench repositories (disjoint from SWE-Bench at the instance level), querying `Claude-Opus-4.6` every $k = 5$ agent steps for trajectory feedback. We collect two corpora: a `CWM-32B`-only corpus, and a mixed corpus that adds `Qwen3-Next-80B-A3B` trajectories on top of it. The mixed corpus is used for the main trained critics; the `CWM-32B`-only corpus is used for the transfer experiment (§8.3.3). We denoise the data by discarding teacher responses that lack the expected error-taxonomy markers (likely format failures or agent text mistakenly logged as feedback). To fit the 32,768-token context, we drop the oldest trajectory and feedback blocks first while preserving the system prompt, task description, final prompt, and teacher response in full. The `CWM-32B`-only corpus yields 4,532 training samples under the high-level prompt and 3,135

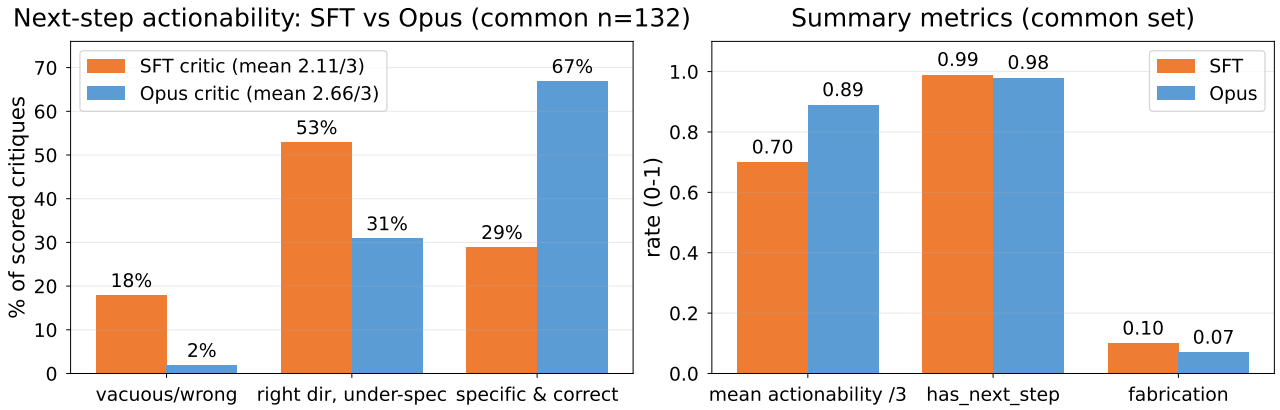


Figure B.2: Guidance quality analysis.

under the detailed prompt; the mixed corpus adds a comparable shard of `Qwen3-Next-80B-A3B` samples. We train `Qwen3-4B` and `Qwen3-8B` for 3 epochs with AdamW (peak LR 5×10^{-6}, cosine schedule, 10% linear warmup, bfloat16); loss is computed only on teacher critique tokens, with agent actions, observations, and prior critiques kept in context but masked out. Global batch size is 8 across $8\times$L40S with gradient checkpointing.

B.3 Method Preliminaries

Division of labor. The coding agent π_A handles concrete repository interaction: inspecting files, running commands, editing code, and deciding when to submit. The critic instead provides high-level judgments about the *trajectory*: whether the agent is making progress, pursuing an unsupported assumption, repeating an unproductive strategy, or needs to reconsider its overall direction.

Why high-level feedback. We do not want the critic to become a second actor that "backseat drives" π_A with commands, file-level instructions, or step-by-step recovery plans. The coding agent is already the more capable model for concrete tool use and code editing; asking a smaller critic to prescribe specific actions risks replacing its reasoning with weaker instructions. We therefore train the critic to provide *high-level strategic feedback*, not action-level control.

Scaffold details. The scaffold gives the coding agent a simple command-line interface rather than a large set of specialized tools. Keeping the scaffold minimal lets us study the effect of the critic without confounding it with scaffold-specific machinery. Each task instance x consists of a natural-language issue description and a repository snapshot. At each step, the coding agent π_A may reason about the task, execute shell commands, inspect files, edit code, or submit a final patch. A run succeeds if the submitted patch passes the benchmark's evaluation harness.

B.4 Critic Prompts

This appendix gives the two critic prompts compared in §8.2.2 and §8.3.3: the *high-level* prompt that constrains the critic to high-level feedback, and the *detailed* prompt that permits action-level prescriptions. Both prompts adapt the trajectory-feedback prompt of Gandhi et al. [26] and share its 12-category trajectory-level error taxonomy. The detailed prompt uses the upstream version unchanged; the high-level prompt adds the framing rules listed in §8.2.2, removes the per-category recovery hints, and tightens the example response. Both critics receive the same agent-side context (task description, trajectory prefix, and prior critic feedback history) at every k steps. Once the agent passes step 100 of its 150-step budget, the budget-aware instructions in §B.4.3 are appended to whichever prompt is in use; they prioritize recommending submission as the budget approaches.

B.4.1 High-Level Prompt

```
You are a supervisor monitoring an LLM-based code agent. Your role is to detect trajectory-level errors and provide
↪ corrective guidance to prevent task failure.
Analyze the provided agent trace for the following error categories. For each detected error, provide specific
↪ evidence and a recovery action.

The agent works in an isolated repo; assume no hidden external edits. There is EXACTLY ONE final submission command;
↪ it is ALWAYS essential.

IMPORTANT RULES:
- Respond ONLY with analytical feedback in the format below. Never generate bash commands, code blocks, or simulated
↪ agent/environment output.
- Be CONSERVATIVE. Default to "On track" unless you see clear evidence of a problem. The agent is competent - let it
↪ work. Unnecessary interventions cause the agent to second-guess itself and loop, which is worse than a slow but
↪ steady trajectory.
- Use "Critical intervention required" ONLY for: (1) source file is corrupted/broken by a bad edit, or (2) agent has
↪ repeated the exact same action 3+ times with no progress. Most other issues are "Needs correction" at worst.
- NEVER suggest specific technical approaches, code changes, or where in the code to look. The agent understands the
↪ codebase better than you. If the agent's approach seems wrong, say so, but do NOT suggest an alternative - let the
↪  agent decide.
- If the agent has applied a fix and verified it works, tell it to submit immediately.
```

- Keep your response SHORT. 1-2 sentences per detected error. 1-2 sentences for OVERALL_GUIDANCE. Long feedback is counterproductive.

SPECIFICATION ERRORS:

1. Task Specification Violations
Definition: Agent fails to adhere to task constraints or requirements

2. Role Specification Violations
Definition: Agent behaves outside its defined role/responsibilities

3. Step Repetition
Definition: Unnecessary repetition of completed steps or actions

4. Termination Condition Unawareness
Definition: Agent continues working when task completion criteria are met

REASONING ERRORS:

5. Problem Misidentification
Definition: Agent misunderstands the core problem or current subtask

6. Tool Selection Errors
Definition: Agent uses inappropriate tools for the current task

7. Hallucinations
Definition: Agent generates false information or fabricates tool outputs

8. Information Processing Failures
Definition: Poor retrieval of relevant information or misinterpretation

COORDINATION ERRORS:

9. Task Derailment
Definition: Agent deviates from intended objective or loses focus

10. Goal Deviation
Definition: Agent pursues goals that don't serve the main objective

11. Context Handling Failures
Definition: Agent loses important context or forgets previous findings

12. Verification Failures
Definition: Inadequate checking of work quality or correctness

==========
RESPONSE FORMAT
==========

For each error category, respond with:
DETECTED: Yes/No
EVIDENCE: (if detected) One sentence.
RECOVERY_ACTION: (if detected) One sentence. No code, no commands.

```
Then provide:
TASK_STATUS: Exactly one of: "On track" / "Needs correction" / "Critical intervention required"
OVERALL_GUIDANCE: 1-2 sentences only.

EXAMPLE:

SPECIFICATION ERRORS:
1. Task Specification Violations: DETECTED: No
2. Role Specification Violations: DETECTED: No
3. Step Repetition: DETECTED: Yes
EVIDENCE: Agent ran the same test command three times with identical results.
RECOVERY_ACTION: Stop repeating and analyze the existing test output.
4. Termination Condition Unawareness: DETECTED: No

REASONING ERRORS:
5. Problem Misidentification: DETECTED: No
6. Tool Selection Errors: DETECTED: No
7. Hallucinations: DETECTED: No
8. Information Processing Failures: DETECTED: No

COORDINATION ERRORS:
9. Task Derailment: DETECTED: No
10. Goal Deviation: DETECTED: No
11. Context Handling Failures: DETECTED: No
12. Verification Failures: DETECTED: No

TASK_STATUS: Needs correction
OVERALL_GUIDANCE: You are repeating the same test. Analyze the output you already have and proceed to the next step.

Now review the agent's trajectory and provide your supervisor feedback.
```

B.4.2 Detailed Prompt

The detailed prompt below is reproduced verbatim from Gandhi et al. [26] for reproducibility. The high-level prompt above is our modification of it; see §8.2.2 for the added framing rules.

```
You are a supervisor monitoring an LLM-based code agent. Your role is to detect trajectory-level errors and provide
↪ corrective guidance to prevent task failure.
Analyze the provided agent trace for the following error categories. For each detected error, provide specific
↪ evidence and a recovery action.

The agent works in an isolated repo; assume no hidden external edits. There is EXACTLY ONE final submission command;
↪ it is ALWAYS essential.

SPECIFICATION ERRORS (System Design Issues)

1. Task Specification Violations
Definition: Agent fails to adhere to task constraints or requirements
Recovery: Redirect agent to original task requirements
```

2. Role Specification Violations
Definition: Agent behaves outside its defined role/responsibilities
Recovery: Remind agent of its specific role and boundaries

3. Step Repetition
Definition: Unnecessary repetition of completed steps or actions
Recovery: Acknowledge completed work and guide to next logical step

4. Termination Condition Unawareness
Definition: Agent continues working when task completion criteria are met
Recovery: Signal completion criteria and instruct to finalize

REASONING ERRORS (Decision Making Issues)

5. Problem Misidentification
Definition: Agent misunderstands the core problem or current subtask
Recovery: Clarify the actual problem and expected approach

6. Tool Selection Errors
Definition: Agent uses inappropriate tools for the current task
Recovery: Suggest correct tools and explain their appropriate usage

7. Hallucinations
Definition: Agent generates false information or fabricates tool outputs
Recovery: Request verification of claims against actual evidence

8. Information Processing Failures
Definition: Poor retrieval of relevant information or misinterpretation
Recovery: Guide agent to correct information sources and interpretation

COORDINATION ERRORS (Process Management Issues)

9. Task Derailment
Definition: Agent deviates from intended objective or loses focus
Recovery: Realign agent with original objectives and priorities

10. Goal Deviation
Definition: Agent pursues goals that don't serve the main objective
Recovery: Refocus on primary goals and expected outcomes

11. Context Handling Failures
Definition: Agent loses important context or forgets previous findings
Recovery: Provide context summary and key information recap

12. Verification Failures
Definition: Inadequate checking of work quality or correctness
Recovery: Instruct specific verification steps and quality checks

==========
RESPONSE FORMAT
==========

For each error category, respond with:

```
DETECTED: Yes/No
EVIDENCE: Specific quote or observation from trace (if detected)
RECOVERY_ACTION: Specific instruction to correct the error (if detected)

Then provide:
TASK_STATUS: On track / Needs correction / Critical intervention required
OVERALL_GUIDANCE: Detailed and specific guidance for the agent

Example Response Structure

SPECIFICATION ERRORS:
1. Task Specification Violations: DETECTED: No
2. Role Specification Violations: DETECTED: No
3. Step Repetition: DETECTED: Yes
EVIDENCE: "Agent ran the same test command three times: 'pytest test_file.py'"
RECOVERY_ACTION: "The test has already been executed successfully. Proceed to analyze the results and move to the
↪ next development step."
4. Termination Condition Unawareness: DETECTED: No

REASONING ERRORS:
5. Problem Misidentification: DETECTED: No
6. Tool Selection Errors: DETECTED: Yes
EVIDENCE: "Agent used text editor to run Python code instead of using the Python interpreter"
RECOVERY_ACTION: "Use the Python interpreter tool for code execution. The text editor is for viewing and modifying
↪ files only."
7. Hallucinations: DETECTED: No
8. Information Processing Failures: DETECTED: No

COORDINATION ERRORS:
9. Task Derailment: DETECTED: No
10. Goal Deviation: DETECTED: No
11. Context Handling Failures: DETECTED: No
12. Verification Failures: DETECTED: No

TASK_STATUS: Needs correction
OVERALL_GUIDANCE: You are repeating actions unnecessarily and using incorrect tools. Specifically:
1. Stop running the same test command repeatedly - the test 'pytest test_file.py' has already been executed
↪ successfully three times with the same result
2. Use the Python interpreter tool for executing Python code, not the text editor which is only for viewing and
↪ modifying files
3. Now focus on analyzing the test results you already obtained to determine what the next development step should be
4. Review the test output to identify any failing tests or areas that need improvement
5. If all tests are passing, proceed to verify your implementation meets the original requirements before considering
↪  the task complete

==================================================
INSTRUCTIONS
==================================================

1. Focus on errors that can be corrected through guidance
2. Provide specific, actionable recovery instructions
3. Be concise but precise in evidence citations
4. Only mark "DETECTED: Yes" if you have clear evidence
```

Agent	Opus Prompt	Resolved %	Localized %	Non-Empty %	Non-Loop %	Avg. Steps	Total Cost
`Qwen3-32B`	High-Level	26.8	67.8	82.4	92.4	25.0	$0.267
	Detailed	**40.6**	**72.6**	**85.0**	**93.8**	24.9	$0.337
`Qwen3-Next-80B-A3B`	High-Level	38.2	81.4	100.0	**99.4**	17.0	$0.172
	Detailed	**48.0**	**85.0**	100.0	98.6	18.1	$0.241

Table B.1: `Claude-Opus-4.6` teacher prompt comparison on SWE-Bench Verified (500 instances). *Non-Loop* is the percentage of trajectories without three consecutive repetitions of the same Bash command. Total cost is per-instance agent-plus-critic cost. Bold marks the better percentage metric within each agent; ties are unbolded.

```
5. Prioritize errors that most threaten task completion

Now review the agent's trajectory and provide your supervisor feedback.
```

B.4.3 Budget-Aware Instructions

After the agent passes step 100 of its 150-step budget, the following budget-aware instructions are appended to whichever critic prompt is in use:

```
NOTE: The agent has used {{current_step}} of its {{step_limit}} total steps. Work that is not submitted before the
↪ step limit will be permanently lost. If the agent has made code changes that address the issue, prioritize
↪ recommending submission. If the agent appears stuck in a loop, recommend that it submit its current changes
↪ immediately rather than continuing to iterate.
```

B.5 Opus Teacher Prompt Ablation

Table B.1 compares detailed and high-level feedback from the `Claude-Opus-4.6` teacher on the two Qwen agents.

B.6 Inference Pricing

All per-instance dollar costs reported in this paper are computed from input/output token counts using the public hosted-inference rates in Table B.2. Qwen-family rates are taken from PricePerToken. `CWM-32B` is not publicly hosted, so we use `Qwen3-32B` as a proxy on the basis that both are dense 32B models. The same `Qwen3-8B` rate is applied to the untrained `Qwen3-8B` critic and to all `Qwen3-8B` SFT variants distilled from it.

Model	Input ($/Mtok)	Output ($/Mtok)	Source
`CWM-32B`	0.080	0.280	Proxy: `Qwen3-32B` (same-size 32B dense)
`Qwen3-32B`	0.080	0.280	PricePerToken[2]
`Qwen3-Next-80B-A3B-Instruct`	0.090	0.780	PricePerToken[3]
`Qwen3-8B` and SFT variants	0.050	0.200	PricePerToken[4]
`Claude-Opus-4.6`	5.500	27.500	API pricing

Table B.2: Hosted-inference rates used to convert token counts to the dollar costs in this paper. Opus critic costs include prompt-caching: cache-write tokens billed at $1.25\times$ input ($6.875/Mtok) and cache-read at $0.10\times$ input ($0.55/Mtok); the reported Opus cost combines these.

Agent Model	Critic	Base (%)	+ Critic (%)	Δ 95% CI	McNemar p
`Qwen3-32B`	`Qwen3-4B` + SFT + DPO	8.8	**14.4 (+5.6)**	$[+2.8, +8.4]$	$1.3\times10^{-4***}$
`Qwen3-Next-80B-A3B`	`Qwen3-4B` + SFT + DPO	20.0	**26.2 (+6.2)**	$[+3.2, +9.2]$	$6.5\times10^{-5***}$
`GPT-OSS-20B`	`Qwen3-4B` + SFT + DPO	3.0	**14.8 (+11.8)**	$[+8.6, +15.0]$	$1.0\times10^{-12***}$
`GLM-4.7-Flash-30B-A3B`	`Qwen3-8B` + SFT	21.6	**37.6 (+16.0)**	$[+12.1, +19.9]$	$8.6\times10^{-15***}$
`GPT-OSS-120B` (medium reasoning)	`Qwen3-4B` + SFT + DPO	20.4	**34.8 (+14.4)**	$[+10.6, +18.2]$	$1.7\times10^{-13***}$
`o3-mini`	`Qwen3-8B` + SFT	19.0	**29.4 (+10.4)**	$[+7.0, +13.8]$	$3.1\times10^{-9***}$

Table B.3: Statistical significance of the headline resolve-rate gains in Table 8.1 on SWE-Bench Verified. Parentheses show percentage-point gains over no critic; 95% CIs are paired McNemar–Wald intervals on these gains. All critic runs use $k = 5$. Significance markers: $^{***}p < .001$, $^{**}p < .01$, and $^{*}p < .05$.

B.7 Statistical Significance

All six headline gains are statistically significant under a two-sided exact McNemar test on paired outcomes from the same 500 instances, and their paired 95% confidence intervals exclude zero (Table B.3; $p < .001$).

B.8 Qualitative Case Studies

Verbatim critic excerpts cited in §8.3.5; ellipses mark omitted rubric boilerplate (§B.4).

B.8.1 Untrained vs. trained critic on the same loop

On `django__django-16595` the untrained critic flags only Verification Failures and prescribes a test the agent has not set up; the agent abandons self-correction and submits a corrupted file (Listing B.1). On `django__django-9296`, under matched surface conditions (`sed` edits to a Django source file), the trained critic flags Step Repetition and Tool Selection Errors jointly and prescribes a heredoc rewrite (Listing B.2); the agent submits a correct patch ten steps later.

Listing B.1: Untrained critic at agent step 10 on `django__django-16595`. Only Verification Failures is flagged, with a recovery prescribing a test the agent has not set up.

```
... 12. Verification Failures: DETECTED: Yes
EVIDENCE: "The agent made changes to the reduce method in
django/db/migrations/operations/fields.py without verifying
if the fix works"
RECOVERY_ACTION: "Run the test script to verify that the fix
resolves the issue with multiple AlterField operations not
being reduced"

TASK_STATUS: Needs correction
OVERALL_GUIDANCE: You have modified the reduce method in the
AlterField class ... Run the test script to ensure that the
optimizer now correctly reduces multiple AlterField operations
... If the tests pass, you can submit your changes.
```

Listing B.2: Trained critic at agent step 10 on `django__django-9296`. Flags Step Repetition and Tool Selection Errors jointly and tells the agent to switch off `sed` append.

```
3. Step Repetition: DETECTED: Yes
EVIDENCE: Agent has repeatedly applied the same sed append
command, each time creating duplicate `__iter__` methods
inside the `Paginator` class.
RECOVERY_ACTION: Stop using sed append commands and instead
rewrite the file using a heredoc or a different approach ...
6. Tool Selection Errors: DETECTED: Yes
EVIDENCE: Agent is using `sed -i '/def page_range(self):/a'`
... but this creates multiple duplicate method definitions
each time it's run because the pattern matches multiple
times after the first insertion.
RECOVERY_ACTION: Use a different editing strategy: write the
entire file using a heredoc, or use a Python script ...

TASK_STATUS: Critical intervention required
OVERALL_GUIDANCE: The file is now corrupted with multiple
duplicate `__iter__` method definitions. You must view the
file ... and then use a Python script (not sed append) to
properly insert a single `__iter__` method ...
```

B.8.2 Routing around a dependency failure

On `sympy__sympy-22714` the unguided agent loops on `ModuleNotFoundError: mpmath`; the trained critic at step 20 identifies the import as the root cause and tells the agent to bypass it (Listing B.3), and the agent submits a correct patch at step 25.

Listing B.3: Trained critic at agent step 20 on `sympy__sympy-22714`.

```
TASK_STATUS: Needs correction
OVERALL_GUIDANCE: The mpmath import in sympy/__init__.py is
preventing any import of sympy. You need to bypass this
import to test, or focus on reading the source code directly
to understand the issue without running code. The key issue
is in `sympy/core/sympify.py` line 472 where `sympify` calls
`parse_expr` with `evaluate=evaluate` ...
```

B.8.3 Backseat driving in the detailed teacher prompt

The detailed prompt at the `Claude-Opus-4.6` teacher mixes patch dictation and architectural prescription in the same firing slot. Listing B.4 is the dictation case on `django__django-12193` (ρ=1.00, resolved): `Claude-Opus-4.6` supplies the before/after blocks for `CheckboxInput.get_context` and the agent transcribes them into a patch over its next three messages. Listing B.5 is the wrong-fork case on `sphinx-doc__sphinx-7590` (ρ=0.70, gold overlap 0.12, fails): `Claude-Opus-4.6` hedges between modifying the existing parser path and introducing a new AST class, the agent commits to the latter and writes 46 lines around an `ASTUserDefinedLiteral` class that the gold patch never introduces.

Listing B.4: `Claude-Opus-4.6` teacher (detailed prompt) at agent step 6 on `django__django-12193`. The agent's submitted patch reproduces both blocks verbatim.

````
OVERALL_GUIDANCE: The agent has correctly identified the
root cause in `CheckboxInput.get_context()` ... The fix
should be to make a copy of `attrs` before modifying it.
Specifically, change the `get_context` method of
`CheckboxInput` from:

```python
def get_context(self, name, value, attrs):
    if self.check_test(value):
        if attrs is None:
            attrs = {}
        attrs['checked'] = True
````
````

```
    return super().get_context(name, value, attrs)
```

to:

```python
def get_context(self, name, value, attrs):
    if self.check_test(value):
        attrs = {**(attrs or {}), 'checked': True}
    return super().get_context(name, value, attrs)
```

Listing B.5: `Claude-Opus-4.6` teacher (detailed prompt) at agent step 6 on `sphinx-doc__sphinx-7590`. Bullet 4's "or if a new … class is needed" is the wrong fork; the agent takes it.

```
3. Implementation approach: After matching a numeric
literal and consuming any standard suffixes (`uUlLfF`),
also try to match an identifier suffix (the ud-suffix)...

4. Check the AST classes - look at `ASTNumberLiteral`,
`ASTStringLiteral`, `ASTCharLiteral` to see if they need
modification, or if a new `ASTUserDefinedLiteral` class
wrapping them is needed.
```

B.8.4 Repeated demand on `django__django-13012`

Listing B.6 shows two of the 13 firings on `django__django-13012` to illustrate the structural similarity of the `OVERALL_GUIDANCE` that the agent stops acting on.

Listing B.6: Trained critic firings 6 (msg 127) and 10 (msg 211) on `django__django-13012`; both prescribe the same fix path.

```
[firing 6, agent step 60]
TASK_STATUS: Critical intervention required
OVERALL_GUIDANCE: You are stuck in a loop viewing the same
code. The `ExpressionWrapper.get_group_by_cols` calls
`self.get_source_expressions()` which returns
`[self.expression]`, and then calls `source.get_group_by_cols()`
on that expression. When `self.expression` is a `Value`,
`Value.get_group_by_cols` returns `[]`, so the constant
should NOT be in GROUP BY. The real issue is likely in
```

`query.py`'s `set_group_by` method ...

[firing 10, agent step 100]
TASK_STATUS: Critical intervention required
OVERALL_GUIDANCE: You are stuck in a loop viewing the same code. Create a minimal reproduction script that only imports `Value`, `IntegerField`, and `Sum` from `django.db.models` (no `functions` or `asgiref`), then look at `set_group_by` in `query.py` (line 1942) to understand how the GROUP BY clause is populated ...

Bibliography

[1] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Unified pre-training for program understanding and generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021. URL https://www.aclweb.org/anthology/2021.naacl-main.211.

[2] Wasi Uddin Ahmad, Md Golam Rahman Tushar, Saikat Chakraborty, and Kai-Wei Chang. Avatar: A parallel corpus for java-python program translation. *arXiv preprint arXiv:2108.11590*, 2021.

[3] Anthropic. Claude-3.5, 2024. URL https://www.anthropic.com/news/claude-3-5-sonnet.

[4] Anthropic. Claude-3.7, 2025. URL https://www.anthropic.com/news/claude-3-7-sonnet.

[5] Anthropic. Claude-4.5, 2025. URL https://www.anthropic.com/news/claude-4-5-sonnet.

[6] Anthropic. Claude-4.6, 2026. URL https://www.anthropic.com/news/claude-sonnet-4-6.

[7] Anthropic. Claude-4.7, 2026. URL https://www.anthropic.com/news/claude-sonnet-4-7.

[8] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Yang Wang. Swe-search: Enhancing software agents with monte carlo tree search and iterative refinement. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=G7sIFXugTX.

[9] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In Jun'ichi Tsujii, James Henderson, and Marius

Paşca, editors, *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 995–1005, Jeju Island, Korea, July 2012. Association for Computational Linguistics. URL https://aclanthology.org/D12-1091/.

[10] Ben Bogin, Kejuan Yang, Shashank Gupta, Kyle Richardson, Erin Bransom, Peter Clark, Ashish Sabharwal, and Tushar Khot. SUPER: Evaluating agents on setting up and executing tasks from research repositories. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 12622–12645, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.702. URL https://aclanthology.org/2024.emnlp-main.702/.

[11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[12] Valerie Chen, Rohit Malhotra, Xingyao Wang, Juan Michelini, Xuhui Zhou, Aditya Bharat Soni, Hoang H. Tran, Calvin Smith, Ameet Talwalkar, and Graham Neubig. How can we assess human-agent interactions? case studies in software agent design, 2025. URL https://arxiv.org/abs/2510.09801.

[13] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors, 2023. URL https://arxiv.org/abs/2308.10848.

[14] Zehui Chen, Kuikun Liu, Qiuchen Wang, Wenwei Zhang, Jiangning Liu, Dahua Lin, Kai Chen, and Feng Zhao. Agent-FLAN: Designing data and methods of effective agent

tuning for large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 9354–9366, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.557. URL https://aclanthology.org/2024.findings-acl.557/.

[15] Yung-Sung Chuang, Yujia Xie, Hongyin Luo, Yoon Kim, James R. Glass, and Pengcheng He. Dola: Decoding by contrasting layers improves factuality in large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=Th6NyL07na.

[16] DeepSeek-AI and Xu et al. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL https://arxiv.org/abs/2606.19348.

[17] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, et al. Deepseek-v3 technical report, 2024. URL https://arxiv.org/abs/2412.19437.

[18] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can AI agents solve long-horizon software engineering tasks? *CoRR*, abs/2509.16941, 2025. doi: 10.48550/ARXIV.2509.16941. URL https://doi.org/10.48550/arXiv.2509.16941.

[19] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation, 2023.

[20] Nouha Dziri, Ehsan Kamalloo, Sivan Milton, Osmar Zaiane, Mo Yu, Edoardo M. Ponti, and Siva Reddy. FaithDial: A faithful benchmark for information-seeking dialogue. *Transactions of the Association for Computational Linguistics*, 10, 2022. doi: 10.1162/tacl_a_00529. URL https://aclanthology.org/2022.tacl-1.84.

[21] Nouha Dziri, Hannah Rashkin, Tal Linzen, and David Reitter. Evaluating attribution in dialogue systems: The BEGIN benchmark. *Transactions of the Association for Computational Linguistics*, 10, 2022. doi: 10.1162/tacl_a_00506. URL https://aclanthology.org/2022.tacl-1.62.

[22] Aleksandra Eliseeva, Alexander Kovrigin, Ilia Kholkin, Egor Bogomolov, and Yaroslav

Zharov. Envbench: A benchmark for automated environment setup, 2025. URL https://arxiv.org/abs/2503.14443.

[23] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.139. URL https://aclanthology.org/2020.findings-emnlp.139.

[24] Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars, 2025. URL https://arxiv.org/abs/2503.01307.

[25] Shubham Gandhi, Atharva Naik, Yiqing Xie, and Carolyn Rose. An empirical study on strong-weak model collaboration for repo-level code generation. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20667–20686, Suzhou, China, November 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.1043. URL https://aclanthology.org/2025.emnlp-main.1043/.

[26] Shubham Gandhi, Jason Tsay, Jatin Ganhotra, Kiran Kate, and Yara Rizk. When agents go astray: Course-correcting swe agents with prms, 2025. URL https://arxiv.org/abs/2509.02360.

[27] Zorik Gekhman, Gal Yona, Roee Aharoni, Matan Eyal, Amir Feder, Roi Reichart, and Jonathan Herzig. Does fine-tuning llms on new knowledge encourage hallucinations?, 2024. URL https://arxiv.org/abs/2405.05904.

[28] Harritxu Gete and Thierry Etchegoyhen. Making the most of comparable corpora in neural machine translation: a case study. *Language Resources and Evaluation*, 56(3):943–971, Sep 2022. ISSN 1574-0218. doi: 10.1007/s10579-021-09572-2. URL https://doi.org/10.1007/s10579-021-09572-2.

[29] Gaurav Rohit Ghosal, Tatsunori Hashimoto, and Aditi Raghunathan. Understanding finetuning for factual knowledge extraction. In *Forty-first International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=cPsn9AcOYh.

[30] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. Critic: Large language models can self-correct with tool-interactive cri-

tiquing. *arXiv preprint arXiv:2305.11738*, 2023.

[31] Xinyi He, Qian Liu, Mingzhe Du, Lin Yan, Zhijie Fan, Yiming Huang, Zejian Yuan, and Zejun Ma. Swe-perf: Can language models optimize code performance on real-world repositories?, 2025. URL https://arxiv.org/abs/2507.12415.

[32] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024. URL https://arxiv.org/abs/2308.00352.

[33] Or Honovich, Leshem Choshen, Roee Aharoni, Ella Neeman, Idan Szpektor, and Omri Abend. q^2: Evaluating factual consistency in knowledge-grounded dialogues via question generation and question answering. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.619. URL https://aclanthology.org/2021.emnlp-main.619.

[34] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report, 2024. URL https://arxiv.org/abs/2409.12186.

[35] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search, 2020.

[36] Ann Irvine. Using comparable corpora to augment statistical machine translation models in low resource settings, 2014.

[37] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. Mapcoder: Multi-agent code generation for competitive problem solving, 2024. URL https://arxiv.org/abs/2405.11403.

[38] Kush Jain, Gabriel Synnaeve, and Baptiste Roziere. Testgeneval: A real world unit test generation and test completion benchmark. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=7o6SG5gVev.

[39] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang,

Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

[40] Naman Jain, Manish Shetty, Tianjun Zhang, King Han, Koushik Sen, and Ion Stoica. R2E: Turning any github repository into a programming agent environment. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 21196–21224. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/jain24c.html.

[41] Naman Jain, Jaskirat Singh, Manish Shetty, Tianjun Zhang, Liang Zheng, Koushik Sen, and Ion Stoica. R2e-gym: Procedural environment generation and hybrid verifiers for scaling open-weights SWE agents. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=7evvwwdo3z.

[42] Nan Jiang, Xiaopeng Li, Shiqi Wang, Qiang Zhou, Soneya Binta Hossain, Baishakhi Ray, Varun Kumar, Xiaofei Ma, and Anoop Deoras. Ledex: Training LLMs to better self-debug and explain code. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=d1XrZ4EINV.

[43] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-Bench: Can language models resolve real-world github issues? In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/edac78c3e300629acfe6cbe9ca88fb84-Paper-Conference.pdf.

[44] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[45] Jiahe Jin, Abhijay Paladugu, and Chenyan Xiong. Beneficial reasoning behaviors in agentic search and effective post-training to obtain them, 2026. URL https://arxiv.org/abs/2510.06534.

[46] Haoqiang Kang, Juntong Ni, and Huaxiu Yao. Ever: Mitigating hallucination in large language models through real-time verification and rectification, 2024. URL https://arxiv.org/abs/2311.09114.

[47] Katie Kang, Eric Wallace, Claire Tomlin, Aviral Kumar, and Sergey Levine. Unfamiliar finetuning examples control how language models hallucinate, 2024. URL https://arxiv.org/abs/2403.05612.

[48] Mohammad Abdullah Matin Khan, M Saiful Bari, Xuan Long Do, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. xcodeeval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval, 2023.

[49] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. Prometheus: Inducing fine-grained evaluation capability in language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=8euJaTveKw.

[50] Marieanne Lachaux, Baptiste Roziere, Marc Szafraniec, and Guillaume Lample. DOBF: A deobfuscation pre-training objective for programming languages. In *Advances in Neural Information Processing Systems*, 2021.

[51] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, 2020.

[52] Dacheng Li, Shiyi Cao, Tyler Griggs, Shu Liu, Xiangxi Mo, Eric Tang, Sumanth Hegde, Kourosh Hakhamaneshi, Shishir G. Patil, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Llms can easily learn to reason from demonstrations structure, not content, is what matters!, 2025. URL https://arxiv.org/abs/2502.07374.

[53] Jia Li, Ge Li, Xuanming Zhang, Yunfei Zhao, Yihong Dong, Zhi Jin, Binhua Li, Fei Huang, and Yongbin Li. Evocodebench: An evolving code generation benchmark with domain-specific evaluations. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=kvjbFVHpny.

[54] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Zhi Jin, Hao Zhu, Huanyu Liu, Kaibo Liu, Lecheng Wang, Zheng Fang, Lanshen Wang, Jiazheng Ding, Xuanming Zhang, Yihong Dong, Yuqi Zhu, Bin Gu, and Mengfei Yang. Deveval: Evaluating code generation in practical software projects, 2024.

[55] Junlong Li, Shichao Sun, Weizhe Yuan, Run-Ze Fan, hai zhao, and Pengfei Liu. Gener-

ative judge for evaluating alignment. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=gtkFw6sZGS.

[56] Junyi Li, Jie Chen, Ruiyang Ren, Xiaoxue Cheng, Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. The dawn after the dark: An empirical study on factuality hallucination in large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), pages 10879–10899, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL https://aclanthology.org/2024.acl-long.586.

[57] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: eliciting truthful answers from a language model. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, 2024.

[58] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 39578–39601, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/aca97732e30bcf1303bc22ac3924fd16-Paper-Conference.pdf.

[59] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), pages 3214–3252, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.229. URL https://aclanthology.org/2022.acl-long.229/.

[60] Jiate Liu, Yiqin Zhu, Kaiwen Xiao, QIANG FU, Xiao Han, Yang Wei, and Deheng Ye. RLTF: Reinforcement learning from unit test feedback. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL https://openreview.net/forum?id=hjYmsV6nXZ.

[61] Simiao Liu, Fang Liu, Liehao Li, Xin Tan, Yinghao Zhu, Xiaoli Lian, and Li Zhang. An empirical study on failures in automated issue solving, 2025. URL https://arxiv.org/abs/2509.13941.

[62] Zexi Liu, Jingyi Chai, Xinyu Zhu, Shuo Tang, Rui Ye, Bo Zhang, Lei Bai, and Siheng Chen. Ml-agent: Reinforcing llm agents for autonomous machine learning engineering. *arXiv preprint arXiv: 2505.23723*, 2025.

[63] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. Starcoder 2 and the stack v2: The next generation, 2024. URL `https://arxiv.org/abs/2402.19173`.

[64] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, MING GONG, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie LIU. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track* (*Round 1*), 2021.

[65] Jeffrey Jian Ma, Milad Hashemi, Amir Yazdanbakhsh, Kevin Swersky, Ofir Press, Enhui Li, Vijay Janapa Reddi, and Parthasarathy Ranganathan. Swe-fficiency: Can language models optimize real-world repositories on real workloads?, 2025. URL `https://arxiv.org/abs/2511.06090`.

[66] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan Thomas Mcdonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919, Online, 2020.

[67] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S.

Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868.

[68] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.741. URL https://aclanthology.org/2023.emnlp-main.741.

[69] MiniMax. Minimax-m2.5: A state-of-the-art model for agents and coding. https://huggingface.co/MiniMaxAI/MiniMax-M2.5, 2026.

[70] Abhika Mishra, Akari Asai, Vidhisha Balachandran, Yizhong Wang, Graham Neubig, Yulia Tsvetkov, and Hannaneh Hajishirzi. Fine-grained hallucination detection and editing for language models. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=dJMTn3QOWO.

[71] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. Swt-bench: testing and validating real-world bug-fixes with code agents. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, 2024.

[72] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. Next: teaching large language models to reason about code execution. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[73] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Compu-*

tational Linguistics (*Volume 1: Long Papers*). Association for Computational Linguistics, 2024.

[74] OpenAI. Gpt-4.5, 2025. URL https://openai.com/index/introducing-gpt-4-5.

[75] OpenAI. Gpt5, 2025. URL https://platform.openai.com/docs/models/gpt-5.

[76] OpenAI. Openai o3-mini, 2025. URL https://openai.com/index/openai-o3-mini/.

[77] OpenAI. Introducing gpt-oss, 2025. URL https://openai.com/index/introducing-gpt-oss/.

[78] OpenAI. Openai o3-mini, 2025. URL https://openai.com/index/openai-o3-mini/.

[79] OpenAI. Introducing gpt-5.3-codex: The most capable agentic coding model to date, February 2026. URL https://openai.com/index/gpt-5-3-codex-system-card/. System Card.

[80] OpenAI. Gpt5.5, 2026. URL https://platform.openai.com/docs/models/gpt-5-5.

[81] Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Re, and Azalia Mirhoseini. Kernelbench: Can LLMs write efficient GPU kernels? In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=yeoN1iQT1x.

[82] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. Repograph: Enhancing AI software engineering with repository-level code graph. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=dw9VUsSHGB.

[83] Artidoro Pagnoni, Vidhisha Balachandran, and Yulia Tsvetkov. Understanding factuality in abstractive summarization with FRANK: A benchmark for factuality metrics. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4812–4829, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.383. URL https://aclanthology.org/2021.naacl-main.383.

[84] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym, 2024. URL https://arxiv.org/abs/2412.21139.

[85] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-gym. In

Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47717–47737. PMLR, 13–19 Jul 2025. URL https://proceedings.mlr.press/v267/pan25g.html.

[86] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ACL '02. Association for Computational Linguistics, 2002. URL https://doi.org/10.3115/1073083.1073135.

[87] Huy Nhat Phan, Tien N. Nguyen, Phong X. Nguyen, and Nghi D. Q. Bui. Hyperagent: Generalist software engineering agents to solve coding tasks at scale, 2025. URL https://arxiv.org/abs/2409.16299.

[88] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks, 2021.

[89] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development, 2024. URL https://arxiv.org/abs/2307.07924.

[90] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023. URL https://arxiv.org/abs/2305.18290.

[91] Abhinav Rastogi, Adam Yang, Albert Q. Jiang, Alexander H. Liu, Alexandre Sablayrolles, Amélie Héliou, Amélie Martin, Anmol Agarwal, Andy Ehrenberg, Andy Lo, Antoine Roux, Arthur Darcet, Arthur Mensch, Baptiste Bout, Baptiste Rozière, Baudouin De Monicault, Chris Bamford, Christian Wallenwein, Christophe Renaudin, Clémence Lanfranchi, Clément Denoix, Corentin Barreau, Darius Dabert Devon Mizelle, Diego de las Casas, Elliot Chane-Sane, Emilien Fugier, Emma Bou Hanna, Gabrielle Berrada, Gauthier Delerce, Gauthier Guinet, Georgii Novikov, Graham Neubig, Guillaume Lample, Guillaume Martin, Himanshu Jaju, Jan Ludziejewski, Jason Rute, Jean-Malo Delignon, JeanHadrien Chabran, Joachim Studnia, Joep Barmentlo, Jonas Amar, Josselin Somerville Roberts, Julien Denize, Karan Saxena, Karmesh Yadav, Kartik Khan-

delwal, Khyathi Raghavi Chandu, Kush Jain, Lélio Renard Lavaud, Léonard Blier, Lingxiao Zhao, Louis Martin, Lucile Saulnier, Luyu Gao, Marie Pellat, Mathilde Guillaumin, Mathis Felardos, Matthieu Dinot, Maxime Darrin, Maximilian Augustin, Mickaël Seznec, Neha Gupta, Nikhil Raghuraman, Olivier Duchenne, Patricia Wang, Patrick von Platen, Patryk Saffer, Paul Jacob, Paul Wambergue, Paula Kurylowicz, Philomène Chagniot, Pierre Stock, Pravesh Agrawal, Rémi Delacourt, Roman Soletskyi, Romain Sauvestre, Sagar Vaze, Sanchit Gandhi, Sandeep Subramanian, Shashwat Dalal, Siddharth Gandhi, Soham Ghosh, Srijan Mishra, Sumukh Aithal, Szymon Antoniak, Teven Le Scao, Thibaut Lavril, Thibault Schueller, Thomas Foubert, Thomas Robert, Thomas Wang, Timothée Lacroix, Tom Bewley, Valeriia Nemychnikova, Victor Paltz, Virgile Richard, Wen-Ding Li, William Marshall, Xingyao Wang, Xuanyu Zhang, Yihan Wan, and Yunhao Tang. Devstral: Fine-tuning language models for coding agent applications, 2025. URL `https://arxiv.org/abs/2509.25193`.

[92] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. Codebleu: a method for automatic evaluation of code synthesis, 2020.

[93] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, 2020.

[94] Baptiste Roziere, Jie M. Zhang, Francois Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. Leveraging automated unit tests for unsupervised code translation, 2022.

[95] Yuling Shi, Songsong Wang, Chengcheng Wan, Min Wang, and Xiaodong Gu. From code to correctness: Closing the last mile of code generation with hierarchical debugging, 2025. URL `https://arxiv.org/abs/2410.01215`.

[96] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 2023.

[97] Huatong Song, Lisheng Huang, Shuang Sun, Jinhao Jiang, Ran Le, Daixuan Cheng, Guoxin Chen, Yiwen Hu, Zongchao Chen, Yiming Jia, Wayne Xin Zhao, Yang Song, Tao Zhang, and Ji-Rong Wen. Swe-master: Unleashing the potential of software engineering agents via post-training, 2026. URL `https://arxiv.org/abs/2602.03411`.

[98] Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, Yang Song, Hongteng Xu, Tao Zhang,

and Ji-Rong Wen. Swe-world: Building software engineering agents in docker-free environments, 2026. URL https://arxiv.org/abs/2602.03419.

[99] Liyan Tang, Philippe Laban, and Greg Durrett. Minicheck: Efficient fact-checking of llms on grounding documents, 2024. URL https://arxiv.org/abs/2404.10774.

[100] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang, Feng Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Huabin Zheng, Shaojie Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2025. URL https://arxiv.org/abs/2507.20534.

[101] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, Yanru Chen, Yanxu Chen, Yicun Chen, Yimin Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen,

Yutian Chen, Zhirong Chen, Ziwei Chen, Dazhi Cheng, Minghan Chu, Jialei Cui, Jiaqi Deng, Muxi Diao, Hao Ding, Mengfan Dong, Mengnan Dong, Yuxin Dong, Yuhao Dong, Angang Du, Chenzhuang Du, Dikang Du, Lingxiao Du, Yulun Du, Yu Fan, Shengjun Fang, Qiulin Feng, Yichen Feng, Garimugai Fu, Kelin Fu, Hongcheng Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Chengyang Gong, Xiaochen Gong, Zhuoma Gongque, Qizheng Gu, Xinran Gu, Yicheng Gu, Longyu Guan, Yuanying Guo, Xiaoru Hao, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Jiaxi Hu, Yangyang Hu, Zhenxing Hu, Ke Huang, Ruiyuan Huang, Weixiao Huang, Zhiqi Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yu Jing, Guokun Lai, Aidi Li, C. Li, Cheng Li, Fang Li, Guanghe Li, Guanyu Li, Haitao Li, Haoyang Li, Jia Li, Jingwei Li, Junxiong Li, Lincan Li, Mo Li, Weihong Li, Wentao Li, Xinhang Li, Xinhao Li, Yang Li, Yanhao Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zheming Li, Weilong Liao, Jiawei Lin, Xiaohan Lin, Zhishan Lin, Zichao Lin, Cheng Liu, Chenyu Liu, Hongzhang Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Tianyu Liu, Weizhou Liu, Xiangyan Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yuanxin Liu, Yue Liu, Zhengying Liu, Zhongnuo Liu, Enzhe Lu, Haoyu Lu, Zhiyuan Lu, Junyu Luo, Tongxu Luo, Yashuo Luo, Long Ma, Yingwei Ma, Shaoguang Mao, Yuan Mei, Xin Men, Fanqing Meng, Zhiyong Meng, Yibo Miao, Minqing Ni, Kun Ouyang, Siyuan Pan, Bo Pang, Yuchao Qian, Ruoyu Qin, Zeyu Qin, Jiezhong Qiu, Bowen Qu, Zeyu Shang, Youbo Shao, Tianxiao Shen, Zhennan Shen, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Feifan Song, Pengwei Song, Tianhui Song, Xiaoxi Song, Hongjin Su, Jianlin Su, Zhaochen Su, Lin Sui, Jinsong Sun, Junyao Sun, Tongyu Sun, Flood Sung, Yunpeng Tai, Chuning Tang, Heyi Tang, Xiaojuan Tang, Zhengyang Tang, Jiawen Tao, Shiyuan Teng, Chaoran Tian, Pengfei Tian, Ao Wang, Bowen Wang, Chensi Wang, Chuang Wang, Congcong Wang, Dingkun Wang, Dinglu Wang, Dongliang Wang, Feng Wang, Hailong Wang, Haiming Wang, Hengzhi Wang, Huaqing Wang, Hui Wang, Jiahao Wang, Jinhong Wang, Jiuzheng Wang, Kaixin Wang, Linian Wang, Qibin Wang, Shengjie Wang, Shuyi Wang, Si Wang, Wei Wang, Xiaochen Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yipu Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhexu Wang, Zihan Wang, Zizhe Wang, Chu Wei, Ming Wei, Chuan Wen, Zichen Wen, Chengjie Wu, Haoning Wu, Junyan Wu, Rucong Wu, Wenhao Wu, Yuefeng Wu, Yuhao Wu, Yuxin Wu, Zijian Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Yuchong Xie, Yifei Xin, Bowei Xing, Boyu Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinbo Xu, Xinran Xu, Yangchuan Xu, Yichang Xu, Yuemeng Xu, Zelai Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Guangyao Yang, Hao Yang, Junwei Yang, Kai Yang, Ningyuan Yang, Ruihan

Yang, Xiaofei Yang, Xinlong Yang, Ying Yang, Yi Yang, Yi Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Dan Ye, Wenjie Ye, Zhuorui Ye, Bohong Yin, Chengzhen Yu, Longhui Yu, Tao Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Xiaokun Yuan, Yang Yue, Weihao Zeng, Dunyuan Zha, Haobing Zhan, Dehao Zhang, Hao Zhang, Jin Zhang, Puqi Zhang, Qiao Zhang, Rui Zhang, Xiaobin Zhang, Y. Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yushun Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Chenguang Zhao, Feifan Zhao, Jinxiang Zhao, Shuai Zhao, Xiangyu Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Junfeng Zhong, Longguang Zhong, Weiming Zhong, M. Zhou, Runjie Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yuxuan Zhu, Zhen Zhu, Jingze Zhuang, Weiyu Zhuang, Ying Zou, and Xinxing Zu. Kimi k2.5: Visual agentic intelligence, 2026. URL https://arxiv.org/abs/2602.02276.

[102] Qwen Team. Qwen3.5-omni technical report, 2026. URL https://arxiv.org/abs/2604.15804.

[103] Katherine Tian, Eric Mitchell, Huaxiu Yao, Christopher D Manning, and Chelsea Finn. Fine-tuning language models for factuality. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=WPZ2yPag4K.

[104] Tu Vu, Kalpesh Krishna, Salaheddin Alzubi, Chris Tar, Manaal Faruqui, and Yun-Hsuan Sung. Foundational autoraters: Taming large language models for better automatic evaluation, 2024. URL https://arxiv.org/abs/2407.10817.

[105] Alex Wang, Kyunghyun Cho, and Mike Lewis. Asking and answering questions to evaluate the factual consistency of summaries. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5008–5020, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.450. URL https://aclanthology.org/2020.acl-main.450.

[106] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=OJd3ayDDoF.

[107] Xingyao Wang, Valerie Chen, Heng Ji, and Graham Neubig. A rubric-supervised critic

from sparse real-world outcomes, 2026. URL https://arxiv.org/abs/2603.03800.

[108] Xingyao Wang, Valerie Chen, Heng Ji, and Graham Neubig. A rubric-supervised critic from sparse real-world outcomes. *arXiv preprint arXiv:2603.03800*, 2026.

[109] Yanlin Wang, Yanli Wang, Daya Guo, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. Rlcoder: Reinforcement learning for repository-level code completion, 2024. URL https://arxiv.org/abs/2407.19487.

[110] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.

[111] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL https://arxiv.org/abs/2201.11903.

[112] Jerry Wei, Chengrun Yang, Xinying Song, Yifeng Lu, Nathan Hu, Jie Huang, Dustin Tran, Daiyi Peng, Ruibo Liu, Da Huang, Cosmo Du, and Quoc V. Le. Long-form factuality in large language models, 2024. URL https://arxiv.org/abs/2403.18802.

[113] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, LINGMING ZHANG, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=ULblO61XZ0.

[114] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. Repoformer: selective retrieval for repository-level code completion. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[115] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

[116] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying llm-based software engineering agents. *Proc. ACM Softw. Eng.*, 2025. URL https://doi.org/10.1145/3715754.

[117] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying llm-based software engineering agents. *Proc. ACM Softw. Eng.*, 2(FSE), June 2025. doi:

10.1145/3715754. URL https://doi.org/10.1145/3715754.

[118] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint*, 2025.

[119] Yiqing Xie, Atharva Naik, Daniel Fried, and Carolyn Rose. Data augmentation for code translation with comparable corpora and multiple references. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 13725–13739, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.917. URL https://aclanthology.org/2023.findings-emnlp.917/.

[120] Yiqing Xie, Alex Xie, Divyanshu Sheth, Pengfei Liu, Daniel Fried, and Carolyn Rose. RepoST: Scalable repository-level coding environment construction with sandbox testing. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=2txrMBpw3q.

[121] Yiqing Xie, Wenxuan Zhou, Pradyot Prakash, Di Jin, Yuning Mao, Quintin Fettes, Arya Talebzadeh, Sinong Wang, Han Fang, Carolyn Rose, Daniel Fried, and Hejia Zhang. Improving model factuality with fine-grained critique-based evaluator. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), Vienna, Austria, July 2025. Association for Computational Linguistics. URL https://aclanthology.org/2025.acl-long.400/.

[122] Yiqing Xie, Wenxuan Zhou, Pradyot Prakash, Di Jin, Yuning Mao, Quintin Fettes, Arya Talebzadeh, Sinong Wang, Han Fang, Carolyn Rose, Daniel Fried, and Hejia Zhang. Improving model factuality with fine-grained critique-based evaluator. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*). Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.acl-long.400/.

[123] Yiqing Xie, Emmy Liu, Gaokai Zhang, Nachiket Kotalwar, Shubham Gandhi, Sathwik Acharya, Xingyao Wang, Carolyn Rose, Graham Neubig, and Daniel Fried. Hybridgym: Training coding agents to generalize across tasks. In *Forty-third International Conference on Machine Learning*, 2026.

[124] Yiqing Xie, Emmy Liu, Gaokai Zhang, Nachiket Kotalwar, Shubham Gandhi, Sathwik Acharya, Xingyao Wang, Carolyn Rose, Graham Neubig, and Daniel Fried. Hybridgym: Training coding agents to generalize across tasks, 2026. URL https://arxiv.org/abs/2602.16819.

[125] Wenda Xu, Guanglei Zhu, Xuandong Zhao, Liangming Pan, Lei Li, and William Yang Wang. Pride and prejudice: Llm amplifies self-bias in self-refinement, 2024. URL https://arxiv.org/abs/2402.11436.

[126] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

[127] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: agent-computer interfaces enable automated software engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.

[128] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL https://arxiv.org/abs/2405.15793.

[129] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. Swe-bench multimodal: Do ai systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=riTiq3i21b.

[130] John Yang, Kilian Lieret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. URL https://openreview.net/forum?id=63iVrXc8cC.

[131] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents. In *Proceedings of the 39th Annual Conference on*

Neural Information Processing Systems (*NeurIPS 2025 D&B Spotlight*), 2025. URL https://arxiv.org/abs/2504.21798. arXiv:2504.21798, accepted at NeurIPS 2025 (Spotlight).

[132] Sidi Yang, Chaofan Tao, Jierun Chen, Tiezheng Yu, Ruoyu Wang, Yuxin Jiang, Yiming Du, Wendong Xu, Jing Xiong, Taiqiang Wu, Lifeng Shang, Xiaohui Li, Ngai Wong, and Haoli Bai. What makes interaction trajectories effective for training terminal agents?, 2026. URL https://arxiv.org/abs/2606.03461.

[133] Z.ai. Glm-5, 2026. URL https://huggingface.co/zai-org/GLM-5.

[134] Z.AI. Glm-4.7-flash model card, 2026. URL https://huggingface.co/zai-org/GLM-4.7-Flash.

[135] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-swe-bench: A multilingual benchmark for issue resolving, 2025. URL https://arxiv.org/abs/2504.02605.

[136] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-swe-bench: A multilingual benchmark for issue resolving, 2025. URL https://arxiv.org/abs/2504.02605.

[137] Chenchen Zhang, Yuhang Li, Can Xu, Jiaheng Liu, Ao Liu, Changzhi Zhou, Ken Deng, Dengpeng Wu, Guanhua Huang, Kejiao Li, Qi Yi, Ruibin Xiong, Shihui Hu, Yue Zhang, Yuhao Jiang, Zenan Xu, Yuanxing Zhang, Wiggin Zhou, Chayse Zhou, and Fengzong Lian. Artifactsbench: Bridging the visual-interactive gap in llm code generation evaluation, 2025. URL https://arxiv.org/abs/2507.04952.

[138] Dylan Zhang, Shizhe Diao, Xueyan Zou, and Hao Peng. **PLUM**: Improving code lms with execution-guided on-policy preference learning driven by synthetic test cases, 2024. URL https://arxiv.org/abs/2406.06887.

[139] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023. URL https://aclanthology.org/2023.emnlp-main.151.

[140] Xiaoying Zhang, Baolin Peng, Ye Tian, Jingyan Zhou, Lifeng Jin, Linfeng Song, Haitao

Mi, and Helen Meng. Self-alignment for factuality: Mitigating hallucinations in LLMs via self-evaluation. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.107. URL https://aclanthology.org/2024.acl-long.107/.

[141] Yakun Zhang, Wenjie Zhang, Dezhi Ran, Qihao Zhu, Chengfeng Dou, Dan Hao, Tao Xie, and Lu Zhang. Learning-based widget matching for migrating gui test cases. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, ICSE '24. ACM, February 2024. URL http://dx.doi.org/10.1145/3597503.3623322.

[142] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. Siren's song in the ai ocean: A survey on hallucination in large language models, 2023. URL https://arxiv.org/abs/2309.01219.

[143] Wenting Zhao, Nan Jiang, Celine Lee, Justin T Chiu, Claire Cardie, Matthias Gallé, and Alexander M Rush. Commit0: Library generation from scratch. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=MMwaQEVsAg.

[144] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL https://arxiv.org/abs/2306.05685.

[145] Yiqi Zhu, Apurva Gandhi, and Graham Neubig. Training versatile coding agents in synthetic environments, 2025. URL https://arxiv.org/abs/2512.12216.